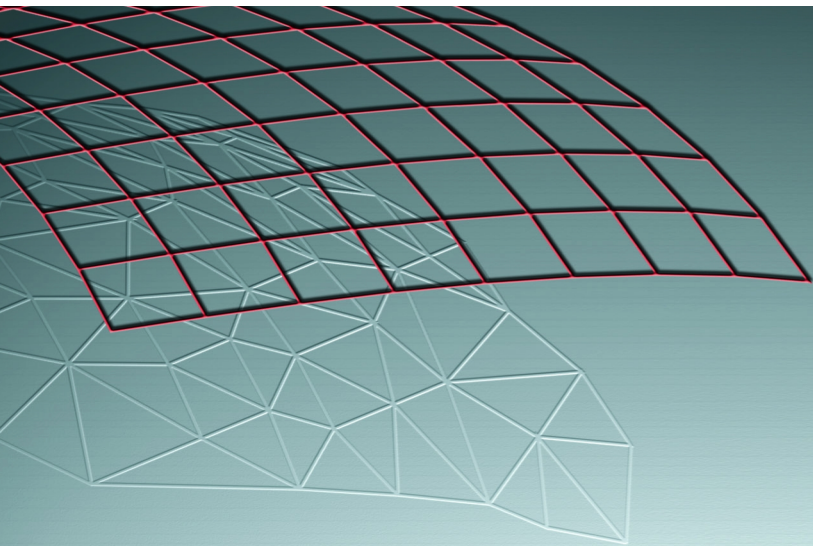




MpCCI 3.0

Help-Documents

Release Notes

Overview

MpCCI Technical Reference

Manuals

Installation

Graphical User Interface

Visualiser

Technical Reference

Tutorials

Setup of Coupling Cases

Software Development Kit



MpCCI 3.0.3
Manuals and Tutorials

December 16, 2004

MpCCI is a registered trademark of Fraunhofer SCAI

Fraunhofer Institute for Algorithms and Scientific Computing SCAI
Schloss Birlinghoven, 53754 Sankt Augustin, GERMANY
<http://www.scai.fraunhofer.de/mpcci>

ABAQUS is a registered trademark of ABAQUS Inc.
ANSYS is a registered trademark of Ansys Inc.
FLUENT is a registered trademark of Fluent Inc.
STAR-CD is a registered trademark of CD adapco Group
FLEXlm is a registered trademark of Macrovision

Contents

1	Introduction	8
1.1	The Intention of Developing MpCCI	8
1.2	Potential Users of MpCCI	8
1.3	MpCCI Functionality	8
1.4	Environment	9
1.5	Programming Languages	10
1.6	Naming Conventions	10
2	The MpCCI Concepts	11
2.1	User Requirements and Realization	11
2.2	The Layer Structure	11
2.2.1	Quantities and grid hierarchy	11
2.3	Communicator Concept	12
2.3.1	Coupling concepts and data transfer	13
2.4	Coupling Regions	15
2.5	Neighborhood Search and Interpolation	16
2.6	The MpCCI coupling server scheme	18
2.6.1	General idea of the coupling server scheme	18
2.6.2	Advanced features	19
3	MpCCI Functions	21
3.1	Naming Conventions and Terminology	21
3.2	MpCCI Data Types	22
3.3	Initialization and Coupling Definition	23
3.4	Coupling Communication	55
3.5	Remeshing	76
3.6	Termination	79
3.7	Control	80
3.8	Miscellaneous functions	92
3.9	Overview of the MpCCI functions	95
4	MpCCI Input File	97
4.1	Structure of the Input File	97
4.2	Code block	98
4.3	Quantities block	100
4.4	Control block	101
4.5	Contact block	104
4.6	Switches block	106
4.7	Jobs block	109
4.8	Parameters block	112
4.9	Coupling block	113

4.10	Additional block	117
4.11	Include Mechanism	119
5	Neighborhood Search	120
5.1	Relationships between geometric components	120
5.1.1	Point-Element Relationships (PE)	120
5.1.2	Element-Elements Relationships (EE)	121
5.1.3	Point-Point Relationships (PP)	121
5.2	Search Algorithms	121
5.3	Pre-contact Search Algorithms and Matching Criteria	122
5.3.1	Pre-Contact Search Algorithms in MpCCI	123
5.3.2	Matching criteria in MpCCI	124
5.4	Matching Grids	125
5.5	Non-Matching Grids	126
5.6	Initial Distance Tracking	130
6	Interpolation	132
6.1	Introduction	132
6.2	Coupling Regions	133
6.3	Available Interpolation	133
6.4	Standard Interpolation	134
6.4.1	Standard Non-Conservative Interpolation	134
6.4.2	Standard Conservative Interpolation	135
6.5	Mappings based on intersection method	137
6.5.1	Non-Conservative Mappings based on intersection methods	137
6.5.2	Conservative Mappings based on intersection methods	138
6.6	Nearest neighbor interpolation	139
6.7	About Remeshing	139
6.8	Elements in MpCCI	141
7	An Example	146
7.1	Startup and Initialization	146
7.2	Coupling Definition	148
7.3	Coupled Computation	151
7.4	Termination	154
A	Mesh quality checks by MpCCI	155
A.1	Checks for quadrilaterals	155
A.2	Checks for triangles	157
A.3	Checks for CCI_ELEM_QUAD8, CCI_ELEM_QUAD9 and CCI_ELEM_TRIANGLE6	158
A.4	Checks for pentagons and hexagons	160
	Bibliography	161

General Index	161
List of MpCCI Functions	162
List of MpCCI Constants	163

List of Figures

1	Layered structure of a process.	12
2	The MPI intra-communicator contexts	13
3	Process view	14
4	Exchange of data via synchronization points	15
5	Algorithmic use of synchronization points	15
6	Exchange of coupling values	16
7	Naming conventions for CCI_Def_mesh.	28
8	Mesh partitioning for parallel applications	30
9	CCI_ELEM_QUAD.	35
10	CCI_ELEM_TRIANGLE.	35
11	CCI_ELEM_QUAD8.	35
12	CCI_ELEM_QUAD9.	36
13	CCI_ELEM_TRIANGLE6.	36
14	CCI_ELEM_PENTAGON.	36
15	CCI_ELEM_HEXAGON.	37
16	CCI_ELEM_LINE.	37
17	CCI_ELEM_TETRAHEDRON.	37
18	CCI_ELEM_HEXAHEDRON.	38
19	CCI_ELEM_PRISM.	38
20	CCI_ELEM_PYRAMID.	39
21	CCI_ELEM_SPH_TRIANGLE.	39
22	CCI_ELEM_SPH_QUAD.	39
23	An example for CCI_Def_elems	40
24	Coordinate transformation	52
25	Data transfer between meshes of different codes	62
26	Intersection points 1	90
27	Intersection points 2	90
28	An example for CCI_Get_intersection_points	91
29	Bucket search in 2-D	124
30	Construction of bounding boxes	127
31	Minimal distance matching and rejection criteria.	129
32	Local cell coordinates of a target point q on a source surface.	132
33	Non-conservative interpolation	135
34	Non-conservative mapping	135
35	Conservative interpolation	136
36	Conservative mapping	137
37	Intersection method	137
38	Projection of triangles	138
39	Projection of points on the sphere	138
40	Original meshes, two-dimensional example.	139
41	Mesh deformation.	140

42	Mesh sliding.	140
43	Mesh refinement and load balancing	141
44	Two-dimensional coupling example of a structures with a fluid code.	146
45	Startup of the coupled application.	147

List of Tables

1	MpCCI data types for C.	22
2	MpCCI data types for Fortran.	23
3	Types of coordinate systems.	50
4	Overview of the MpCCI functions (1).	95
5	Overview of the MpCCI functions (2).	96
6	Types of geometric relationships.	120
7	Real time of search algorithms	125
8	Pre-contact algorithms for matching grids.	125
9	Matching criteria for matching grids.	126
10	Pre-contact algorithms for non-matching grids.	127
11	Matching criteria for non-matching grids.	128
12	Interpolation types.	134
13	Element types.	145
14	Values of parameters in interface definition for the example.	149
15	Parameters for the quadrilateral mesh quality tests.	155
16	Parameters for the triangle mesh quality tests.	157
17	Parameters for mesh quality tests of CCI_ELEM_QUAD8, CCI_ELEM_QUAD9 and CCI_ELEM_TRIANGLE6.	158

1 Introduction

1.1 The Intention of Developing MpCCI

Simulations of coupled problems become more and more important for industrial applications as well as for university based research programs. It turns out that numerical simulations of simple isolated problems have reached its limits. Therefore, a challenging task is to bring together different software packages that have been developed in the past for special applications, like the motion of fluids under certain assumptions or the deformation of a solid body under the action of stress. A fluid in motion, e.g. air, sets up a pressure gradient on a rigid body, which could be the wing of an airplane. This force leads to a deformation of the rigid structure. The deformation on the other hand counteracts on the fluid moving around this obstacle. Examples of other “classical” coupled problems are the climate system or the air flow through a turbine engine and combustion.

For end-users as well as code owners it is desirable to avoid a redevelopment of new simulation codes for coupled problems and to maintain the knowledge and experience of well accepted and verified high standard mono-disciplinary codes. Therefore, we developed the Mesh-based parallel Code Coupling Interface (MpCCI). The idea behind was to generate a standardized and portable library for solving coupled problems even with codes that are provided by independent software vendors with no access to the sources by the end-user.

As implied by its name, the library takes into account the fact that different codes in general might work on different numerical grids. Since most of the coupled problems are very CPU time consuming, these applications are first candidates for parallel computing.

The data structure going along with the parallelization of the numerical codes in question is also supported by MpCCI leading to efficient communication of data between processes of the simulation codes.

As a result of this, MpCCI can be regarded as the well known Message Passing Interface (MPI, [?]) on a higher level. In fact, many useful features and conventions of MPI are made use of by MpCCI. In the current stage message passing within MpCCI is realized with MPI routines. Nevertheless MpCCI can also be ported to any other communication library like PVM or CORBA in future releases.

1.2 Potential Users of MpCCI

As mentioned above coupled computation is not only restricted to some academical research, but also has found its way into industrial application. While universities in general have their own inhouse codes for research, research labs in industry often use commercial codes. Thus, MpCCI has to address simultaneously to independent software vendors or software engineers which provide their tools to the researcher, and to end-users that do not have a detailed insight into the codes.

1.3 MpCCI Functionality

With MpCCI the main benefit in contrast to any other message passing tool kit arises from the support of transferring mesh based quantities. Each code may use its own grid with a mesh generation independently from any other software. There is no restriction that the different numerical grids specified for each code

separately have to match at the coupling surface. Neighborhood search and interpolation is handled by MpCCI, even if the coupling surface is distributed over different processes.

The interface to MpCCI and therefore to any remote code is kept as easy to handle as possible. The user does not have to care about the interprocessor communication. Only a minimum training for end-users is required to build in the subroutine calls needed to exchange data. Since the coupling algorithms work independent from any numerical strategy used within the codes, MpCCI does not depend on the different parallelization techniques used in the stand-alone codes.

An easy to handle visualization tool may assist the end-user if a detailed insight into the processes along the coupling surface is required. Monitoring the data exchange between the different code is possible with the help of a control process.

1.4 Environment

The MpCCI library is available for various hardware architectures. Hitherto, it has been successfully configured and tested for

- SUN (Solaris),
- SGI (IRIX),
- IBM (AIX),
- Hitachi,
- Fujitsu VPP,
- CRAY T3E (UNICOSmk)¹,
- PC (WINDOWS NT),
- PC (Linux)

platforms.

The low level communication routines required for the data exchange between different processors follow the MPI standard. Any other MPI derivative for low level communication that follows this standard is also appropriate for using MpCCI. Besides

- native MPI, other communication libraries like
- MPICH,
- MetaMPI, or
- WMPI and

¹On UNICOSmk systems there is a general drawback that two executables cannot be started within one communication environment. Test runs have been performed by connecting a T3E with an IBM SP2 as well as T3E - T3E internal coupling via MetaMPI.

- MPICHNT

have served for the message passing within MpCCI successfully.

All developed porting procedures are optimized with respect to communication.

1.5 Programming Languages

The source code is completely written in C++ which supports object-oriented technology in a better way than FORTRAN 77. Nevertheless MpCCI provides interfaces to both

- C and
- FORTRAN 77/90

1.6 Naming Conventions

With the naming of the various routines provided by MpCCI we followed the naming conventions of MPI. Also the concept of communicators is adapted to MpCCI. Any end-user familiar with MPI should have no problems including the MpCCI calls into the code. On the other hand, no knowledge about MPI is required if the individual codes are running sequentially.

2 The MpCCI Concepts

The main goal of MpCCI is to establish a library which provides an easy to use interface originally developed for loose coupling of two or more stand-alone simulation codes, regardless of whether they are programmed as distinct executables or merged into a new program with each of the codes formulated as a subroutine. In the layered approach described in detail below we have in mind that MpCCI is situated between the application and the MPI library. Similar to MPI [?][?], MpCCI uses the communicator concept. Instead of just transferring data from one process to another MpCCI takes into account the grids on which the data are located. Thus tools for neighborhood search and interpolation are provided. Additionally, MpCCI is able to support several grids used within each of the codes. This concept is obviously required in the multigrid context. Finally, the MpCCI coupling server scheme offers more flexibility.

2.1 User Requirements and Realization

MpCCI is independent of the parallelization strategy used by a code. It integrates SPMD (single program multiple data) as well as MPMD (multiple program multiple data) programs. The user can use the codes he is familiar with. No training for new application software packages is required. Only minimal source code changes of the simulation codes themselves are necessary.

It is possible to include any interpolation scheme that might be useful for special applications. An interface to user defined functions helps to provide as much flexibility of this library as possible. The user is able to profit from the neighborhood search and message passing even when using parallel applications. Furthermore, the user is able to implement other individual interpolation mechanisms based on the grid information which is provided by MpCCI.

2.2 The Layer Structure

Due to its mesh-based character, MpCCI in a way elevates MPI to a higher level. Based on MPI, MpCCI performs communication as directly as possible between the individual processes of different codes. Usually, all communication between these different codes is performed by MpCCI. Additionally, the user is free to include message passing by using MPI to exchange data between any two coupled codes. MpCCI provides MPI communicators between the codes for that purpose.

The hierarchical order of the different packages involved is illustrated in Fig. 1

From the application code only MpCCI is called for a transfer of data to a remote code. MpCCI itself uses the MPI library to perform the message passing. Taking into account only the mesh based data transfer between any two codes the application code has to be linked with the MpCCI library only. Code, MpCCI and MPI operate in one thread.

2.2.1 Quantities and grid hierarchy

The values transferred by MpCCI are referred to as quantities. A quantity can be any type of variable used in any of the codes. An arbitrary dimension can be assigned to these quantities. Velocity, temperature, size of time-steps are typical examples for quantities. Obviously there are two types of quantities: On the one hand, some quantities are represented by discrete functions, evaluated during the computation on an

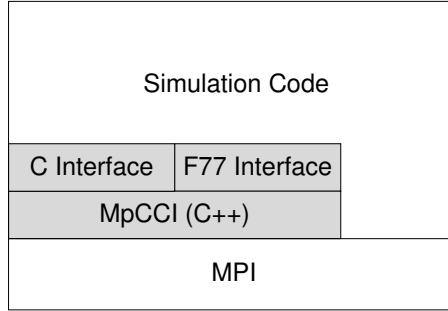


Figure 1: Layered structure of a process.

arbitrary mesh. Those values are assigned to a specific geometric position. On the other hand, there are quantities represented by one or more single values, which are independent of a geometrical position. The former are referred to as grid-based quantities, whereas the latter are described as global quantities.

A grid in MpCCI consists of a certain amount of parts, called coupling partitions. Each partition consists of a certain amount of geometrical positions referred to as nodes. Nodes can be combined using various types of elements in order to specify connectivity information used in the simulation codes. In the multigrid context this structure is applied to any of the grid levels separately.

Each grid-based quantity has to be assigned to exactly one grid. On the other hand, more than one quantity can be defined on each grid. Further, more than one grid can exist in each of the simulation codes participating in the coupled computation. Application of a multigrid algorithm could be one reason for using several grids covering the same geometrical region. Chimera techniques can be considered a more prominent example for the existence of several grids in one code. A staggered grid can be seen as a number of grids covering the same geometrical position.

2.3 Communicator Concept

An important MPI concept is the so-called communicator concept. A communicator denotes a communication domain and is the way in which the global message space is partitioned. This means that communication in each communication domain does not interfere with that in other communication domains. Therefore, a communicator must be given in each MPI call to indicate which communication domain has to be used.

The two most important properties of a communicator are the group of processes it refers to and its context. The group specifies the physical processes that participate in the communication domain. The context is attached to any message sent and compared with incoming messages when messages are received. In this way, the different communication domains are automatically separated from each other. It is clear that this property supports the development of parallel libraries. In particular, code-internal communication is automatically separated from library-internal communication (Figure 2²). It is furthermore

²Figure 2 is a simplified explanation. The actual MPI implementation may use different representations for the same context on different processes and may therefore have to store tables to match contexts.

guaranteed that code-internal communication does not interfere with library-internal communication.

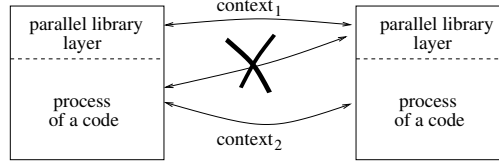


Figure 2: The MPI intra-communicator contexts

In addition to the (intra-)communicators explained above, which are needed to perform communication within a certain group of processes, there are inter-communicators in MPI, which are used to communicate between two disjoint groups of processes: the local and remote groups. An inter-communicator works just like an intra-communicator and also has context to partition message space. In most MPI subroutines, an inter-communicator can be used wherever an intra-communicator can. The reader is encouraged to read the MPI documentation for the fine details, see [??].

MPI inter-communicators are used by MpCCI internally to perform communication between different codes. That is, for each pair of codes, a separate inter-communicator is used. In Figure 3 for example, separate inter-communicators are used for the communication between the interface layers of codes A and B, B and C, and A and C. Just like the MPI library, MpCCI requires such a communicator in each of its calls to indicate the communication domain.

Communication in MpCCI is always between pairs of codes. Therefore, the code id of a remote code uniquely identifies the MPI inter-communicator to be used internally by MpCCI. A code id (or job id) is an integer that identifies a specific code running a specific job. The usage of a remote code id instead of an MPI inter-communicator enables MpCCI to manage its MPI inter-communicators itself. This avoids the problem of a code accidentally using this communicator for MPI communication directly, and thereby possibly interfering with MpCCI communication.

For optimal flexibility, MpCCI also provides duplicates of the internally-used MPI inter-communicators to a code. This enables a code to perform communication with another code directly through MPI outside of MpCCI. This aspect is of major importance for extending MpCCI towards specific coupling algorithms. In effect, it enables MpCCI to be used as a building block for more advanced libraries.

2.3.1 Coupling concepts and data transfer

Since MpCCI can be regarded as a message passing interface, e.g. MPI, on a higher level, data transfer is one of the main tasks of MpCCI. Coupling values are transferred both transparently and efficiently. Full flexibility as well as high comfort are offered to the user. To ensure a high degree of scalability and performance communication takes place exclusively among processes participating in the coupled computation.

MpCCI supports two kinds of concepts for the data exchange.

Blocking and non-blocking communication can be invoked by routines similar to the according routines in MPI. This concept is MPI-like based on send and receive actions and will not be described in this

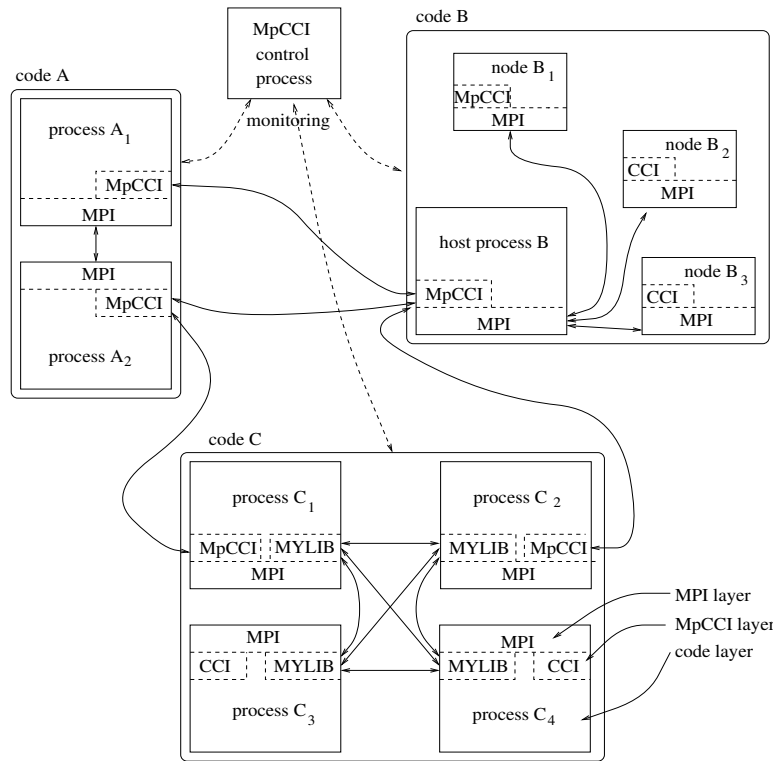


Figure 3: Process view. Coupling through a parallel communications library: Code C uses a communications library ‘MYLIB’ for its own parallelization.

paragraph. For more information see the descriptions of the subroutines (CCI.Send and CCI.Recv) in chapter 3 or read the example in chapter 7.

The second concept is based on so-called synchronization points which offers the opportunity of bi-directional communication: Reaching a synchronization point a process may send and receive data. Only those quantities are transferred to other processes, that are required by the remote processes. An illustration is given in figure 4. Codes taking part in a synchronization point may be defined in a flexible way.

An example for a simple algorithmic use of synchronization points is given in figure 5.

For a more detailed description see the definitions of the subroutines (CCI_Def_sync_point, CCI.Sync_point_info and CCI_Reach_sync_point) in chapter 3 and section 4.9.

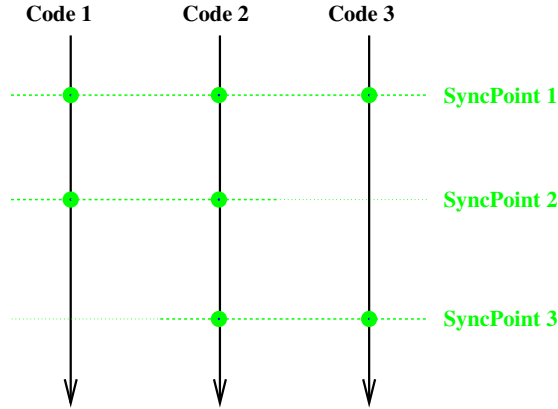


Figure 4: Exchange of data via synchronization points

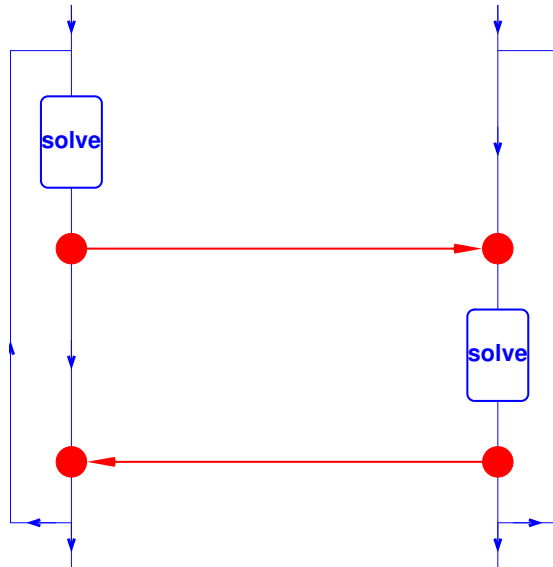


Figure 5: Algorithmic use of synchronization points

2.4 Coupling Regions

MpCCI supports different kinds of coupling regions and spaces. In the two-dimensional and three-dimensional space line, surface and volume coupling based on element definitions are possible. Meshes consisting of spherical triangles or (exclusive) quadrilaterals are treated as a special case. Furthermore MpCCI is able to perform data transfer on nodes without element definition.

2.5 Neighborhood Search and Interpolation

The exchange of grid based coupling quantities is a non-trivial task. Due to the basic discretisation in the simulation codes the coupling data are usually arranged on different structured or unstructured meshes. As an example the grid of one multigrid level of each code is shown in Fig. 6. Therefore, MpCCI has to perform neighborhood search and interpolation between the meshes to realize a sensible exchange of coupling data. Each of the meshes is usually distributed on several parallel processes. The complex problem raised by this distribution is solved by MpCCI efficiently.

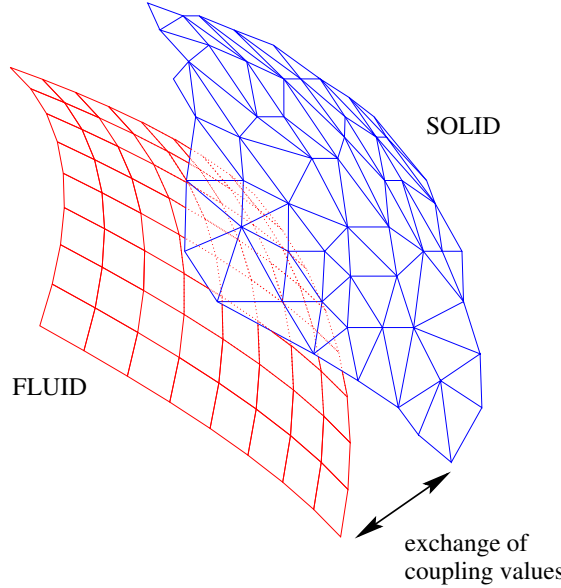


Figure 6: Data exchange between two non-matching grids (distance exaggerated)

The main problem is to determine how these surfaces fit together, the so-called neighborhood computation. This information is used then to establish the interprocessor communication, and to assist the codes in the interpolation of coupling values between the different grids. MpCCI will support the interpolations as described in Chap. 6. There we also discuss the aspect of moving meshes.

The simulation codes must specify their coupling region to MpCCI. The coupling regions may consist of the element types described in Chap. 6. The coordinate systems must be global in the MpCCI standard version to enable MpCCI to detect the distances and relations between the specified regions. It is allowed that the coupling regions are not identical but distant from each other. The maximum distance allowed is adjustable by a parameter in the input file of MpCCI. In the case of non-matching grids the data arranged on the nodes of the source grid are interpolated by MpCCI on the nodes of the target grid. In MpCCI advanced and professional version coordinate systems can be transformed so that they match each other.

In the coupling definition phase each process of a code announces its parts of the coupling region with their nodes and elements of the coupling surface to MpCCI. Each coupling partition has to be assigned

with a partition identifier (see Chap. 3) which must be used in subsequent MpCCI calls to indicate the involved part of the coupling region. When each code has specified the grid data of its side of the coupling surface, MpCCI performs the contact or neighborhood search which is necessary for the interpolation.

A linear and a bilinear interpolation depending on the type of the element is implemented. Also, the user can choose between a conservative and a non-conservative interpolation (see Chap. 6 for further details).

With respect to the whole coupling process the main computation is carried out in the communication phase. In this phase the coupling data are specified to the library. MpCCI sends the data to the receiver code, interpolates the data onto the nodes of the target grid and distributes the interpolated data to the specified arrays. Then the computations of the receiver code, which depend on the coupling data, can start.

Based on this information the communication is optimized in the following way: Only those processors exchange data, that have grid points for which a connectivity was detected, instead of distributing all data to all processors.

2.6 The MpCCI coupling server scheme

MpCCI offers two server communication schemes:

- the well-established "old" MPI-based scheme where all codes build up a big common MPI world and must be started jointly,
- a more flexible scheme based on "coupling servers", where each code lives in its own MPI world (if it is parallel in itself) and is started independently of the others.

This chapter describes the general idea of the coupling server scheme and names more advanced features. The basic handling of the coupling server scheme and a reference of all configuration switches is given in the MpCCI SDK Tutorial which also describes the configuration of the coupling server communication in some typical scenarios.

2.6.1 General idea of the coupling server scheme

In the original MpCCI communication scheme the processes of all codes (plus the MpCCI control process) were in one big common MPI world. Therefore, all codes had to be linked to the same MPI. They communicated in this world via MPI, and it was MpCCI's task to build up MPI communicators for each code where the code internal communication could then take place (CCI_MPI_COMM.CODE etc).

The idea of the coupling server scheme is to separate the codes into their own independent communication worlds: each code uses its own MPI for its internal parallelization, and there are no restrictions or interferences with the MPIs of the remote codes. The codes are started independently in the same way as in the standalone case. The coupling is done via additional "coupling server" processes (one for each code process), which live in a normal MpCCI world using the "old" MPI-based communication mode. In case of two coupled codes there are three a priori independent "worlds":

- the processes of code A
- the processes of code B
- the MpCCI coupling server processes, one process for each code process of A and B

The "coupling server" processes execute the special binary "cciserver" which is provided by the MpCCI distribution. Each coupling server process is associated to exactly one code process. The coupling server does nothing else but listen for MpCCI requests from its code process. If such an MpCCI request arrives, the coupling server calls the corresponding CCI subroutine, and communicates the results of the CCI subroutine back to its code process.

The communication between a coupling server and its code process is not MPI based (because they do not live in the same MPI world), but instead based on low-level interprocess communication (e.g. sockets). Therefore, the code processes themselves are not linked to the normal MpCCI library "libcci.a", but to the "libcciclient.a" from the MpCCI distribution. The library "libcciclient.a" has exactly the same interface as "libcci.a", so that you only have to relink your code and no recompilation (or even changes in the code) are necessary.

The difference between the normal "libcci.a" and the "libcciclient.a" is that "libcciclient.a" delegates the MpCCI calls to the corresponding coupling server which in turn lives in a "normal" MpCCI world. The coupling server is permanently listening for requests from its corresponding code process (therefore the name "server", the code process is a "client" of its coupling server). If such a request arrives, the coupling server (which itself is linked to the "normal" MpCCI library) does what the normal CCI routine does, e.g. sending data to the other side (via MPI). Note that on the other side there will also be a coupling server process if the other code uses the MpCCI server communication scheme, too.

Of course it is possible, that only one code A uses the server communication scheme and the other code B uses the "old" MpCCI communication mode. In this case, the processes of code B would live in a common MPI world with the coupling server processes of code A, which in turn would live in a separate communication world.

The advantages of the coupling server scheme are:

- Each code lives in its own MPI world (no MPI version problems).
- Each code is started independently (in the same way as in the standalone case).
- The codes can run on different platforms (even one code on UNIX and the other on Windows) and can use the native MPIs of the platforms for their code internal parallelization.
- The library "libcciclient.a" which is linked to the codes is a very thin software layer (for the C interface of MpCCI it consists of only one single object file). It is much smaller than the original "libcci.a" (the reason is that "libcciclient.a" is only responsible for the ping-pong like client-server communication, whereas "libcci.a" contains the more complicated MPI-based communication between the MpCCI processes and, in addition, the contact search and interpolation algorithms).
- The total memory consumption of MpCCI is the same as for the original MpCCI communication. For the coupling server scheme, the MpCCI internal data are not stored on the code process itself, but on the corresponding cciserver process.

Of course, there are also some disadvantages:

- There are two additional communication steps:
code → cciserver ⇒ remote cciserver → remote code
- The double number of processes is needed (one cciserver for each code process).

2.6.2 Advanced features

Let us begin with a few technical words about the cciserver setup phase. This will help you understand some of the advanced configuration parameters.

How do the cciserver processes find their clients? This is non-trivial because the cciservers and the clients are started independently of each other. Therefore, the setup phase of the cciserver approach is a bit tricky.

In our implementation, the first cciserver process (the so called "leading cciserver") plays a special role: it mediates the communication setup between each code process and its corresponding cciserver. This is done in the following way:

- The leading cciserver knows from the MpCCI input file, how many clients are to be expected. It opens a socket connection and listens to a specific port for client registration messages.
- The clients, i.e. the processes of the codes read a setup file "\$HOME/.cciclientrc" (if CCI_CLIENT_HOME is set and a writable directory: "\$CCI_CLIENT_HOME/.cciclientrc") which has been written by the ccirun script when starting the MpCCI world of the cciservers. This setup file contains information about the IP address and the port number of the leading cciserver. So the first action of each client during CCI_Init is to contact the leading cciserver for registration.
- After receiving the registration message from a code process, the leading cciserver associates one of the cciservers to the client. It sends the IP address and a port number to both the client and the corresponding cciserver. The relevant cciserver then listens to this port, and the client connects to this port. In this way, the client sets up a connection to its cciserver. When it is set up, all subsequent communication between client and server will be handled via this connection.

In addition to this setup mechanism where the leading cciserver mediates the connections, there is an alternative setup where clients and servers find themselves directly. This direct setup requires additional information both for the cciservers and the clients.

There are a lot of configuration switches by means of which the user can adjust the behaviour of the coupling servers. They are described in the MpCCI SDK Tutorial.

3 MpCCI Functions

This chapter describes the basic MpCCI subroutines needed to perform the coupling algorithms described previously. Before we start with a detailed description of the individual calls we give a brief introduction to the terminology used. The reader who is familiar with FORTRAN 77, C and MPI may skip Sec. 3.1. For a brief overview of the complete set of MpCCI functions the reader is referred to Tab. 4 on page 95 and to the index list at the end of this manual.

3.1 Naming Conventions and Terminology

The naming conventions for MpCCI subroutines are similar to those in MPI. Each subroutine name consists of a list of words separated by underscores '_'. Subroutine names start with the word 'CCI' as in **Code Coupling Interface**. The first letter of the second word is capitalized as in **CCI.Send**. The FORTRAN 77 versions will have names that are all upper case, e.g. **CCI_SEND**. To achieve portability, the names of MpCCI subroutines will be at most 30 characters long. The names of non-blocking routines start with **CCI.l** (followed by lower case letters in C). For example, the non-blocking counterpart of **CCI.Send** is **CCI.lsend**, in FORTRAN 77 it is **CCI_SEND**.

According to [?] we will use the following terminology in the description of MpCCI subroutines:

- **local:** If the procedure of the subroutine depends only on the local executing process. Such an operation does not require an explicit communication with another process.
- **non-local:** If completion of the procedure may require the execution of a MpCCI procedure on another process.
- **blocking:** If return from a MpCCI procedure indicates the user is allowed to re-use resources specified in the call. Any visible change in the state of the calling process is completed before the call returns.
- **non-blocking:** If the procedure may return before the operation initiated by the call completes, and before the user is allowed to re-use resources (such as arrays specified in the call).
- **collective:** If all processes in a certain process group must invoke the procedure.

In addition to MPI, the following terminology is used:

- **one-side collective:** If completion of the operation initiated by the call is a collective operation over the MpCCI processes of the local code.
- **two-side collective:** If completion of the operation initiated by the call depends on the MpCCI processes of both the local and remote code.

MpCCI procedures are specified in language independent notation. Subroutine arguments are marked as

- **IN**, for arguments whose values may be used but are not changed,

- OUT, for arguments whose values are not used on input, but may be changed by the call,

or

- INOUT, for arguments whose values may be used on input to the call, and may be changed by the call.

Arguments of type OUT may have arbitrary input values. Aliasing of arguments in procedure calls is only allowed if these arguments are all IN variables. Otherwise, the results are undefined.

Concrete C and FORTRAN 77 descriptions are added for MpCCI subroutines that have been implemented. Note that all MpCCI subroutines in FORTRAN 77 have an additional argument `iererror` at the end of the argument list. This argument is a FORTRAN 77 integer and has the same meaning as the return value of the routine in C. In FORTRAN 77, MpCCI and MPI routines are subroutines, and are invoked with the call statement.

The index order of array elements differs in FORTRAN 77 and C: In our examples we refer to the FORTRAN 77 index ordering where the first element of an array `ary` is `ary(1)`³.

MpCCI subroutines for receiving coupling values have a return status output argument. This argument contains more detailed information about the information received.

In C, `status` is a structure of type `CCI_Status`. In FORTRAN 77, it is an integer array of length `CCI_STATUS_SIZE`. The constant `CCI_STATUS_SIZE` is defined in `ccif.h`.

3.2 MpCCI Data Types

Some of the MpCCI functions described in the following have an argument concerning the data type of values. To avoid problems with the different data types on various platforms MpCCI defines constants of its own that should be used.

The different data types for C and Fortran are collected in the tables below.

Table 1 lists the available MpCCI data types for C:

<i>C type</i>	<i>MpCCI type</i>
int	CCI_INT
float	CCI_FLOAT
double	CCI_DOUBLE
long double	CCI_LONG_DOUBLE
char []	CCI_STRING
bool	CCI_BOOL

Table 1: MpCCI data types for C.

Table 2 lists the available MpCCI data types for Fortran:

³In C it would be `ary[0]`.

<i>C type</i>	<i>MpCCI type</i>
integer	CCI_INTEGER
real	CCI_REAL
real*4	CCI_REAL4
real*8	CCI_REAL8
double precision	CCI_DOUBLE_PRECISION
character*(*)	CCI_STRING
logical	CCI_LOGICAL

Table 2: MpCCI data types for Fortran.

3.3 Initialization and Coupling Definition

Connections between different codes are established by MpCCI in the startup and initialization phase. Also, several MPI communicators are determined for use by the codes.

In the MpCCI initialization phase, several constants and MPI communicators are computed. To access these constants from C, one must use:

```
#include "cci.h"
```

To access these constants from FORTRAN 77, one must use the following:

```
c    --- include common block containing MpCCI constants
      include 'ccif.h'
```

In the coupling definition phase, each code specifies its coupling region to MpCCI. This coupling region in general consists of the part of the grid coordinates that take part at the coupling. After that the initial neighborhood computation takes place. The search algorithm is explained in detail in Chap. 5 on page 120.

CCI_Init

pArgc	INOUT	pointer to pass the argument count to MPI_Init
pArgv	INOUT	pointer to pass the arguments to MPI_Init

```
int CCI_Init( int* pArgc, char*** pArgv )
```

```
subroutine CCI_INIT( ierror )
```

```
integer ierror
```

This subroutine enrolls a code in the coupled computation. CCI_Init initializes MpCCI for both C and FORTRAN 77/ FORTRAN 90 usage. Afterwards, the complete environment for the caller has been set as specified in the MpCCI input file (working directory, environment variables). Since the environment is not set before this call, a code should not do I/O before CCI_Init. Since the initialization of MPI is required by the MpCCI routines MPI_Init is called from within CCI_Init if it has not been called before; in this case CCI_Finalize (see page 79) will also call MPI_Finalize. In case MPI_Init is called from the program anyway CCI_Init has to be called after invoking MPI_Init.

CCI_Init initializes the following external variables defined in `cci.h` (for C), `ccif.h` (for FORTRAN 77) and `ccif90.f90` (for FORTRAN 90):

- CCI_NCODES

The number of codes (jobs) involved in the current computation.

- CCI_MYCODEID

The code id of the current code (job), $1 \leq \text{CCI_MYCODEID} \leq \text{CCI_NCODES}$.

A code id CCI_MYCODEID is always in the range $1 \leq \text{CCI_MYCODEID} \leq \text{CCI_NCODES}$. If the same code occurs twice in the coupled computation, CCI_MYCODEID will have different values at the different instances of this code.

Also, the following MPI communicators will be available:

- CCI_MPI_COMM_WORLD

MPI intra-communicator involving all processes of the coupled computation.

- CCI_MPI_COMM_CODE

MPI intra-communicator involving all processes of the current code.

- `CCI_MPI_ICOMM_CODE`

Array of length `CCI_NCODES` of MPI inter-communicators for the current code; one for each remote code. It holds that

$$\text{CCI_MPI_ICOMM_CODE}(\text{CCI_MYCODEID}) = \text{MPI_COMM_NULL}.$$

For the C interface, it also holds that

$$\text{CCI_MPI_ICOMM_CODE}[0] = \text{MPI_COMM_NULL}.$$

These communicators can be used directly by a code without risk of interfering with MpCCI internal communication.

A code must perform its local communication using `CCI_MPI_COMM_CODE` instead of `MPI_COMM_WORLD`. `MPI_COMM_WORLD` is unsuitable for this because it contains all processes of the coupled computation, i.e. processes of other codes and the control process.

The MPI communicators `CCI_MPI_COMM_WORLD` and `CCI_MPI_ICOMM_CODE` can be used for direct communication between different codes, using MPI, and not involving MpCCI.

`CCI_Init` is a blocking collective operation over all processes of all codes, to be performed after startup of a code, and before any other MpCCI subroutines are called. No MPI calls should take place between call to `MPI_Init` and `CCI_Init`, and no I/O should take place before `CCI_Init`. Also the communicator `MPI_COMM_WORLD` should not be used for any communication in the whole coupled computation.

Class	non-local, blocking
Position	first MpCCI call

CCI_Init_with_id_string

pArgc	INOUT	pointer to pass the argument count to MPI_Init
pArgv	INOUT	pointer to pass the arguments to MPI_Init
idString	IN	id string

```
int CCI_Init_with_id_string( int* pArgc, int*** pArgv, const char* idString )
```

```
subroutine CCI_INIT_WITH_ID_STRING( idString, ierror )
```

```
character*(*) idString
```

```
integer ierror
```

In addition to the task of the subroutine CCI_Init this subroutine offers the possibility to overwrite the id string of the code block of the MpCCI input file with the string **idString**. The subroutines CCI_Init and CCI_Init_with_id_string must not be called both in one code.

Class	non-local, blocking
Position	first MpCCI call

CCI_Def_mesh

meshId	IN	grid identifier within each code
idString	IN	identifier(s) of the mesh

```
int CCI_Def_mesh( int meshId, const char idString[] )

subroutine CCI_DEF_MESH( meshId, idStringLength, idString, ierror )

integer      meshId, idStringLength, ierror
character(*) idString
```

Subroutine `CCI_Def_mesh` enables the user to define expressive names for the different meshes. These names can be used in the coupling block of the `MpCCI` input file.

The argument `idString` may contain different names separated by comma. All substrings are compared with those of the other code to determine the matching meshes.

Note: this subroutine has to be called before `CCI.Sync_point_info` and `CCI.Comm_info`. Otherwise the meshes defined in the latter calls are unknown to `MpCCI`.

Fig. 7 illustrates the naming conventions of CCI_Def_mesh:

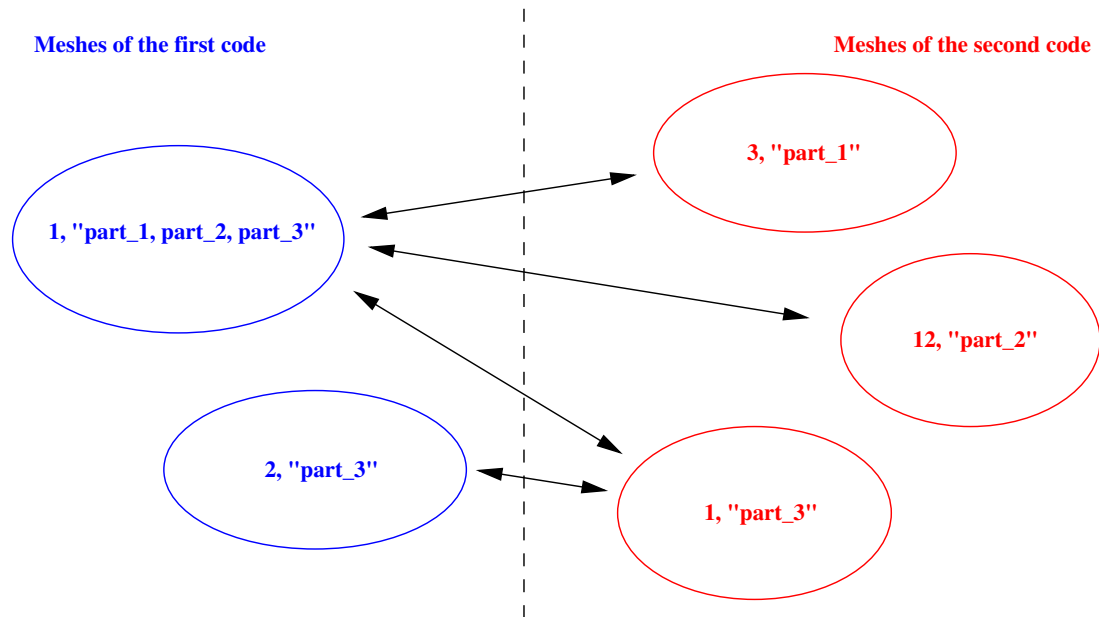


Figure 7: Naming conventions for CCI_Def_mesh.

Class	local, non-blocking
Position	after closing the initial phase, first definition

CCI_Def_partition

<code>meshId</code>	IN	grid identifier within each code
<code>partId</code>	IN	partition identifier of each grid

```
int CCI_Def_partition( int meshId, int partId )

subroutine CCI_DEF_PARTITION( meshId, partId, ierror )

integer    meshId, partId, ierror
```

With `CCI_Def_partition` a partition of a grid is defined which will take part in the coupling process. The call of `CCI_Def_partition` with the parameters `meshId` and `partId` defines a partition with the identifier `partId` on the grid with identifier `meshId`.

Each code may use several (N) grids for solving the equations (multigrid, staggered grid). Every grid of each code is identified by the `meshId`. This numbering of the different grids is arbitrary.

Each grid identified by the `meshId` may consist of several partitions. From the view of `MpCCI` the grid partitions are the smallest parts of a grid, the “grid atoms”. Every `MpCCI` process may work on a set of partitions.

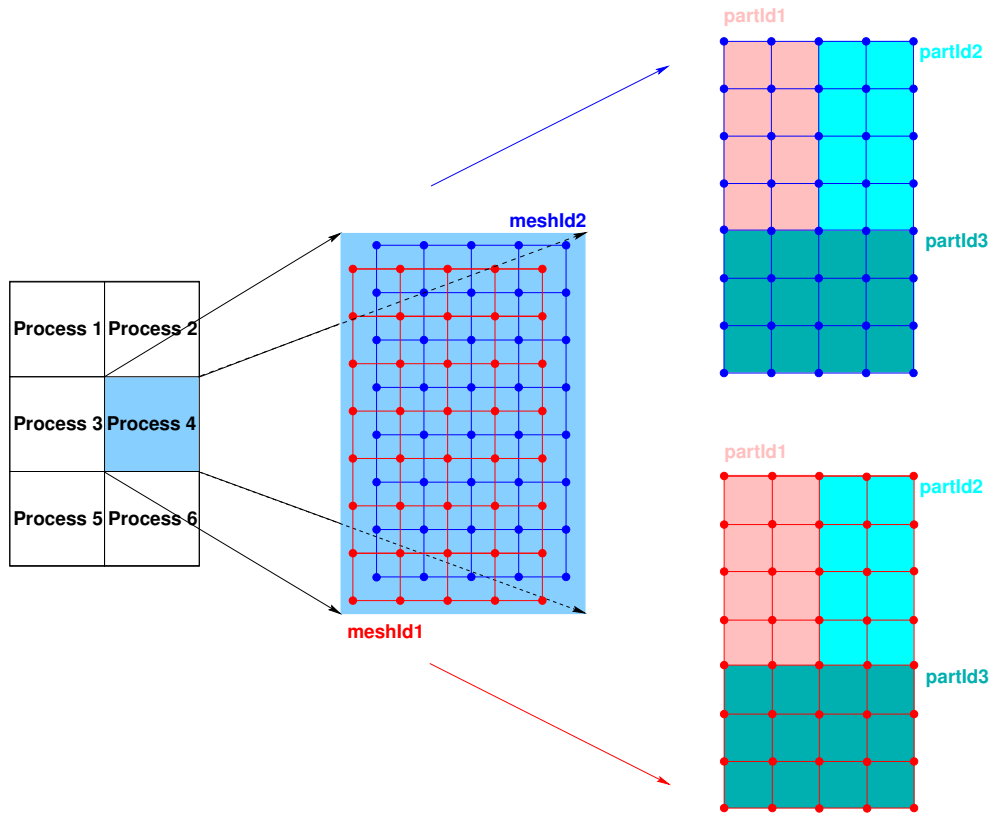


Figure 8: Mesh partitioning for parallel applications

The following example defines two grids with three partitions on both grids like in figure 8 (the number of partitions on each grid may not be equal):

```
CCI_Def_partition( meshId1, partId1 );
CCI_Def_partition( meshId1, partId2 );
CCI_Def_partition( meshId1, partId3 );
CCI_Def_partition( meshId2, partId1 );
CCI_Def_partition( meshId2, partId2 );
CCI_Def_partition( meshId2, partId3 );
```

Class	local, non-blocking
Position	after closing the initial phase, second definition (after calls of CCI_Def_mesh, if used)

CCI_Def_nodes

<code>meshId</code>	IN	grid identifier within each code
<code>partId</code>	IN	partition identifier of each grid
<code>globalDim</code>	IN	dimension of the global coordinate system
<code>nNodes</code>	IN	number of nodes
<code>nNodeIds</code>	IN	switch node numbering, dimension of array <code>nodeIds</code>
<code>nodeIds</code>	IN	node identifier
<code>realType</code>	IN	data type of elements of <code>coords</code>
<code>coords</code>	IN	address of coordinate array

```
int CCI_Def_nodes( int meshId, int partId, int globalDim, int nNodes,
                  int nNodeIds, const int nodeIds[], int realType,
                  const void* coords )
```

```
subroutine CCI_DEF_NODES( meshId, partId, globalDim, nNodes,
                        nNodeIds, nodeIds, realType,
                        coords, ierror)
```

```
integer          meshId, partId, globalDim, nNodes, nNodeIds,
                 nodeIds(nNodeIds), realType, ierror
```

```
<real type>      coords(globalDim*nNodes)
```

The purpose of subroutine `CCI_Def_nodes` is to specify the nodes (grid points) within each partition that will take part in the coupling. `CCI_Def_nodes` is a strictly local operation. Based on these nodes the elements are defined.

There are three different ways of numbering the nodes. They can be selected by the actual choice of variable `nNodeIds`:

1. `nNodeIds = 0` The nodes are numbered consecutively from 1 to `nNodes`. `nodeIds` is ignored.
2. `nNodeIds = 1` The nodes are numbered consecutively from `nodeIds(1)` to `nodeIds(1)+nNodes`. `nodeIds(1)` has to be specified of course.
3. `nNodeIds = nNodes` The numbering of the nodes will be according to the elements of array `nodeIds`. The number of the *i*-th node is `nodeIds(i)`.

The layout of array `coords` containing the coordinates of the nodes is the same in FORTRAN 77/ FORTRAN 90 and C. If, for `globalDim = 3`, x_i , y_i , and z_i are the coordinates of the *i*-th point with

$1 \leq i \leq \text{nNodes}$, then the points are stored in the following way:

$$x_1, y_1, z_1, x_2, y_2, z_2, \dots, x_{\text{nNodes}}, y_{\text{nNodes}}, z_{\text{nNodes}}.$$

For the value of variable `realType` please refer to table 1 and table 2.

It is possible to redefine coordinates for any `nodeId` later in the program. But this has to be done by a call of `CCI_Mod_nodes` within a remeshing operation (and not in a second call to `CCI_Def_nodes`).

`CCI_Def_nodes` has to be called for each partition defined. This implies that `CCI_Def_partition` has to be called first before invoking `CCI_Def_nodes` for each partition. For each partition `CCI_Def_nodes` must only be called once. It is not possible to add nodes to a partition by calling `CCI_Def_nodes` a second time.

Example: The following example defines the nodes of partition `partId` on grid `meshId` (see figure 23, page 40):

```
/**
 * Variables for CCI_Def_nodes:
 */

/* Global dimension: 3 coordinates per node. */
int globalDim = 3;
/* Number of nodes for the partition: */
int nNodes = 9;
/* Node numbering according to the first item: */
int nNodeIds = 0;
int* nodeIds = 0;
/* Data type of the coordinates of the nodes: */
int realType = CCI_FLOAT;
/* Coordinates of the nodes: */
float coordsData[] = { 0, 0, 0, /* node 1 */
                      0.5, 0, 0, /* node 2 */
                      1, 0, 0, /* node 3 */
                      0, 0.5, 0, /* node 4 */
                      0.5, 0.5, 0, /* node 5 */
                      1, 0.5, 0, /* node 6 */
                      0, 1, 0, /* node 7 */
                      0.5, 1, 0, /* node 8 */
                      1, 1, 0 }; /* node 9 */

...

/* Call of the subroutine */
CCI_Def_nodes( meshId,
              partId,
```

```
globalDim,  
nNodes,  
nNodeIds,  
nodeIds,  
realType,  
coordsData );
```

Class	local, non-blocking
Position	behind CCI.Def_partition (or within a remeshing)

CCI_Def_elems

meshId	IN	grid identifier within each code
partId	IN	partition identifier of each grid
nElems	IN	number of elements
nElemIds	IN	switch for element numbering, dimension of array <code>elemIds</code>
elemIds	IN	element identifier
nElemTypes	IN	length of array <code>elemTypes</code>
elemTypes	IN	element type of the specified elements
nNodesPerElem	IN	number of nodes needed to describe one specific element
elemNodes	IN	indices of the nodes which define the elements

```

int CCI_Def_elems( int meshId, int partId, int nElems,
                  int nElemIds, const int elemIds[],
                  int nElemTypes, const int elemTypes[],
                  const int nNodesPerElem[], const int elemNodes[] )

subroutine CCI_DEF_ELEMS( meshId, partId, nElems, nElemIds,
                        elemIds, nElemTypes, elemTypes,
                        nNodesPerElem, elemNodes, ierror )

integer    meshId, partId, nElems,
           nElemIds, elemIds(nElemIds), nElemTypes,
           elemTypes(nElemTypes), nNodesPerElem( ), elemNodes( ), ierror

```

This strictly local operation is used to specify the topology of a coupling partition by specifying the number of elements, element type, and for each element the indices of the nodes that define it. The node indices correspond to the ordering of the node specification in the call of `CCI_Def_nodes` (see page 31).

`nElems` elements can be defined. These may consist of only one type (`nElemTypes` = 1) up to `nElems` types with different element types `elemTypes`. Thus, different element types can be specified for the same coupling partition with just one call of this subroutine.

For nearest-neighbor interpolation from node to node `CCI_Def_elems` can be omitted.

In `MpCCI` the interpolation type depends on the element type which is chosen and the settings in the `MpCCI` input file. For every element type it is possible to define element-based and node-based quantities.

Seven types of surface elements, where two element types can also be used for volume coupling in the two-dimensional space, four types of volume elements in the three-dimensional space, one line element, i.e. surface element in the two-dimensional space, and two element types for coupling on the sphere are provided in the current version:

1. **CCI_ELEM_QUAD**: The number of nodes is four. The nodes can be enumerated anti-clockwise or clockwise, but the direction has to be the same for the whole partition. This element type can be used for surface coupling in the three-dimensional space and volume coupling in the two-dimensional space.

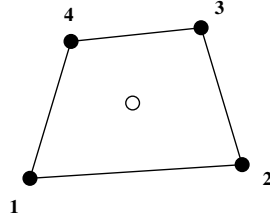


Figure 9: CCI_ELEM_QUAD.

2. **CCI_ELEM_TRIANGLE**: The number of nodes is three. This element type can be used for surface coupling in the three-dimensional space and volume coupling in the two-dimensional space.

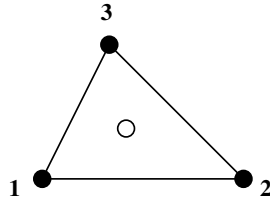


Figure 10: CCI_ELEM_TRIANGLE.

3. **CCI_ELEM_QUAD8**: The number of nodes is eight. The first node stored in `elemNodes` must be a node at a vertex of the element, the other nodes must be enumerated anti-clock-wise.

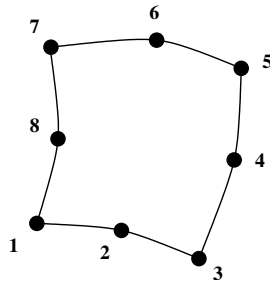


Figure 11: CCI_ELEM_QUAD8.

4. `CCI_ELEM_QUAD9`: The number of nodes is nine. The first node stored in `elemNodes` must be a node at a vertex of the element, the other nodes must be enumerated in rows.

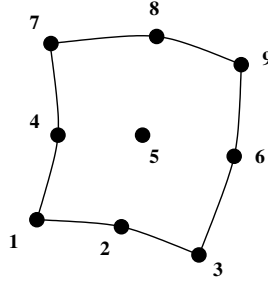


Figure 12: `CCI_ELEM_QUAD9`.

5. `CCI_ELEM_TRIANGLE6`: The number of nodes is six. The first node stored in `elemNodes` must be a node at a vertex of the element, the other nodes must be enumerated anti-clock-wise.

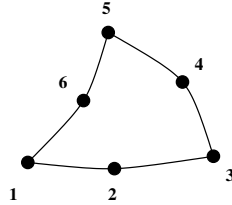


Figure 13: `CCI_ELEM_TRIANGLE6`.

6. `CCI_ELEM_PENTAGON`: The number of nodes is five. The nodes are enumerated anti-clock-wise. The interpolation will be done via triangles.

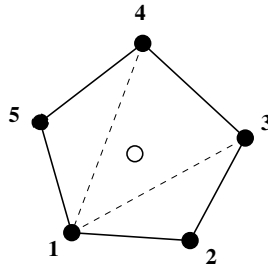


Figure 14: `CCI_ELEM_PENTAGON`.

7. `CCI_ELEM_HEXAGON`: The number of nodes is six. The nodes are enumerated anti-clock-wise. The interpolation will be done via quadrilaterals.

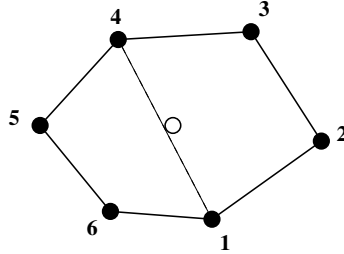


Figure 15: `CCI_ELEM_HEXAGON`.

For line coupling in the 2-dimensional space there is currently implemented only one element type: `CCI_ELEM_LINE`. This element type represents the line segment between two nodes.



Figure 16: `CCI_ELEM_LINE`.

The volume elements are described in the following enumeration:

1. `CCI_ELEM_TETRAHEDRON`: The number of nodes is four. The interpolation defined for this element is linear.

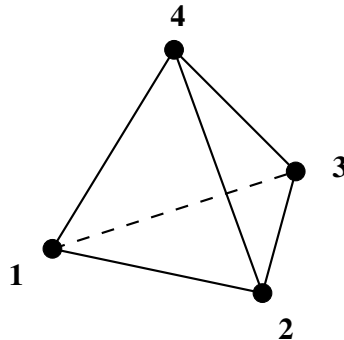


Figure 17: `CCI_ELEM_TETRAHEDRON`.

2. `CCI_ELEM_HEXAHEDRON`: The number of nodes is eight. The numbering of the nodes is shown in the figure. The interpolation defined for this element is linear.

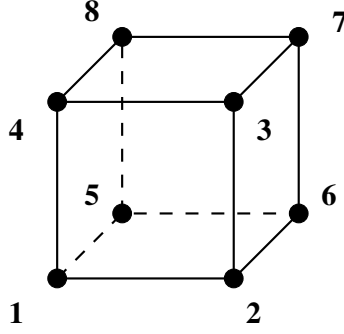


Figure 18: CCI_ELEM_HEXAHEDRON.

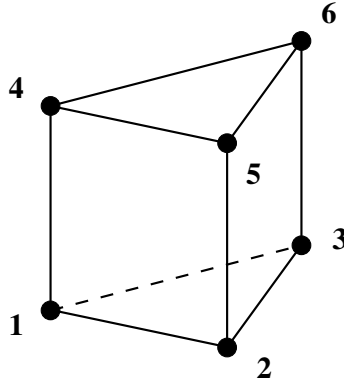


Figure 19: CCI_ELEM_PRISM.

3. CCI_ELEM_PRISM: The number of nodes is six. The interpolation defined for this element is linear.

4. CCI_ELEM_PYRAMID: The number of nodes is five. The interpolation is non-linear.

The element types for surface coupling, line coupling and volume coupling must not be mingled on the same mesh. The global dimension of nodes for line coupling and volume coupling in the two-dimensional space is 2, for surface and volume coupling in the three-dimensional space the global dimension is 3.

MpCCI supports coupling on the sphere, but several restrictions must be regarded. The midpoint of the sphere must be the origin of the coordinate system. All meshes must consist of the same spherical element type. This is due to the fact that the spherical quadrilateral cannot be decomposed in two spherical triangles. The spherical element types are defined as follows:

- CCI_ELEM_SPH_TRIANGLE: The number of nodes is three. The nodes are connected by curves on the sphere which have the minimal length.

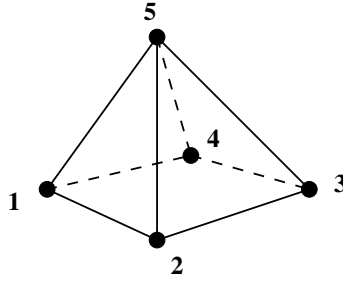


Figure 20: CCI_ELEM_PYRAMID.

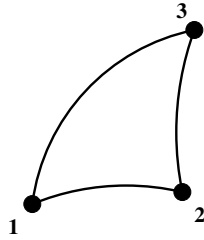


Figure 21: CCI_ELEM_SPH_TRIANGLE.

- CCI_ELEM_SPH_QUAD: The number of nodes is four. The nodes can be enumerated anti-clockwise or clockwise. Degenerated quadrilaterals may be defined if the "north" or the "south pole" of the sphere is a doubly defined node. The north and the south pole must not be an inner point or lie on an edge of the spherical quadrilateral. The edges of the spherical quadrilateral must be a line of "latitude" or "longitude" on the sphere.

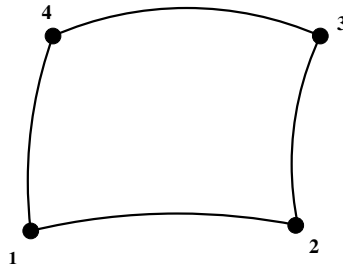


Figure 22: CCI_ELEM_SPH_QUAD.

The sphere or parts of the sphere may also be treated as a "normal" surface in the three-dimensional space and all surface elements may be used to define it.

Based on the nodes specified by the user, MpCCI automatically computes the centers of the elements (open circles). Coupling quantities can be assigned to the nodes by CCI_Put_nodes. CCI_Get_nodes is used to get values located at the nodes. The centers are referred to by CCI_Put_elems and CCI_Get_elems. Further element types are intended to be included in later versions of MpCCI.

nElemIds must be set either to 0 or to nElems. The elements are numbered consecutively if you choose nElemIds = 0. If e.g. the first 10 triangles (CCI_ELEM_TRIANGLE) and thereafter 20 quadrilaterals (CCI_ELEM_QUAD) are specified, then the 11th element is the first quadrilateral. If you choose nElemIds = nElems the elements will be numbered according to elemIds (see also CCI_Def_nodes).

All nodes defined by CCI_Def_nodes must be used in the topology specification calls for a coupling partition. The various elements can be defined by more than one call to CCI_Def_elems. It is the caller's responsibility to ensure that no degenerate elements occur and that different elements do not overlap.

The ordering of array elemNodes is according to the order specified with routine CCI_Def_nodes for array coords.

Example: In the following example we refer to the definitions made in the example of CCI_Def_nodes. With the help of the nodes defined there we build a mesh consisting of three quadrilaterals with the ids 1, 2, 3 and two triangles with the ids 4, 5.

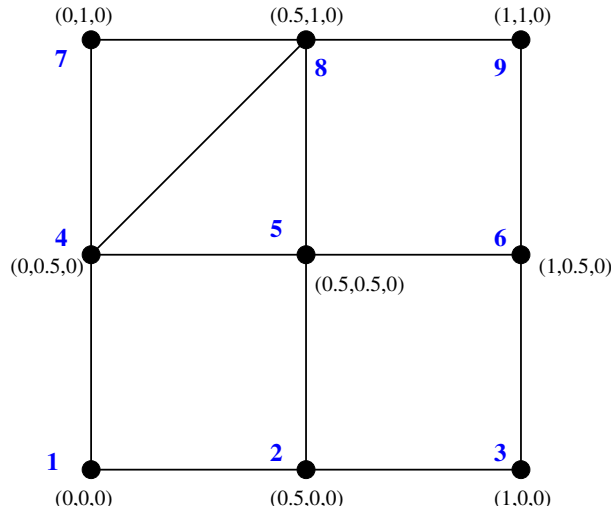


Figure 23: Mesh for the example.

```
/**
 * Variables for CCI_Def_elems:
 */

/* Number of quadrilaterals: */
int nElems1 = 3;
```

```

/* Numbering of the quadrilaterals: 1, 2, 3 */
int nElemIds1      = 0;
int* elemIds1      = 0;
/* Elementtypes ( must be 1 ):                */
int nElemTypes1    = 1;
int elemTypes1[]   = { CCI_ELEM_QUAD };
/* Nodes per quadrilateral:                    */
int nNodesPerElem1[] = { 4 };
/* Node ids of the quadrilaterals:            */
int elemNodes1[]   = { 1, 2, 5, 4, /* quadrilateral 1 */
                      2, 3, 6, 5, /* quadrilateral 2 */
                      5, 6, 9, 8 }; /* quadrilateral 3 */

/* Number of triangles:                        */
int nElems2       = 2;
/* Numbering of the triangles: 4, 5            */
int nElemIds2     = 2; /* or = 0 */
int elemIds2[]    = { 4, 5 };
/* Elementtypes ( must be one ):                */
int nElemTypes2   = 1;
int elemTypes2[]  = { CCI_ELEM_TRIANGLE };
/* Nodes per element:                          */
int nNodesPerElem2[] = { 3 };
/* Node ids of the triangles:                  */
int elemNodes2[]   = { 4, 5, 8, /* triangle 4 */
                      4, 8, 7 }; /* triangle 5 */

...

/* Call of the subroutine to define the quadrilaterals: */
CCI_Def_elems( meshId,
               partId,
               nElems1,
               nElemIds1,
               elemIds1,
               nElemTypes1,
               elemTypes1,
               nNodesPerElem1,
               elemNodes1 );

/* Call of the subroutine to define the triangles:      */
CCI_Def_elems( meshId,

```

```
partId,  
nElems2,  
nElemIds2,  
elemIds2,  
nElemTypes2,  
elemTypes2,  
nNodesPerElem2,  
elemNodes2 );
```

Class	local, non-blocking
Position	behind CCI.Def.nodes

CCI_Def_comm

<code>localCommId</code>	IN	communicator of local code
<code>remoteCodeId</code>	IN	id of the remote code
<code>remoteCommId</code>	IN	communicator of remote code
<code>nQuantityIds</code>	IN	length of array <code>quantityIds</code>
<code>quantityIds</code>	IN	ids as defined in the input file
<code>nLocalMeshIds</code>	IN	length of array <code>localMeshIds</code>
<code>localMeshIds</code>	IN	id of local meshes

```
int CCI_Def_comm( int localCommId, int remoteCodeId,
                  int remoteCommId,
                  int nQuantityIds, const int quantityIds[],
                  int nLocalMeshIds, const int localMeshIds[] )

subroutine CCI_DEF_COMM( localCommId, remoteCodeId, remoteCommId,
                        nQuantityIds, quantityIds, nLocalMeshIds,
                        localMeshIds, ierror )

integer    localCommId, remoteCodeId, remoteCommId, nQuantityIds,
           quantityIds(nQuantityIds), nLocalMeshIds,
           localMeshIds(nLocalMeshIds), ierror
```

By calling `CCI_Def_comm` communicators are built which group processes for later communication (see page 55).

`CCI_Def_comm` has to be called by all local `MpCCI` processes that take part at least at one of the local grids that are defined by `localMeshIds`. Besides these requirements any other local process may call `CCI_Def_comm` as well. This routine collects all the information which is needed to build up the communication but does not interpret it in contrast to `MPI_Comm_create`. It is also possible to specify communicator parameters in the coupling block of the `MpCCI` input file. To get those information `CCI_Comm_info` has to be called before calling `CCI_Def_comm`.

By calling `CCI_Def_comm` an `MpCCI` communicator is created on the calling side. In the `MpCCI` communication routines (e.g. `CCI_Send`, `CCI_Recv`, see pages 61 and 65) this communicator will be addressed by the variable `localCommId`. Every `MpCCI` communicator is finally based on an MPI inter-communicator. Any communication like `CCI_Send` or `CCI_Recv` has to be called by all processes that share the same identifier `localCommId`.

`localCommId` is a local variable in the sense that it is only known on every process within the local code. On the remote side it finds its counterpart in the variable `remoteCommId`. In combination with the variable `localCommId` an `MpCCI` communicator is defined unmistakably as a pair consisting of a communicator ID

and a code ID [`remoteCommId`, `remoteCodeId`] or [`localCommId`, `localCodeId`]. Predefined constants for these identifiers are `CCI_NCODES` and `CCI_MYCODEID`. Note that with every MpCCI communicator a MPI communicator is defined.

If the input file is not used to specify the parameters of the communicators all variables `localCommId` should be documented since these values are required by the remote code when calling `CCI_Def_comm`.

Both variables `localCommId` and `remoteCommId` need to be greater than zero.

MpCCI provides a predefined inter-communicator `CCI_COMM_RCODE` between all MpCCI processes of the coupled codes. `CCI_COMM_RCODE` is an array of length `CCI_NCODES`. Instead of creating a new communicator with `CCI_Def_comm` `CCI_COMM_RCODE` can be used. We recommend the use of the predefined communicator if the codes have one mesh only. If `CCI_COMM_RCODE` is used with specifications in the input file no call of `CCI_Def_comm` is necessary.

In the communicator the `nQuantityIds` quantities which are stored in `quantityIds` may be communicated. The quantities and their identifiers must be defined in the MpCCI input file. Instead of naming all quantities, a wild card `CCI_ANY_QUANTITY` can be used. This constant is also very helpful when coupling more than two codes.

In `localMeshIds` the corresponding mesh identifiers are to be stored. There are two ways to do this. If there is only one mesh involved for all quantities of `quantityIds`, `nlocalMeshIds` may be set to 1 and the (only) entry of `localMeshIds` is the identifier of that mesh. If the quantities are to be communicated on different meshes `nLocalMeshIds` equals `nQuantityIds` and the corresponding mesh identifiers must be stored in `localMeshIds`.

In the MpCCI input file the quantities of the codes which match each other must be specified in the quantities block. Make sure that the local communicator (`localCommId`) and the communicator of the remote code (`remoteCommId`) communicate matching quantities. It is possible to communicate one quantity on different meshes in one communicator. In this case the order of those meshes in `localMeshIds` in the local communicator definition and in the definition of the remote communicator in the remote code are used to put or get the values.

Example: In the following example we define a communicator for a code with the identifier 1. `localCommId` is 112 and the corresponding communicator of the remote code which is code 2 in this case has the identifier 221. The identifier of the quantities are 1, 2, 3, so in the MpCCI input file those quantities must be defined in the code block of code 1. We assume that there is only one mesh involved in the communication of this communicator. The identifier of that mesh is 1.

```
/**
 * Variables for CCI_Def_comm:
 */

/* Identifier of the communicator: */
int localCommId    = 112;
/* Identifier of the remote code: */
int remoteCodeId   = 2;
/* Identifier of the communicator of the remote code: */
int remoteCommId   = 221;
```

```

/* Number of quantities: */
int nQuantityIds = 3;
/* Identifier of the quantities: */
int quantityIds[] = { 1, 2, 3 };
/* Number of mesh identifiers: */
int nLocalMeshIds = 1; /* or = 3 */
/* Identifiers of the meshes: */
int localMeshIds[] = { 1 }; /* or = { 1, 1, 1 } */

/* Call of the subroutine: */
CCI_Def_comm( localCommId,
              remoteCodeId,
              remoteCommId,
              nQuantityIds,
              quantityIds,
              nLocalMeshIds,
              localMeshIds );

```

Class local, non-blocking
Position before CCI_Close_setup

CCI_Def_sync_point

syncPointId	IN	identifier of the synchronization point
nQuantitiesToSend	IN	number of quantities to be sent
quantitiesToSend	IN	array of quantity-identifiers, length <code>nQuantitiesToSend</code>
nMeshIdsToSend	IN	length of the array <code>meshIdsToSend</code> , either 1 or <code>nQuantitiesToSend</code> ,
meshIdsToSend	IN	array of mesh-identifiers, length <code>nMeshIdsToSend</code>
nQuantitiesToRecv	IN	number of quantities to be received
quantitiesToRecv	IN	array of quantity-identifiers, length <code>nQuantitiesToRecv</code>
nMeshIdsToRecv	IN	length of the array <code>meshIdsToRecv</code> , either 1 or <code>nQuantitiesToRecv</code> ,
meshIdsToRecv	IN	array of mesh-identifiers, length <code>nMeshIdsToRecv</code>

```

int CCI_Def_sync_point( int syncPointId,
                        int nQuantitiesToSend, const int quantitiesToSend[],
                        int nMeshIdsToSend, const int meshIdsToSend[],
                        int nQuantitiesToRecv, const int quantitiesToRecv[],
                        int nMeshIdsToRecv, const int meshIdsToRecv[] )

subroutine CCI_DEF_SYNC_POINT( syncPointId, nQuantitiesToSend,
                               quantitiesToSend, nMeshIdsToSend,
                               meshIdsToSend, nQuantitiesToRecv,
                               quantitiesToRecv, nMeshIdsToRecv,
                               meshIdsToRecv, ierror )

integer syncPointId, nQuantitiesToSend, quantitiesToSend( nQuantitiesToSend ),
nMeshIdsToSend, meshIdsToSend( nMeshIdsToSend ), nQuantitiesToRecv,
quantitiesToRecv( nQuantitiesToRecv ), nMeshIdsToRecv,
meshIdsToRecv( nMeshIdsToRecv ), ierror

```

A synchronization point initiates a number of send and receive operations. As it is obvious from the parameter list above the receivers of the send operation and the senders of the receive operations are not to be specified. All synchronization points which are going to be used in the code must be defined in the initialization phase. To start the action see `CCI_Reach_sync_point`, to get the synchronization point information specified via the input file see `CCI_Sync_point.info` (the coupling block of the input file is concerned with synchronization points). Due to the matching criterion from the input file or if no matching criterion was given in the input file due to the numbering of the synchronization points of the different codes MpCCI automatically determines the sending and receiving codes for the quantities. If there is no sender resp. receiver found there will be no error message, so the user has to care about that. But if

there is more than one sender found an error message will be given.

Every code which takes part in a synchronization point communication has to call `CCI_Def_sync_point`. The identifier of the synchronization point (i.e. `syncPointId`) may be different in different codes, the matching of the synchronization points of the codes must then be specified in the coupling block of the input file, otherwise synchronization points with the same identifier match each other.

The identifiers of the quantities the code is going to send (receive) at the synchronization point are stored in the arrays `quantitiesToSend` (`quantitiesToRecv`) of length `nQuantitiesToSend` (`nQuantitiesToRecv`). The corresponding mesh identifiers are stored in the arrays `meshIdsToSend` (`meshIdsToRecv`) of length `nMeshIdsToSend` (`nMeshIdsToRecv`). The value of `nMeshIdsToSend` (`nMeshIdsToRecv`) must be equal to `nQuantitiesToSend` (`nQuantitiesToRecv`) or 1, if the same mesh is valid for all quantities to be sent (received). The values `nQuantitiesToSend` and `nQuantitiesToRecv` must be non-negative and may not exceed `CCI_MAX_NQUANTITIES`. The value 0 is allowed here and implies that there are no send or no receive operations for the code at the current synchronization point.

Example: A synchronization point will be defined without using the `MpCCI` input file. The quantities 2 and 3 on mesh 1 are to be sent and the quantity 1 is to be received on mesh 1. (An example for defining a synchronization point with the help of the input file can be found in in Chap. 4.)

```
/**
 * Variables for CCI_Def_sync_point:
 */

/* Identifier of the synchronization point */
int syncPointId = 1;
/* Number of quantities to be sent: */
int nQuantitiesToSend = 2;
/* Identifier of the quantities to be sent: */
int quantitiesToSend[] = { 2, 3 };
/* Number of the mesh identifiers: */
int nMeshIdsToSend = 1;
/* Identifiers of the meshes: */
int meshIdsToSend[] = { 1 };
/* Number of quantities to be received: */
int nQuantitiesToRecv = 1;
/* Identifier of the quantities to be received: */
int quantitiesToRecv[] = { 1 };
/* Number of the mesh identifiers: */
int nMeshIdsToRecv = 1;
/* Identifiers of the meshes: */
int meshIdsToRecv[] = { 1 };

/* Call of the subroutine: */
CCI_Def_sync_point( syncPointId,
```



```
nQuantitiesToSend,  
quantitiesToSend,  
nMeshIdsToSend,  
meshIdsToSend,  
nQuantitiesToRecv,  
quantitiesToRecv,  
nMeshIdsToRecv,  
meshIdsToRecv );
```

Class	local, non-blocking
Position	before CCI_Close_setup

CCI_Def_search_tags

meshId	IN	identifier of a mesh
partId	IN	identifier of a partition
nSearchTags	IN	number of search tags (length of searchTags)
searchTags	IN	array of search tags

```
int CCI_Def_search_tags( int meshId, int partId, int nSearchTags,
                        const int searchTags[] )

subroutine CCI_DEF_SEARCH_TAGS( meshId, partId, nSearchTags,
                               searchTags, ierror )

integer    meshId, partId, nSearchTags, searchTags(nSearchTags), ierror
```

For the coupled computation the neighborhood of a local and remote grids must be determined. If the subroutine `CCI_Def_search_tags` is not used `MpCCI` determines the neighborhood between all grids and partitions which take part in the coupled computation. `CCI_Def_search_tags` allows the user to decide between which partitions the neighborhood must be computed. The tags stored in **searchTags** are global identifiers. If `CCI_Def_search_tags` is called in the initialization phase `MpCCI` only determines the neighborhood between such partitions which have a search tag (an element of **searchTags**) in common. If `CCI_Def_search_tags` is not called by a code `MpCCI` will interpret that all possible search tags are set.

Class	local, non-blocking
Position	before <code>CCI_Close_setup</code>

CCI_Def_coord_transform

<code>meshId</code>	IN	grid identifier within each code
<code>globalDim</code>	IN	global dimension
<code>type</code>	IN	type of the original coordinate system
<code>equalsZero</code>	IN	Values smaller than this parameter are treated as zero
<code>origin</code>	IN	coordinates of the "old" origin in the "new" coordinate system, length <code>globalDim</code>
<code>unitVectors</code>	IN	unit vectors of the "new" coordinate system length <code>globalDim*globalDim</code>

```
int CCI_Def_coord_transform( const int  meshId, const int  globalDim,
                           const int  type,  const float equalsZero,
                           const void* origin, const void* unitVectors )
```

In coupled computation the different codes may define meshes with different underlying coordinate systems. They can differ in the location of the origin, in the unit vectors and even in the types of the coordinates such as cartesian, spherical or cylindrical coordinates. The coordinate transformation module supports different kinds of coordinate systems in the two- and three-dimensional space. The computations carried out by MpCCI are based on cartesian coordinate systems so that spherical and cylindrical coordinates are always transformed to cartesian coordinates internally.

Doubly defined nodes may occur after the transformation from e.g. a cylindrical coordinate system to a cartesian coordinate system. These nodes are mapped on one node by MpCCI and the element definitions are adjusted: Quadrilaterals e.g. may be deformed to triangles in the cartesian coordinate system.

MpCCI supports cartesian, cylindrical and spherical coordinate systems whereas cylindrical coordinates must not be used in the two-dimensional case (see table 3). Currently there is only a C-interface implemented for coordinate transformations.

<i>Type</i>	<i>two-dimensional space</i>	<i>three-dimensional space</i>	<i>Name</i>
cartesian coordinate system	yes	yes	CCI_COORD_CART
cylindrical coordinate system	no	yes	CCI_COORD_CYL
spherical coordinate system	yes	yes	CCI_COORD_SPH

Table 3: Types of coordinate systems.

The mesh `meshId` is prepared for a coordinate transformation. The parameter `globalDim` must equal 2 for the two-dimensional space and must equal 3 for the three-dimensional space. The parameter `type` determines the type of the original coordinate system:

- **CCI_COORD_CART** Abbreviation for cartesian coordinate systems.
- **CCI_COORD_CYL** Abbreviation for cylindrical coordinate systems. A point is determined by (r, φ, z) where the radius r is a non-negative real number, the height z is a real number, and the angle φ varies between 0 and 2π . The coordinates are transformed to cartesian coordinates (x, y, z) according to

$$\begin{aligned}x &= r \cos \varphi \\y &= r \sin \varphi \\z &= z\end{aligned}$$

in the three-dimensional space. Cylindrical coordinates are not defined for the two-dimensional space. Afterwards **origin** and **unitVectors** are transformed according to the parameters as described below.

- **CCI_COORD_SPH** Abbreviation for spherical coordinate systems. A point is determined by (r, φ, ψ) where the radius r is a non-negative real number, the angle φ varies between 0 and 2π and the angle ψ varies between 0 and π . The coordinates are transformed to cartesian coordinates (x, y, z) according to

$$\begin{aligned}x &= r \cos \varphi \sin \psi \\y &= r \sin \varphi \sin \psi \\z &= r \cos \psi\end{aligned}$$

in the three-dimensional space and according to

$$\begin{aligned}x &= r \cos \varphi \\y &= r \sin \varphi\end{aligned}$$

in the two-dimensional space. Afterwards **origin** and **unitVectors** are transformed according to the parameters as described below.

The coordinates of the origin of the original cartesian coordinate system in the "new" coordinates are given by **origin**. The array **unitVectors** consists of two two-dimensional vectors w_1, w_2 if **globalDim** equals 2 or of three three-dimensional vectors v_1, v_2, v_3 in the three-dimensional case. In the two-dimensional space the unit vectors $(1, 0)$, and $(0, 1)$ of the original cartesian coordinate systems are mapped on w_1, w_2 . In the three-dimensional space the unit vectors $(1, 0, 0)$, $(0, 1, 0)$, and $(0, 0, 1)$ of the original cartesian coordinate systems are mapped on v_1, v_2, v_3 .

The parameter **equalsZero** is used to determine whether two points have the same coordinates after the coordinate transformation.

If a coordinate transformation is defined for a mesh, then the elements of a partition of that mesh must be defined by only one call of the subroutine **CCI_Def_elems**. This is necessary because the type of some elements may change during the transformation.

The vector-valued quantities are transformed to the new coordinate system and re-transformed to the original coordinate system automatically during the communication.

The subroutine `CCI_Def_coord_transform` is only one of two methods to define a coordinate transformation. The `additional` block of the MpCCI input file can also be used.

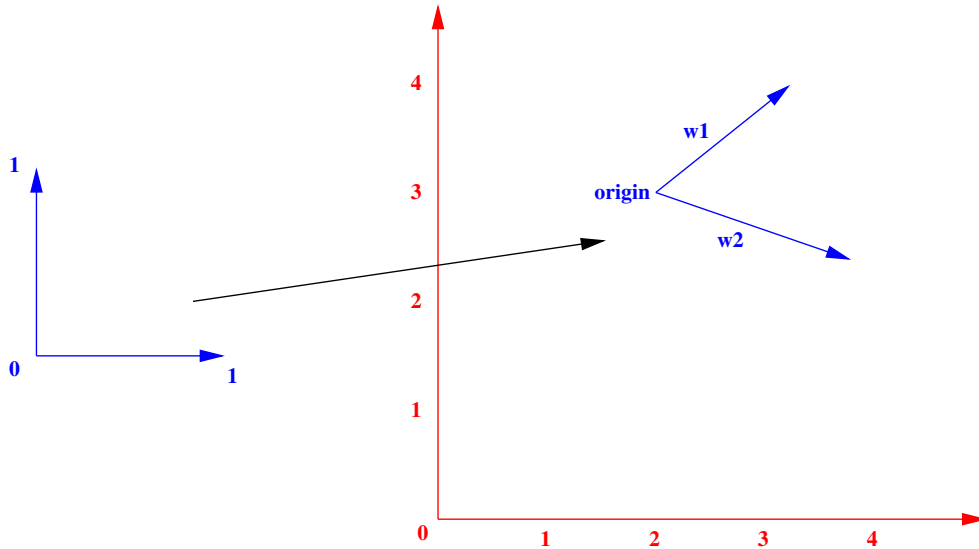


Figure 24: Coordinate transformation

The following example defines a partition with a cartesian coordinate system:

```
int    meshId      = 1;
int    globalDim   = 3;
int    type        = CCI_COORD_CART;
float  equalsZero   = 1.0e-6;
double origin[]    = { 1.0, 1.0, 1.0 };
double unitVectors[] = { 1.0, 1.0, 1.0,
                        1.0, 0.0, 0.0,
                        0.0, 2.0, 0.0 };

CCI_Def_coord_transform( meshId,
                        globalDim,
                        type,
                        equalsZero,
                        origin,
                        unitVectors );
```

Class	local, non-blocking
Position	before node definition on the partitions

CCI_Close_setup

cciProcess IN flag for MpCCI process

int CCI_Close_setup(int cciProcess)
subroutine CCI_CLOSE_SETUP(cciProcess, ierror)
integer cciProcess, ierror

Subroutine CCI_Close_setup terminates the coupling and definition phase. MpCCI processes must call this subroutine with `cciProcess` $\neq 0$, other processes with `cciProcess` = 0. The exact value of `cciProcess` $\neq 0$ is not important.

CCI_Close_setup must be called collectively by all processes of all codes, this means, by all processes that called CCI_Init.

After calling this subroutine, the following MPI communicators are available through `cci.h` (for C), `ccif.h` (for FORTRAN 77) and `ccif90.f90` (for FORTRAN 90):

CCI_MPI_COMM_CWORLD MPI intra-communicator involving all MpCCI processes of all codes.
 MPI intra-communicator involving all MpCCI processes of the current code.

CCI_MPI_COMM_CCODE

Also the following MPI communicators are valid:

CCI_MPI_ICOMM_CCODE Array of length CCI_NCODES of MPI inter-communicators of the MpCCI processes of this code together with those of other codes. It holds that CCI_MPI_ICOMM_CCODE (CCI_ICODE) = MPI_COMM_NULL. For the C interface, it also holds that CCI_MPI_ICOMM_CCODE [0] = MPI_COMM_NULL.

MpCCI provides these communicators to be used directly by a code without risk of interfering with MpCCI-internal communication.

Class	non-local, blocking, collective
Position	following the last definition routine, e.g. CCI_Def_partition, CCI_Def_nodes, CCI_Def_elems, CCI_Def_comm, CCI_Def_sync_point

3.4 Coupling Communication

Communicators

Similar to MPI MpCCI makes use of the communicator concept. In a single MPI application `MPI_COMM_WORLD` includes all processes of the application. Subgroups of processes can be defined by using e.g. `MPI_Comm_create`.

In a coupled application `MPI_COMM_WORLD` will include all processes of all codes that have been started by the `ccirun` command. As a first step towards a coupled application the MPI communicator `MPI_COMM_WORLD` has to be substituted by a communicator `CCI_MPI_COMM_CODE` that groups only processes of each of the individual codes. `CCI_MPI_COMM_WORLD` may serve as a MpCCI communicator to group all processes of the coupled application.

CCI_Put_nodes

meshId	IN	grid identifier within each code
partId	IN	partition identifier of each grid
quantityId	IN	identifier specified in the input file
quantityDim	IN	dimension of quantity
nNodes	IN	number of nodes
nNodeIds	IN	switch node numbering, dimension of array nodeIds
nodeIds	IN	node identifier
valuesDataType	IN	identifier of input data
values	IN	returned data values

```

int CCI_Put_nodes( int meshId, int partId, int quantityId,
                  int quantityDim, int nNodes, int nNodeIds,
                  const int nodeIds[], int valuesDataType,
                  const void* values )

subroutine CCI_PUT_NODES( meshId, partId, quantityId, quantityDim,
                        nNodes, nNodeIds, nodeIds, valuesDataType,
                        values, ierror )

integer    meshId, partId, quantityId, quantityDim, nNodes,
           nNodeIds, nodeIds(nNodeIds), valuesDataType, ierror

<real type> values(quantityDim*nNodes)

```

Subroutine CCI.Put_nodes, a strictly local operation, is used for specifying coupling values at the nodes to MpCCI.

The data items in array **values** are ordered according to the node identifier specified in array **nodeIds**. The specified coupling values are later sent to the remote code by using CCI.Send or CCI.Isend (see pages 61 and 63). According to the type of **values** the parameter **valuesDataType** must be chosen: for the value of variable **valuesDataType** please refer to table 1 and table 2.

The quantity with the identifier **quantityId** must be defined in the relevant code block of the MpCCI input file with the location specification **loc = nodes**. (If **loc = elem** was chosen CCI.Put_elems must be applied.) The dimension **quantityDim** of the quantity also depends on the specification of the code block of the input file. If **scalar** was chosen the value of **quantityDim** must be 1, if **vector** was chosen the value of **quantityDim** must equal the dimension of the vector.

The number of nodes **nNodes** is at most the total number of the nodes of the partition **partId** specified via CCI.Def_nodes. It is possible to call CCI.Put_nodes for a subset of the nodes of a partition.

The value of `nNodeIds` and `nodeIds` may be chosen for grid based quantities according to the enumeration in the description of `CCI_Def_nodes`.

Example: In the description of `CCI_Def_nodes` an example is given which will be extended now. There a mesh is defined with nine nodes numbered from 1 to 9. The mesh with identifier `meshId` consists of one partition `partId`.

```
/**
 * Variables for CCI_Put_nodes:
 */

/* Quantity identifier: This quantity must be specified */
/* in the input file. */
int   quantityId   = 1;
/* Dimension of the quantity: We assume that quantity 1 */
/* is of scalar type. */
int   quantityDim   = 1;
/* Number of nodes to which the subroutine will be applied: */
int   nNodes        = 9;
/* The following choice selects all nodes of the partition. */
int   nNodeIds       = 0;
int*   nodeIds       = 0;
/* Data type for the values: */
int   valuesDataType = CCI_FLOAT;
/* Values at the nodes: */
float values[]       = { 1, 2, 3, 4, 5, 6, 7, 8, 9 };

/* Call of the subroutine:
CCI_Put_nodes( meshId,
               partId,
               quantityId,
               quantityDim,
               nNodes,
               nNodeIds,
               nodeIds,
               valuesDataType,
               values );
```

Class	local, non-blocking
Position	following the definition phase, after <code>CCI_Close_setup</code> and before a corresponding <code>CCI_Send</code> of <code>CCI_Isend</code>

CCI_Put_elems

meshId	IN	grid identifier within each code
partId	IN	partition identifier of each grid
quantityId	IN	quantity identifier specified in the input file
quantityDim	IN	dimension of quantity
nElems	IN	number of elements
nElemIds	IN	switch for element numbering, dimension of array <code>elemIds</code>
elemIds	IN	element identifier
nDataPointsPerElem	IN	length of array <code>dataPointsPerElem</code>
dataPointsPerElem	IN	number of data belonging to one element
valuesDataType	IN	identifier of input data
values	IN	coupling values

```

int CCI_Put_elems( int meshId, int partId, int quantityId, int quantityDim
                  int nElems, int nElemIds, const int elemIds[],
                  int nDataPointsPerElem, const int dataPointsPerElem[],
                  int valuesDataType, const void* values )

subroutine CCI_PUT_ELEMS( meshId, partId, quantityId, quantityDim,
                        nElems, nElemIds, elemIds, nDataPointsPerElem,
                        dataPointsPerElem, valuesDataType, values, ierror )

integer      meshId, partId, quantityId, quantityDim, nElems,
             nElemIds, elemIds(nElemIds), nDataPointsPerElem,
             dataPointsPerElem(nDataPointsPerElem), valuesDataType, ierror

<real type>  values(quantityDim*nPoints)

```

Subroutine `CCI_Put_elems`, which is a strictly local operation, is used for specifying coupling values at the data locations of an element to `MpCCI` which can be communicated by `CCI_Send` or `CCI_Isend`.

The quantity with the identifier `quantityId` must be defined in the relevant code block of the `MpCCI` input file with the location specification `loc = elem`. (If `loc = node` was chosen `CCI_Put_nodes` must be applied.) The `quantityDim` also depends on the specification of the code block of the input file. If `scalar` was chosen the value of `quantityDim` must be 1, if `vector` was chosen the value of `quantityDim` must equal the dimension of the vector.

The number of elements `nElems` is at most the total number of elements specified to `MpCCI` for the coupling partition in different calls of `CCI_Def_elems` for this partition. It is possible to call `CCI_Put_elems` for a subset of the elements of a partition.

The value of `nElemIds` and `elemIds` may be chosen according to the three possibilities of the enumeration listed in the description of `CCI_Def_nodes`.

In the current version `nDataPointsPerElem` must be chosen 1 and the (only) entry of `dataPointsPerElem` must be 1, too. This is due to the interpolations which are currently implemented in `MpCCI`.

For further information about `CCI_Put_elems` see also the description of the subroutine `CCI_Put_nodes` (page 56). `CCI_Put_elems` specifies the coupling values at the element centers to `MpCCI` in the same way as `CCI_Put_nodes` specifies the coupling values at the nodes to `MpCCI`.

Example: We now go on with the example which started in the description of `CCI_Def_nodes` and `CCI_Def_elems`. It should be recalled that five elements were defined for partition `partId` on mesh `meshId`.

```

/**
 * Variables for CCI_Put_elems:
 */

/* Quantity identifier: This quantity must be specified */
/* in the input file. */
int    quantityId      = 7;
/* Dimension of the quantity: In the input file quantity 7 */
/* was defined 'vector'. */
int    quantityDim     = 3;
/* Number of elements which are involved in the call of */
/* the subroutine: Must be smaller or equal the number */
/* of the elements of the partition which is 5. */
int    nElems         = 5;
/* Ids of the elements involved. The following choice */
/* selects all elements of this example. */
int    nElemIds       = 0;
int*   elemIds        = 0;
/* Number of data points per element: There is no other */
/* choice than to use the following. */
int    nDataPointPerElem = 1;
int    dataPointPerElem[] = { 1 };
/* Data type of the values: */
int    valuesDataType  = CCI_FLOAT;
/* Values which are associated with the data points: */
float  values[]        = { 1, 1, 1,      /* vector 1 */
                          2, 2, 2,      /* vector 2 */
                          3, 3, 3,      /* vector 3 */
                          4, 4, 4,      /* vector 4 */
                          5, 5, 5 };    /* vector 5 */

/* Call of the subroutine: */

```

```

CCI_Put_elems( meshId,
               partId,
               quantityId,
               quantityDim,
               nElems,
               nElemIds,
               elemIds,
               nDataPointsPerElem,
               dataPointsPerElem,
               valuesDataType,
               values );

```

Class	local, non-blocking
Position	as for CCI_Put_nodes

CCI_Send

nQuantityIds	IN	length of array quantityIds
quantityIds	IN	ids as defined in the input file
nLocalMeshIds	IN	length of array localMeshIds
localMeshIds	IN	id of local meshes
comm	IN	MpCCI communicator

```
int CCI_Send( int nQuantityIds, const int quantityIds[],
              int nLocalMeshIds, const int localMeshIds[],
              int comm )

subroutine CCI_SEND( nQuantityIds, quantityIds, nLocalMeshIds,
                   localMeshIds, comm, ierror )

integer      nQuantityIds, quantityIds(nQuantityIds),
             nLocalMeshIds, localMeshIds(nLocalMeshIds),
             comm, ierror
```

CCI_Send is a blocking two-side collective operation over the MpCCI processes of the local and remote code (see also subroutine CCI_Isend on page 63).

This subroutine sends the computed coupling values (specified using CCI_Put_nodes and CCI_Put_elems) to a remote code with coupling partitions through which coupling occurs with the calling code.

All MpCCI processes of the calling code must call this routine, even if they have no coupling partitions through which coupling occurs with the other code. This enables MpCCI to do the computations required for keeping the neighborhood information up to date in parallel using all MpCCI processes of a code.

Example: In the description of CCI_Def_nodes a communicator with the identifier 112 was defined. Here it is used to send some quantities.

```
/**
 * Variables for CCI_Send:
 */

/* Number of quantities: */
int nQuantityIds = 3;
/* Identifier of the quantities: */
int quantityIds[] = { 1, 2, 3 };
/* Number of mesh identifier: Here the same mesh for all. */
int nLocalMeshIds = 1;
```

```

/* Identifier of the mesh: */
int localMeshIds[] = { 1 };
/* Identifier of the communicator: */
int comm          = 112;

/* Call of the subroutine: */
CCI_Send( nQuantityIds,
          quantityIds,
          nLocalMeshIds,
          localMeshIds,
          comm );

```

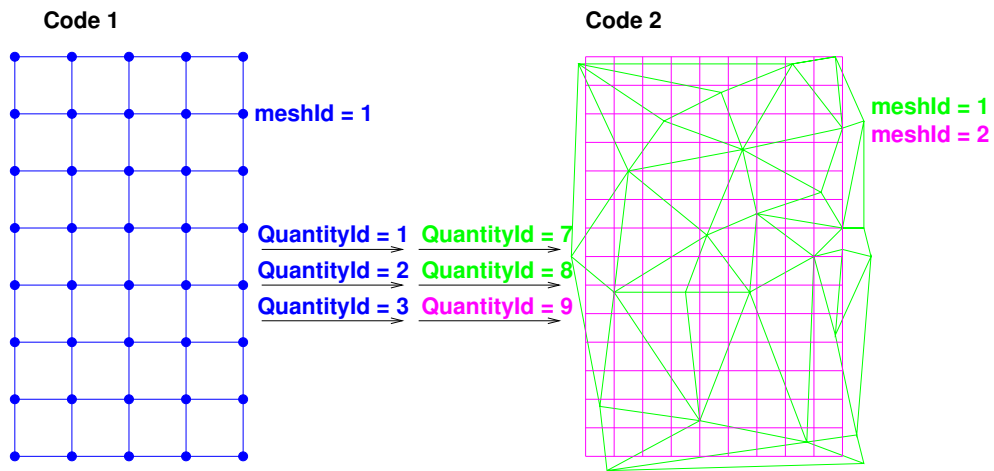


Figure 25: Data transfer between meshes of different codes

Class	non-local, blocking, collective
Position	following the CCI.Put routines

CCI_Isend

nQuantityIds	IN	length of array quantityIds
quantityIds	IN	ids as defined in the input file
nLocalMeshIds	IN	length of array localMeshIds
localMeshIds	IN	id of local meshes
comm	IN	MpCCI communicator
request	OUT	request handle

```
int CCI_Isend( int nQuantityIds, const int quantityIds[],
              int nLocalMeshIds, const int localMeshIds[],
              int comm, CCI_Request* request )

subroutine CCI_ISEND( nQuantityIds, quantityIds, nLocalMeshIds,
                    localMeshIds, comm, request, ierror )

integer      nQuantityIds, quantityIds(nQuantityIds),
            nLocalMeshIds, localMeshIds(nLocalMeshIds),
            comm, request, ierror
```

CCI_Isend is a non-blocking one-side collective operation over the MpCCI processes of the local code (see also subroutine CCI_Send on page 61). Its completion is independent of other processes. After this call, new calls to other MpCCI subroutines may be issued. The subroutine returns a handle to a MpCCI request object, which must be explicitly tested for completion by a code using CCI_Wait (see page 67). If a code does not explicitly test for requests to finish, then in the course of computation many such request objects will be created, and the system can run out of resources.

Class	local, non-blocking, collective
Position	behind a set of CCI_Put routines

CCI_Iprobe

<code>nQuantityIds</code>	IN	length of array <code>quantityIds</code>
<code>quantityIds</code>	IN	ids as defined in the input file
<code>nLocalMeshIds</code>	IN	length of array <code>localMeshIds</code>
<code>localMeshIds</code>	IN	id of local meshes
<code>comm</code>	IN	MpCCI communicator
<code>flag</code>	OUT	boolean
<code>status</code>	OUT	status object

```
int CCI_Iprobe( int nQuantityIds, const int quantityIds[],
               int nLocalMeshIds, const int localMeshIds[],
               int comm, int* flag, CCI_Status* status )

subroutine CCI_IPROBE( nQuantityIds, quantityIds, nLocalMeshIds,
                    localMeshIds, comm, flag, status, ierror )

integer      nQuantityIds, quantityIds(nQuantityIds),
            nLocalMeshIds, localMeshIds(nLocalMeshIds),
            comm, status(CCI_STATUS_SIZE), ierror

logical      flag
```

This subroutine tests whether there is a message matching the first five parameters (see also `CCI_Recv`) which can be received. The value of `flag` will be one (i.e. `true`) if all of the processes of communicator `comm` can receive that message, otherwise `flag` will be zero (i.e. `false`). In the first case the status parameter `status` contains the information of the message as it is described in the description of `CCI_Recv`.

Class	non-local, non-blocking, collective
Position	before a call of <code>CCI_Recv</code>

CCI_Recv

nQuantityIds	IN	length of array quantityIds
quantityIds	IN	ids as defined in the input file
nLocalMeshIds	IN	length of array localMeshIds
localMeshIds	IN	id of local meshes
comm	IN	MpCCI communicator
status	OUT	status object

```
int CCI_Recv( int nQuantityIds, const int quantityIds[],
              int nLocalMeshIds, const int localMeshIds[],
              int comm, CCI_Status* status )

subroutine CCI_RECV( nQuantityIds, quantityIds, nLocalMeshIds,
                   localMeshIds, comm, status, ierror )

integer      nQuantityIds, quantityIds(nQuantityIds),
             nLocalMeshIds, localMeshIds(nLocalMeshIds),
             comm, status(CCI_STATUS_SIZE), ierror
```

CCI_Recv is a blocking two-side collective operation over the MpCCI processes of the local code. This subroutine is used for receiving coupling values from a remote code with coupling partitions through which coupling occurs with the calling code (to be obtained using CCI_Get_nodes and CCI_Get_elems). All MpCCI processes of the calling code must call this routine, even if they have no partitions through which coupling occurs with the other code.

If nQuantityIds is set to a value greater than 0, the next message containing at least the given quantities is received.

In case nQuantityIds equals zero, quantities are received in the same order as they have been sent. The status variable contains the identifier of received quantities.

In C, status is a structure of type CCI_Status that contains fields constants nQuantities, quantityIds and localMeshIds. Thus status.nQuantities contains the number of quantities, status.quantityIds the ids of the received quantities and status.localMeshIds the identifier for the local mesh on which the received quantities have to be placed.

In FORTRAN 77, status is an array of length CCI_STATUS_SIZE, which is set to $1 + 2 \times \text{CCI_MAX_NQUANTITIES}$.⁴ Analogous to the C programming syntax status (CCI_NQUANTITIES) contains the number of quantities, status (CCI_QUANTITYIDS) the identifier of the received quantities and status (CCI_LOCALMESHID) the identification for the local mesh on which the received quantities have to be placed.

⁴With CCI_MAX_NQUANTITIES the maximum number of quantities is specified that can be transferred with MpCCI. In the current implementation this value is set to 30.

Example: The example in the description of `CCI.Send` is extended here. Code 1 has sent three quantities 1, 2, 3. We assume that the matching quantities of code 2 are 7, 8, 9 resp.. This must have been specified in the quantities block of the `MpCCI` input file. In the example of `CCI.Def_comm` the communicator 112 of code 1 matches with the communicator 221 of the remote code 2. We assume that code 2 is prepared to receive the quantities 7, 8, 9 via the communicator 221. So the following is a part of code 2.

```

/**
 * Variables for CCI_Recv:
 */

/* Number of quantities: */
int      nQuantityIds  = 3;          /* or = 0 */
/* Identifiers of the quantities: */
int      quantityIds[] = { 7, 8, 9 };
/* Number of mesh identifiers: Here different meshes. */
int      nLocalMeshIds = 3;
/* Identifiers of the meshes: */
int      localMeshIds[] = { 1, 1, 2 };
/* Identifier of the communicator: */
int      comm          = 221;
/* Status object: */
CCI_Status status;

/* Call of the subroutine:
CCI_Recv( nQuantityIds,
         quantityIds,
         nLocalMeshIds,
         localMeshIds,
         comm,
         &status );

```

Class	non-local, blocking, collective
Position	before <code>CCI_Get_nodes</code> and/or <code>CCI_Get_elems</code>

CCI_Wait

<code>request</code>	INOUT	request handle
<code>status</code>	OUT	status object

```
int CCI_Wait( CCI_Request* request, CCI_Status* status )
```

```
subroutine CCI_WAIT( request, status, ierror )
```

```
integer request, status(CCI_STATUS_SIZE), ierror
```

The function `CCI_Wait` is a one-side collective operation over the `MpCCI` processes of the local code. It blocks all processes until all MPI requests associated with the non-blocking `MpCCI` operation have finished. It is used to complete non-blocking `MpCCI` sends (see page 63) .

Completion of a non-blocking `MpCCI` send indicates that all non-blocking MPI sends associated with this call have terminated. On completion of a non-blocking `MpCCI` send, `MpCCI` performs some final tasks like unpacking and storing received messages, (possibly) interpolation, and deletion of buffers allocated internally. If `CCI_Wait` is never called for requests, then the system can run out of resources during coupled computation.

`CCI_Wait` deallocates the request object pointed to by `request` and sets the request handle to `CCI_REQUEST_NULL`. Information on the completed operation is returned in `status`. `CCI_Wait` with `request = CCI_REQUEST_NULL` is a no-operation.

Class	non-local, blocking
Position	after a corresponding <code>CCI_Isend</code>

CCI_Reach_sync_point

<code>syncPointId</code>	IN	identifier of a synchronization point
<code>status</code>	OUT	status object

```
int CCI_Reach_sync_point ( int syncPointId, CCI_Status* status )  
  
subroutine CCI_REACH_SYNC_POINT( syncPointId, status, ierror )  
  
integer      syncPointId, status( CCI_STATUS_SIZE ), ierror
```

`CCI_Reach_sync_point` performs the communication between those codes which are specified in the `match_syncpt` statement of the coupling block of the input file or if this statement was not used between those codes which have defined a synchronization point with the identifier `syncPointId`. It must therefore be called by all of the concerned codes. The send and receive operations are determined by the synchronization point definition (see `CCI_Def_sync_point`) of the codes.

The communication at the synchronization point with the identifier `syncPointId` is carried out. The identifier `syncPointId` is a code local identifier if the `match_syncpt` statement was used, otherwise the identifier is global.

For the interpretation of the parameter `status` see `CCI_Recv`.

Class	non-local, blocking, collective
Position	between a couple of <code>CCI_Put</code> and <code>CCI_Get</code> routines

CCI_Get_nodes

meshId	IN	grid identifier within each code
partId	IN	partition identifier of each grid
quantityId	IN	identifier specified in the input file
quantityDim	IN	dimension of quantity
nNodes	IN	number of nodes
nNodeIds	IN	switch for node numbering, dimension of array nodeIds
nodeIds	IN	node identifier
valuesDataType	IN	identifier of input data
values	OUT	returned data values
maxnEmptyNodes	IN	maximum number of nodes with(out) data
emptyNodes	OUT	returned nodes containing (no) data
nEmptyNodes	OUT	length of array emptyNodes

```

int CCI_Get_nodes( int meshId, int partId, int quantityId,
                  int quantityDim, int nNodes, int nNodeIds,
                  const int nodeIds[], int valuesDataType,
                  void* values, int maxnEmptyNodes,
                  int emptyNodes[], int* nEmptyNodes )

subroutine CCI_GET_NODES( meshId, partId, quantityId, quantityDim,
                        nNodes, nNodeIds, nodeIds, valuesDataType,
                        values, maxnEmptyNodes, emptyNodes,
                        nEmptyNodes, ierror )

integer      meshId, partId, quantityId, quantityDim, nNodes, nNodeIds,
             nodeIds(nNodeIds), valuesDataType, maxnEmptyNodes,
             emptyNodes(maxnEmptyNodes), nEmptyNodes, ierror

<real type>  values(quantityDim*nNodes)

```

This subroutine **CCI_Get_nodes**, a strictly local operation, is used for obtaining coupling values received from the code by the previous **CCI_Recv**. According to the type of **values** the parameter **valuesDataType** must be chosen: for the value of variable **valuesDataType** please refer to table 1 and table 2.

nNodes is the number of nodes specified for this coupling partition in the call of subroutine **CCI_Def_nodes**.

The array **emptyNodes** indicates the nodes where no new values were obtained or where new values were obtained depending on the sign of **maxnEmptyNodes**. If **maxnEmptyNodes** is positive the MpCCI internal identifiers of the empty nodes are returned and if **maxnEmptyNodes** is negative the internal identifiers of the

non-orphaned nodes are determined. The user is advised to check whether the returned value `nEmptyNodes` exceeds absolute value of the user defined value of `maxnEmptyNodes`. In cases where `nEmptyNodes` $\geq$ `|maxnEmptyNodes|` some information is lost about the location of data gaps. If a coordinate transformation is defined the user defined node identifiers are returned instead of the MpCCI internal identifiers. The orphaned and non-orphaned nodes cannot be retrieved if `overlap = partial` is specified in the `contact` block of the MpCCI input file unless `warn_orphaned_points = on` is specified in the `contact` block or the `additional` block contains a `non_orphaned_objects` statement for the mesh `meshId`.

Example: In the example at the end of the description of `CCI.Recv` among other quantity 7 was received by code 2. Now we apply `CCI.Get_nodes` to obtain the values for quantity 7 on partition 1 of mesh 1. We assume that the number of nodes of that partition is 6 and that all nodes should obtain a value. We assume that in the quantities block for code 2 of the MpCCI input file the location `loc = node` was chosen. Otherwise `CCI.Get_elems` must be applied. Quantity 7 of code 2 matches with quantity 1 of code 1, which was a scalar quantity. This implies that the dimension of quantity 7 of code 2 has dimension 1.

```
/**
 * Variables for CCI_Get_nodes:
 */

/* Identifier of the mesh: */
int meshId = 1;
/* Identifier of the partition: */
int partId = 1;
/* Identifier of the quantity: */
int quantityId = 7;
/* Dimension of the quantity: */
int quantityDim = 1;
/* Number of nodes: */
const int nNodes = 6;
/* Node numbering: All nodes of the partition are chosen. */
int nNodeIds = 0;
int* nodeIds = 0;
/* Data type of the values: */
int valuesDataType = CCI_FLOAT;
/* Data values: */
float values[nNodes];
/* Maximum number of nodes with(out) data: */
int maxnEmptyNodes = 0;
/* Identifiers of nodes with(out) data: */
int* emptyNodes = 0;
/* Number of nodes with(out) data: */
int nEmptyNodes;

/* Call of the subroutine:
```

```

CCI_Get_nodes( meshId,
               partId,
               quantityId,
               quantityDim,
               nNodes,
               nNodeIds,
               nodeIds,
               valuesDataType,
               values,
               maxnEmptyNodes,
               emptyNodes,
               &nEmptyNodes );

```

Class	local, non-blocking
Position	behind CCI_Recv

CCI_Get_elems

meshId	IN	grid identifier within each code
partId	IN	partition identifier of each grid
quantityId	IN	identifier specified in the input file
quantityDim	IN	dimension of quantity
nElems	IN	number of elements
nElemIds	IN	switch for element numbering, dimension of array <code>elemIds</code>
elemIds	IN	element identifier
nDataPointsPerElem	IN	number of elements specified for the given coupling partition
dataPointsPerElem	IN	number of data belonging to one element
valuesDataType	IN	identifier of input data
values	OUT	coupling values
maxnEmptyElems	IN	maximum number of elements without data
emptyElems	OUT	returned elements containing no data
nEmptyElems	OUT	length of array <code>emptyElems</code>

```

int CCI_Get_elems( int meshId, int partId, int quantityId,
                  int quantityDim, int nElems, int nElemIds,
                  const int elemIds[], int nDataPointsPerElem,
                  const int dataPointsPerElem[], int valuesDataType,
                  void* values, int maxnEmptyElems,
                  int emptyElems[], int* nEmptyElems )

subroutine CCI_GET_ELEMS( meshId, partId, quantityId, quantityDim,
                        nElems, nElemIds, elemIds, nDataPointsPerElem,
                        dataPointsPerElem, valuesDataType, values,
                        maxnEmptyElems, emptyElems, nEmptyElems, ierror )

integer      meshId, partId, quantityId, quantityDim, nElems,
             nElemIds, elemIds(nElemIds), nDataPointsPerElem,
             dataPointsPerElem(nDataPointsPerElem), valuesDataType

<real type>  values(quantityDim*nPoints)

integer      maxnEmptyElems, nEmptyElems,
             emptyElems(maxnEmptyElems), ierror

```

Subroutine CCI_Get_elems is a strictly local operation. It retrieves coupling values from element centers

received from the code by the previous CCI_Recv. The number of elements `nElems` is the total number of elements specified for this coupling partition in consecutive calls of `CCI_Def_elems`.

`nEmptyElems` tells the number of elements that did not receive a value and the array `emptyElems` describes these elements.

In the current version `nDataPointsPerElem` must be 1 and the (only) entry of `dataPointsPerElem` must be 1, too. See `CCI_Put_elems` (page 58).

For further information about `CCI_Get_elems` see also the description of the subroutine `CCI_Get_nodes` (page 69). `CCI_Get_elems` is used for retrieving the coupling values at the element centers in the same way as `CCI_Get_nodes` is used for retrieving the coupling values at the nodes.

Example: For all 3 elements of the partition 1 of mesh 1 values for quantity 11 will be obtained. We assume that quantity 11 is a scalar quantity.

```

/**
 * Variables for CCI_Get_elems:
 */

/* Identifier of the mesh: */
int      meshId      = 1;
/* Identifier of the partition: */
int      partId      = 1;
/* Identifier of the quantity: */
int      quantityId   = 11;
/* Dimension of the quantity: */
int      quantityDim  = 1;
/* Number of elements: Here all 3 elements are chosen. */
const int nElems      = 3;
/* Node numbering: Here all elements are chosen. */
int      nElemIds     = 0;
int*     elemIds      = 0;
/* There is no choice for the following settings: */
int      nDataPointsPerElems = 1;
int      dataPointsPerElems[] = { 1 };
/* Data type of the values: */
int      valuesDataType = CCI_FLOAT;
/* Data values: */
float     values[nElems];
/* Currently the following variables have no relevance: */
/* Maximum number of elements without data: */
int      maxnEmptyElems = 0;
/* Identifiers of elements without data: */
int*     emptyElems    = 0;
/* Number of elements without data: */
int      nEmptyElems;

```

```

/* Call of the subroutine:                                     */
CCI_Get_elems( meshId,                                       */
               partId,
               quantityId,
               quantityDim,
               nElems,
               nElemIds,
               elemIds,
               nDataPointsPerElems,
               dataPointsPerElems,
               valuesDataType,
               values,
               maxnEmptyElems,
               emptyElems,
               &nEmptyElems );

```

Class	local, non-blocking
Position	following a CCI.Recv

CCI_Check_convergence

myConvergence	IN	code of local convergence state
globalConvergence	OUT	global convergence state
comm	IN	MpCCI communicator

```
int CCI_Check_convergence( int myConvergence, int *globalConvergence,
                          int comm )
```

```
subroutine CCI_CHECK_CONVERGENCE( myConvergence, globalConvergence,
                                comm, ierror )
```

```
integer    myConvergence, globalConvergence, comm, ierror
```

`CCI_Check_convergence` is a blocking two-side collective operation over the MpCCI processes of the given communicator `comm`. In addition, the constant `CCI_ANY_CODE` can be used as the `comm` argument. In this case, `CCI_Check_convergence` must be called by all codes and exchanges the convergence states.

Each code specifies its local convergence state `myConvergence` by an integer. These integers are exchanged between the codes and the maximum of all integers is returned to each code in `globalConvergence`.

MpCCI defines four constants for the local convergence state with the following meaning:

<code>CCI_CONVERGED</code>	(= 0)	I have converged and could stop here, but I can also do a further coupling step.
<code>CCI_CONTINUE</code>	(= 1)	I have not yet converged and would like to do a further coupling step.
<code>CCI_STOP</code>	(= 2)	I have converged and I must stop here. No further coupling step is possible for me!
<code>CCI_DIVERGED</code>	(= 3)	I have diverged and I must stop here.

This subroutine has to be called to start the remeshing (see Sec. 3.5).

Another important meaning of this subroutine is the creation of coupling steps when writing a MpCCI tracefile. `CCI_Check_convergence` introduces a new coupling step in the tracefile. With a suitable visualisation tool for MpCCI tracefiles the user can observe the data sent and received in the different steps in an animation.

Class	non-local, blocking, collective
Position	following the <code>CCI_Send</code> or <code>CCI_Recv</code> routines to initiate a new coupling step; after the remeshing subroutines to actually start the remeshing

3.5 Remeshing

In the current version MpCCI supports the remeshing in a straight-forward and simple way only. First, changes of the mesh of one code can be defined after a coupling step by subsequent calls of `CCI_Remesh`, `CCI_Def_nodes` and `CCI_Def_elems` or `CCI_Mod_nodes`, and finally `CCI_Close_remesh`. Second, the remote code has to be informed about the changes by calling `CCI_Check_convergence`. This will lead to a new setup of the coupling, i.e. a complete neighborhood search from scratch.

It is planned to introduce a faster remeshing that takes into account the existing neighborhood information into the next version.

CCI_Remesh

```
int CCI_Remesh()

subroutine CCI_REMESH( ierror )

integer    ierror
```

The subroutine `CCI_Remesh` introduces a remeshing operation (mesh deformation or mesh redefinition). Within this operation the nodes of the meshes can be totally redefined with `CCI_Def_nodes` and `CCI_Def_elems` as in the definition phase. If the nodes have new coordinates only and the elements are unchanged, the nodes can be modified with `CCI_Mod_nodes`. At the end of the remeshing operation, a call of `CCI_Close_remesh` is necessary. It is not allowed to call an other MpCCI subroutine in between.

Note that the remeshing is a local operation. In order to inform the remote code about the remeshing the subroutine `CCI_Check_convergence` has to be called after the remeshing definition has been finished with `CCI_Close_remesh`.

Class	local, non-blocking
Position	before <code>CCI_Close_remesh</code>

CCI_Mod_nodes

meshId	IN	grid identifier within each code
partId	IN	partition identifier of each grid
globalDim	IN	dimension of the global coordinate system
nNodes	IN	number of nodes
nNodeIds	IN	switch node numbering, dimension of array <code>nodeIds</code>
nodeIds	IN	node identifier
realType	IN	data type of elements of <code>coords</code>
coords	IN	address of coordinate array

```
int CCI_Mod_nodes( int meshId, int partId, int globalDim, int nNodes,
                  int nNodeIds, const int nodeIds[], int realType,
                  const void* coords )
```

```
subroutine CCI_MOD_NODES( meshId, partId, globalDim, nNodes,
                          nNodeIds, nodeIds, realType,
                          coords, ierror)
```

```
integer          meshId, partId, globalDim, nNodes, nNodeIds,
                  nodeIds(nNodeIds), realType, ierror
```

```
<real type>      coords(globalDim*nNodes)
```

The subroutine `CCI_Mod_nodes` defines the nodes within each partition that are to be modified. For the arguments of the subroutine, please refer to `CCI_Def_nodes`.

Class	local, non-blocking
Position	between <code>CCI_Remesh</code> and <code>CCI_Close_remesh</code>

CCI_Close_remesh

nMeshIds	IN	length of array meshIds
meshIds	IN	id of meshes to remesh
nRemeshingModes	IN	length of array remeshingModes
remeshingMode	IN	modes for remeshing

```
int CCI_Close_remesh( int nMeshIds, const int meshIds[],
                    int nRemeshingModes, const int remeshingMode[])

subroutine CCI_CLOSE_REMESH( nMeshIds, meshIds, nRemeshingModes,
                           remeshingMode, ierror )

integer    nMeshIds, meshIds(nMeshIds), nRemeshingModes,
           remeshingMode(nRemeshingModes), ierror
```

With the subroutine CCI.Close_remesh the remeshing definition is closed. nMeshIds is the length of the array meshIds that holds the numbers of the meshes. The array remeshingMode contains the mode of remeshing for every mesh in meshIds.

In the current version, the only remeshing mode is CCI_TOTAL_REMESH.

Class	local, non-blocking
Position	after CCI_Remesh and CCI_Mod_nodes (or CCI_Def_nodes and CCI_Def_elems), before CCI_Check_convergence

3.6 Termination

CCI_Finalize

```
int CCI_Finalize()

subroutine CCI_FINALIZE( ierror )

integer    ierror
```

Subroutine `CCI_Finalize` terminates the coupling and has to be called collectively by all those processes of any code that called `CCI_Init`. If `MPI_Init` has been called from within `CCI_Init` (see page 24) `MPI_Finalize` will also be called by `CCI_Finalize`. If `MPI_Init` has been called explicitly before invoking `CCI_Init`, `MPI_Finalize` has to be called accordingly **after** `CCI_Finalize`. No `MpCCI` subroutines may be called after this subroutine (even `MPI_Init`).

When the codes have collectively determined that the coupled computation has converged and must be terminated, they must collectively notify `MpCCI` of this by calling subroutine `CCI_Finalize`. This must be done by all processes of all codes (not only the `MpCCI` processes). If the coupled computation has been started by simultaneously invoking a control process `CCI_Finalize` sends a message to the control process telling that the coupled computation has finished, and it cleans up the local `MpCCI` layers. After the control process (when present) has received termination messages from all processes it performs its local cleanup, writes its output files, and calls `MPI_Finalize`.

The subroutine `CCI_Finalize` returns control to the code, which can continue its own termination phase, such as writing output files (different to `CCI_Init`). The order in which the code-local termination proceeds and when `CCI_Finalize` is called is insignificant. A code can just as well first write all its output files and then call `CCI_Finalize`.

Class	non-local, blocking
Position	final <code>MpCCI</code> call

3.7 Control

Sometimes the user finds it necessary to have the opportunity to read in parameter values or any other information that might be helpful to gain some control over the coupled computation. Such functionality is provided by the routine `CCI_Parameter_info`.

CCI_Parameter_info

<code>idString</code>	IN	string to identify input value
<code>valueDataType</code>	IN	datatype of value
<code>value</code>	OUT	return value set in input file
<code>flag</code>	OUT	return flag

```
int CCI_Parameter_info( const char* idString, int valueDataType,  
                        void* value, int* flag )
```

```
subroutine CCI_PARAMETER_INFO( idString, valueDataType,  
                               value, flag, ierror )
```

```
character*(*) idString  
integer       valueDataType, ierror  
<type>       value  
logical      flag
```

In the `MpCCI` input file (see Chap. 4) arbitrary code specific values can be specified which can be copied into the address space of the corresponding program with the help of routine `CCI_Parameter_info`.

Each input value is identified by a character string `idString` of arbitrary length. Note that spaces are not allowed to define the identifier string. The value associated with `idString` can be of all data types named in table 1 and table 2. `valueDataType` has to be set according to these types. In case of the type `CCI_STRING` the argument `value` should be defined with the length `CCI_MAX_STRING_LEN` to ensure that the complete value can be returned.

The return flag indicates whether the requested value has been defined in the input file (`flag = 1`) or not (`flag = 0`).

Class	local
Position	between <code>CCI_Init</code> and <code>CCI_Finalize</code>

CCI_General_info

item	IN	identifier of the type of information
subItem	IN	identifier of the required subItem
maxnIntParams	IN	maximum number of elements of intParams
intParams	OUT	array of information of integer type
nIntParams	OUT	number of elements of intParams
maxnRealParams	IN	maximum number of elements of realParams
realParams	OUT	array of information of real type
nRealParams	OUT	number of elements of realParams
maxStringParamLength	IN	maximum length of stringParam
stringParam	OUT	string
stringParamLength	OUT	length of stringParam

```

int CCI_General_info( int item, int subItem, int maxnIntParams,
                    int intParams[], int* nIntParams,
                    int maxnRealParams, double realParams[],
                    int* nRealParams, int maxStringParamLength,
                    char stringParam[], int* stringParamLength )

subroutine CCI_GENERAL_INFO( item, subItem, maxnIntParams,
                           intParams, nIntParams, maxnRealParams,
                           realParams, nRealParams, maxStringParamLength,
                           stringParam, stringParamLength, ierror )

integer      item, subItem,
             maxnIntParams, intParams( maxnIntParams ), nIntParams,
             maxnRealParams, realParams( maxnRealParams ), nRealParams,
             maxStringParamLength, stringParamLength, ierror
character*(*) stringParam

```

This subroutine extracts the settings of the subitem **subItem** of type **item** from the input file. Currently possible types are information about quantities (**CCI_QUANTITY_INFO**) and about the matching of the grids (**CCI_MATCHING_INFO**) .

For the item **CCI_QUANTITY_INFO** there is a quantity identifier required for **subItem**. **maxnIntParams** should be set to **CCI_QUANTITY_INFO_LENGTH** . In case of a failure **nIntParams** is 0. Otherwise, the result is stored in the arrays **intParams** and **stringParam** which have the following entries:

- **intParams[CCI_QUANTITY_DIM]** contains the dimension.

- `intParams[CCI_QUANTITY_TYPE]` contains the type:
 - `CCI_QUANTITY_TYPE_MESH`,
 - `CCI_QUANTITY_TYPE_FIELD`,
 - `CCI_QUANTITY_TYPE_FLUX` or
 - `CCI_QUANTITY_TYPE_USERDEF`.
- `intParams[CCI_QUANTITY_LOCATION]` contains the location:
 - `CCI_QUANTITY_LOCATION_NODE`,
 - `CCI_QUANTITY_LOCATION_ELEM` or
 - `CCI_QUANTITY_LOCATION_GLOBAL`
- `intParams[CCI_QUANTITY_IPOL]` contains the `ipol` entry of the quantities block of the input file.
- `stringParam` consists of the name of the quantity.

`nIntParams` is set to 4 and `stringParamLength` contains the length of the name of the quantity.

For the item `CCI_MATCHING_INFO` the value of `subItem` is ignored, because no further choices are possible. The result is stored in `intParams` which has the length 1 in this case:

- `intParams[0]` contains the matching information about the grids:
 - `CCI_MATCHING_GRIDS` or
 - `CCI_NON_MATCHING_GRIDS`

Class	local
Position	between <code>CCI_Init</code> and <code>CCI_Finalize</code>

CCI_Sync_point_info

syncPointId	IN	identifier of the synchronization point
maxnQuantToSend	IN	maximum number of quantities to be sent
quantitiesToSend	OUT	array of quantity identifiers, length maxnQuantToSend
meshIdsToSend	OUT	array of the mesh identifiers, length maxnQuantToSend
nQuantitiesToSend	OUT	number of relevant entries in the arrays quantitiesToSend and meshIdsToSend
maxnQuantToRecv	IN	maximum number of quantities to be received
quantitiesToRecv	OUT	array of quantity identifiers, length maxnQuantToRecv
meshIdsToRecv	OUT	array of the mesh identifiers, length maxnQuantToRecv
nQuantitiesToRecv	OUT	number of relevant entries in the arrays quantitiesToRecv and meshIdsToRecv

```

int CCI_Sync_point_info( int syncPointId, int maxnQuantToSend,
                        int quantitiesToSend[], int meshIdsToSend[],
                        int* nQuantitiesToSend, int maxnQuantToRecv,
                        int quantitiesToRecv[], int meshIdsToRecv[],
                        int* nQuantitiesToRecv )

subroutine CCI_SYNC_POINT_INFO( syncPointId, maxnQuantToSend,
                                quantitiesToSend, meshIdsToSend,
                                nQuantitiesToSend, maxnQuantToRecv,
                                quantitiesToRecv, meshIdsToRecv,
                                nQuantitiesToRecv, ierror )

integer    syncPointId, maxnQuantToSend,
            quantitiesToSend( maxnQuantToSend ),
            meshIdsToSend( maxnQuantToSend ),
            nQuantitiesToSend, maxnQuantToRecv,
            quantitiesToRecv( maxnQuantToRecv ),
            meshIdsToRecv( maxnQuantToRecv ),
            nQuantitiesToRecv, ierror

```

CCI_Sync_point_info gets information from the `syncpt` statement of the code and coupling blocks of the

MpCCI input file for a synchronization point. Only the information for the local code is read.

For the synchronization point with the identifier `syncPointId` the communication information is extracted from the input file. The integer values `maxnQuantToSend` and `maxnQuantToRecv` specify the maximal number of quantities which may be sent or received. A natural choice for these parameters is `CCI_MAX_NQUANTITIES` as this value is not a restriction (see `CCI_Def_sync_point`). The identifiers for the quantities which are going to be sent (received) are stored in the array `quantitiesToSend` (`quantitiesToRecv`). The corresponding mesh identifiers are stored in the array `meshIdsToSend` (`meshIdsToRecv`). The number of the relevant entries of the arrays are given by `nQuantitiesToSend` (`nQuantitiesToRecv`).

To define the synchronization point `CCI_Sync_point_info` must be followed by a call of `CCI_Def_sync_point` with the same identifier `syncPointId`.

Class	local, non-blocking
Position	between <code>CCI_Init</code> and <code>CCI_Close_setup</code>

CCI_Comm_info

localCommId	IN	identifier of the communicator
remoteCodeId	OUT	identifier of the remote code
remoteCommId	OUT	identifier of the remote communicator
maxnQuantToSend	IN	maximum number of quantities to be sent
quantitiesToSend	OUT	array of quantity identifiers, length maxnQuantToSend
meshIdsToSend	OUT	array of the mesh identifiers, length maxnQuantToSend
nQuantitiesToSend	OUT	number of relevant entries in the arrays quantitiesToSend and meshIdsToSend
maxnQuantToRecv	IN	maximum number of quantities to be received
quantitiesToRecv	OUT	array of quantity identifiers, length maxnQuantToRecv
meshIdsToRecv	OUT	array of the mesh identifiers, length maxnQuantToRecv
nQuantitiesToRecv	OUT	number of relevant entries in the arrays quantitiesToRecv and meshIdsToRecv

```

int CCI_Comm_info( int localCommId, int* remoteCodeId, int* remoteCommId,
                  int maxnQuantToSend, int quantitiesToSend[],
                  int meshIdsToSend[], int* nQuantitiesToSend,
                  int maxnQuantToRecv, int quantitiesToRecv[],
                  int meshIdsToRecv[], int* nQuantitiesToRecv )

subroutine CCI_COMM_INFO( localCommId, remoteCodeId, remoteCommId,
                          maxnQuantToSend, quantitiesToSend,
                          meshIdsToSend, nQuantitiesToSend,
                          maxnQuantToRecv, quantitiesToRecv,
                          meshIdsToRecv, nQuantitiesToRecv, ierror )

integer      localCommId, remoteCodeId, remoteCommId,
              maxnQuantToSend, quantitiesToSend( maxnQuantToSend ),
              meshIdsToSend( maxnQuantToSend ), nQuantitiesToSend,
              maxnQuantToRecv, quantitiesToRecv( maxnQuantToRecv ),
              meshIdsToRecv( maxnQuantToRecv ), nQuantitiesToRecv, ierror

```

CCI_Comm_info gets information from the `comm` statement of the coupling block of the MpCCI input file

about a communicator. Only the information for the local code is read.

For the communicator with the identifier `localcommId` the communication information is extracted from the input file. The identifiers of the remote code and the remote communicator are to be stored in `remoteCodeId` and `remoteCommId`. The integer values `maxnQuantToSend` and `maxnQuantToRecv` specify the maximum number of quantities which may be sent or received. A natural choice for these parameters is `CCI_MAX_NQUANTITIES` as this value is not a restriction (see `CCI_Def.comm`). The identifiers for the quantities which are going to be sent (received) are stored in the array `quantitiesToSend` (`quantitiesToRecv`). The corresponding mesh identifiers are stored in the array `meshIdsToSend` (`meshIdsToRecv`). The number of the relevant entries of the arrays are given by `nQuantitiesToSend` (`nQuantitiesToRecv`).

To define the communicator `CCI_Comm_info` must be followed by a call of `CCI_Def.comm` with the same identifier `localCommId`. There is only one exception to this. There may be some specifications for the standard communicator `CCI_COMM_RCODE` in the input file. In this case no call of `CCI_Def.comm` is necessary to define the communicator.

Class	local, non-blocking
Position	between <code>CCI_Init</code> and <code>CCI_Close_setup</code>

CCI_Get_id_string

jobId	IN	identifier of the job
maxCodeIdStringLength	IN	maximal length of <code>codeIdString</code>
codeIdString	OUT	id string from the code block
codeIdStringLength	OUT	length of <code>codeIdString</code>
maxJobIdStringLength	IN	maximal length of <code>jobIdString</code>
jobIdString	OUT	id string from the job block
jobIdStringLength	OUT	length of <code>jobIdString</code>

```
int CCI_Get_id_string( int jobId, int maxCodeIdStringLength,
                      char codeIdString[], int* codeIdStringLength,
                      int maxJobIdStringLength, char jobIdString[],
                      int* jobIdStringLength )

subroutine CCI_GET_ID_STRING( jobId, codeIdString, codeIdStringLength,
                             jobIdString, jobIdStringLength, ierror )

character*(*) codeIdString, jobIdString

integer        jobId, codeIdStringLength, jobIdStringLength, ierror
```

The subroutine `CCI_Get_id_string` extracts the id string from the code block and the job block of the MpCCI input file and stores them in `codeIdString` and `jobIdString` resp. In the call of the FORTRAN 77 subroutine the number of parameters is smaller than in the call of the C subroutine: `maxCodeIdStringLength` and `maxJobIdStringLength` are missing.

Class	local, non-blocking
Position	between <code>CCI_Init</code> and <code>CCI_Finalize</code>

CCI_Get_intersection_points

meshId	IN	grid identifier within each code
partId	IN	partition identifier of each grid
remoteJobId	IN	identifier of a remote job
remoteMeshId	IN	identifier of a mesh of the remote job
globalDim	IN	global dimension
nElems	IN	number of elements
nElemIds	IN	length of identifier array beneath
elemIds	IN	identifier of the elements
nRemoteElemsPerLocalElem	OUT	array of numbers of intersecting remote elements
nTotalElemPairs	IN/OUT	length of array beneath
nIntersPointsPerElemPair	OUT	array of numbers of intersection points per element pair
nTotalIntersPoints	IN/OUT	length of the coordinates array
coordsValueType	IN	type of the coordinates array below
intersPointsCoords	OUT	array of numbers of intersection points per element pair

```

int CCI_Get_intersection_points( int meshId, int partId,
                                int remoteJobId, int remoteMeshId,
                                int globalDim, int nElems,
                                int nElemIds, const int elemIds[],
                                int nRemoteElemsPerLocalElem[],
                                int* nTotalElemPairs,
                                int nIntersPointsPerElemPair[],
                                int* nTotalIntersPoints,
                                int coordsValueType,
                                void* intersPointsCoords )

```

The subroutine `CCI_Get_intersection_points` retrieves the intersection points computed in the neighborhood search of type EE to determine the size of the overlap regions of the remote elements w.r.t a local element. Please note that this subroutine is defined in "cciext.h" and has a C-Interface only .

To use this function two specifications in the `MpCCI` input file are necessary:

- In the `contact` block an EE neighborhood search algorithm must be specified for the mesh pair `meshId` for the local job and `remoteMeshId` for the remote job with the identifier `remoteJobId`.
- There must be an `inters_points` statement in the `additional` block for this mesh pair.

`globalDim` is the number of coordinates of a point. The parameters `nElems`, `nElemIds` and `elemIds` have the same meaning as in `CCI_Def_elems`.

According to the type of `intersPointsCoords` the parameter `coordsValueType` must be chosen: for the value of variable `coordsValueType` please refer to table 1 and table 2.

The function `CCI_Get_intersection_points` must be called twice:

- First call to retrieve the length of the arrays `nIntersPointsPerElemPair` and `intersPointsCoords`, here the parameters `nTotalElemPairs` and `nTotalIntersPoints` must have a negative value when the subroutine is called. Both parameters are IN+OUT parameters.
- Second call to fill the data arrays `nTotalElemPairs`, `nIntersPointsPerElemPair` and `intersPointsCoords`. The subroutine must be called with the integer values retrieved in the first call. `nTotalElemPairs` and `nTotalIntersPoints` are IN parameters now.

After the second call of the subroutine the coordinates of the intersection points are stored in `intersPointsCoords`. These intersection points can be grouped into intersection points of element pairs according to the entries of `nIntersPointsPerElemPair` and finally these element pairs can be associated with the local elements with the help of the information stored in `nRemoteElemsPerLocalElem`.

Notice the following items:

- The intersection points are computed by projecting elements of the remote mesh onto the local mesh elements in a surface coupling, so the elements are not intersecting in the intersection points but one projected element and a non-projected element.
- During the neighborhood computation the elements are decomposed into triangles, tetrahedrons or line segments, so all projections, computation of overlap regions etc. are done w.r.t. these three types. The computed intersection points are just approximations to the 'real' intersection points in two ways: the usual numerical approximation and the approximation of an element by decomposing it into triangles, tetrahedrons or line segments (fig. 27).
- The underlying algorithms are implemented to compute the size of the element overlaps and not to determine the minimal number of points to describe the figure. Caused by this the number of intersection points may be bigger than necessary. It is possible that a point on an edge of the intersection figure is retrieved by this subroutine even if this point is not necessary to describe the figure (fig. 26 and 27).

Example: The following very simple meshes are defined:

After the first call of the function with

```
nElems           = 2
globalDim        = 2
nElemIds         = 0
nTotalElemPairs  = -1
nTotalIntersPoints = -1
```

we retrieve the values

```
nTotalElemPairs  = 4
nTotalIntersPoints = 12
```

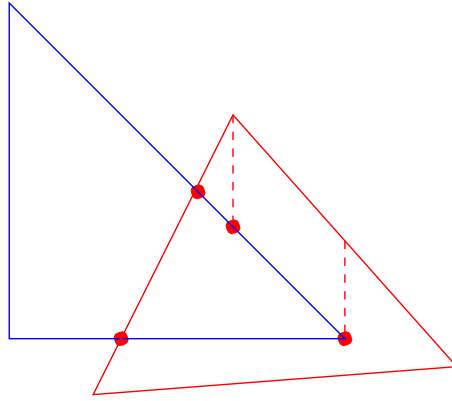


Figure 26: Intersection points.

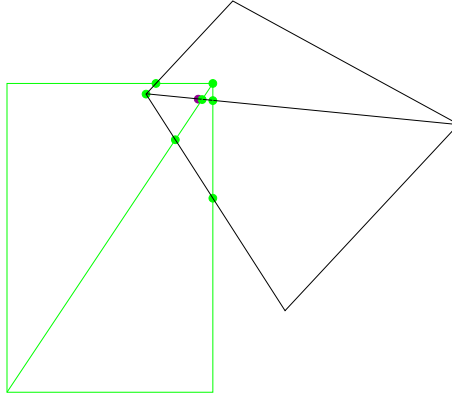


Figure 27: Intersection points. The lilac point is not necessary to describe the intersection figure.

Now it is known the total number of element pairs is 4 and the total number of intersection points is 12. The second call of the subroutine will retrieve for example the following array values:

```
nRemoteElemsPerLocalElem = { 2, 2 }
```

Both elements on the local mesh intersect with two elements on the remote mesh.

```
nIntersPointsPerElemPair = { 3, 3, 3, 3 }
```

Each intersection figure is described by three points.

```
intersPointsCoords      = { 0.0, 0.0, 0.5, 0.5, 0.0, 1.0,
                           0.0, 0.0, 1.0, 0.0, 0.5, 0.5,
```

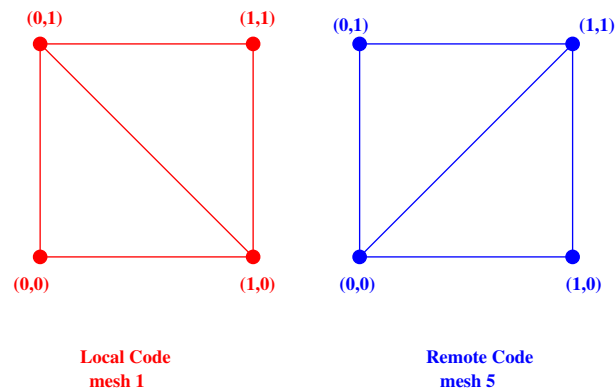


Figure 28: Meshes for the example.

```
0.0, 1.0, 0.5, 0.5, 1.0, 1.0,
1.0, 0.0, 0.5, 0.5, 1.0, 1.0 }
```

The coordinate array depends on which element was defined first and the order of the nodes defining the elements in `CCI_Def_elems`. So the result of the coordinate array may be given in another order than above.

Class	local, non-blocking
Position	after <code>CCI_Close_setup</code>

3.8 Miscellaneous functions

There are a few miscellaneous functions that might be helpful in some situations. These functions are counterparts to the corresponding MPI_ functions. They are listed in a separate section as they do not belong to the MpCCI core functions.

CCI_Abort

<code>commId</code>	IN	id of the communicator - currently unused
<code>errorCode</code>	IN	number of error code
<code>message</code>	IN	message text

```
int CCI_Abort( int commId, int errorCode, char message[] )

subroutine CCI_ABORT( commId, errorCode, message, ierror )

character*(*) message

integer      commId, errorCode, ierror
```

The subroutine CCIAbort prints the string `message` and stops the process by calling MPIAbort. The argument `commId` is currently not used.

Class	local, non-blocking
Position	between CCI.Close_setup and CCI.Finalize

CCI_Wtick

```
double CCI_Wtick()
```

```
double precision CCI_WTICK()
```

The function CCI_Wtick returns the resolution of CCI_Wtime in seconds (as MPI_Wtick).

Class	local, non-blocking
Position	between CCI.Close_setup and CCI.Finalize

CCI_Wtime

```
double CCI_Wtime()
```

```
double precision CCI_WTIME()
```

The function `CCI_Wtime` returns a floating-point number of seconds, representing the elapsed wall-clock time since some time in the past. It enables the user to easily measure the time spent in a certain part of the program (as `MPI_Wtick`). The resolution is given by `CCI_Wtick`.

An example shows the usage of this function:

```
{
    double starttime, endtime;
    starttime = CCI_Wtime();
    ... stuff to be timed ...
    endtime = CCI_Wtime();
    printf("stuff took %f seconds\n", endtime - starttime);
}
```

Class	local, non-blocking
Position	between <code>CCI_Close_setup</code> and <code>CCI_Finalize</code>

3.9 Overview of the MpCCI functions

Initialization and Definition		
CCI_Init	Enroll in the coupled computation.	page 24
CCI_Init_with_id_string	Enroll in the coupled computation.	page 26
CCI_Def_mesh	Define a name for a mesh.	page 27
CCI_Def_partition	Define the partitions of each grid.	page 29
CCI_Def_nodes	Specify the nodes of a coupling partition.	page 31
CCI_Def_elems	Specify the elements of a coupling partition.	page 34
CCI_Def_comm	Define communicators for process groups.	page 43
CCI_Def_sync_point	Define synchronization point.	page 46
CCI_Def_search_tags	Define tags for neighborhood search.	page 49
CCI_Def_coord_transform	Specify a coordinate transformation.	page 50
CCI_Close_setup	Terminate the initialization phase.	page 54
Coupled Computation		
CCI_Put_nodes	Put coupling values node wise.	page 56
CCI_Put_elems	Put coupling values at element centers.	page 58
CCI_Send	Send coupling values.	page 61
CCI_Isend	Send coupling values (non-blocking).	page 63
CCI_Iprobe	Test of incoming messages.	page 64
CCI_Recv	Receive coupling values.	page 65
CCI_Wait	Wait for MpCCI request to finish.	page 67
CCI_Reach_sync_point	Reach synchronization point.	page 68
CCI_Get_nodes	Get coupling values node wise.	page 69
CCI_Get_elems	Get coupling values from element centers.	page 72
CCI_Check_convergence	Check state of convergence.	page 75

Table 4: Overview of the MpCCI functions (1).

Remeshing		
CCI_Remesh	Begin the remeshing definition.	page 76
CCI_Mod_nodes	Specify the nodes to be modified.	page 77
CCI_Close_remesh	Terminate the remeshing definition.	page 78
Termination		
CCI_Finalize	Terminate the coupling.	page 79
Control		
CCI_Parameter_info	Read in input values.	page 80
CCI_General_info	Read in input values.	page 81
CCI_Sync_point_info	Get synchronization point info.	page 83
CCI_Comm_info	Get communicator info.	page 85
CCI_Get_id_string	Get id strings from the input file.	page 87
CCI_Get_intersection_points	Get intersection points.	page 88
Miscellaneous		
CCI_Abort	Abort MpCCI processes.	page 92
CCI_Wtick	Give resolution of wtime.	page 93
CCI_Wtime	Give elapsed wall-clock time.	page 94

Table 5: Overview of the MpCCI functions (2).

4 MpCCI Input File

In the coupled computation different types of information have to be provided to the MpCCI library. At first, MpCCI needs to know the application codes that take part in the coupling, their default environment and the way to start them. Of great importance are the different coupling quantities the codes can send and receive, and how the codes refer to them. The information so far can be described as “code configuration file” that should be shipped with the distribution of the application codes. Then, an important information is the mapping between the coupling quantities that allows each code to use different numbers for the same physical quantity. Finally there are a lot of settings for MpCCI itself, like

- parameters for algorithms, e.g. search algorithms or mesh quality checks,
- switches controlling the output level for debugging, or
- information about the coupling algorithm.

MpCCI provides defaults for almost all settings thus reducing the work for the user. On the other hand all such default values can be changed via the input file. In order to keep the input file readable and taking into account that settings of the codes can be supplied by the code owners, an input file can consist of several files. An effective mechanism for collecting input informations distributed to different files is described in Sec. 4.11.

Note that all MpCCI input files are case-sensitive. After reading the user input file, the control process, when active, checks if the coupling is possible.

In the following description **<key>** indicates mandatory keywords or identifiers and **[args]** indicates optional arguments. A loose syntax description will be used. For instance, **< key₁ >, . . . , < key_n >** denotes a comma-separated list of an arbitrary number of keywords.

4.1 Structure of the Input File

The MpCCI input file consists of several blocks:

- code block(s)
- quantities block(s)
- control block
- contact block
- switches block
- jobs block
- parameters block
- coupling block

- additional block

Each of these blocks starts with a main keyword and ends with the word **end**:

```
<MainKeyword>
...
end
```

The different blocks in the MpCCI input file are described below. Some of these blocks are mandatory, others are optional. The blocks must occur in the input file in the same ordering as listed above.

When working with the MpCCI GUI an other block named **addcontrol** ("additional control") might appear at the end of the input file. This block is generated by MpCCI and used internally only.

4.2 Code block

code *codename*

```
% coupling quantities of the code, at least one quantity:
quantity1(no = no1, loc = node|elem, dim = scalar|vector, type = field|flux);
quantity2(no = no2, loc = node|elem, dim = scalar|vector, type = field|flux);
quantity3(no = no3, loc = node|elem, dim = scalar|vector| < n >, type = userdef,
        ipol = max|min|sum|prod);
quantity4(no = no4, loc = global, dim = scalar|vector, ipol = max|min|sum|prod);
mesh(no = no5);
quantity6(no = no6, loc = < .. >, dim = < .. >, type = < .. >, SI = a * Q + b);
% information about startup
env(env_var_name) = "env_var_value";
pre_exec = "pre_exec_string";
exec = "exec_string";
post_exec = "post_exec_string";
% additional command line arguments for the exec command:
add_exec_args = "add_exec_arguments";
% id string for this code:
id_string = "id_string";
% enumeration of the parameters for the current code block (for MpCCI GUI only):
parameter parameter_name1( type = string|integer|real|logical,
        default = default_value1, doc = "doc_string1" );
parameter parameter_name2( type = string|integer|real|logical,
        default = default_value2 );
parameter parameter_name3;
% enumeration of the synchronization points (for MpCCI GUI only):
syncpt no : send( send_quantity_id1, send_quantity_id2, send_quantity_id_n ),
```

```

recv( recv_quantity_id1, recv_quantity_id2, recv_quantity_id_n );
end

```

For each code in the coupling there must be a **code** block, i.e. in a coupling of three different codes, there will be three code blocks, whereas in a coupling of the same code one code block is sufficient.

The **code** block describes what quantities there are, their name, number, location in the grid, dimension and type. At present, five types of quantities are supported, namely **field**, **flux**, **userdef**, **global** and **mesh**.

For the first two types (**field** or **flux**), an arbitrary name must be specified together with a number, the location in the grid (**node** or **elem**) with the attribute **loc** (in former MpCCI versions: attribute **where**), the dimension and the type. Type **field** is recommended for quantities located at the nodes on the sending side, i.e. sending values from nodes to nodes or elements. Type **flux** is recommended for quantities located at the nodes on the receiving side, i.e. sending values from nodes or elements to nodes.

For the user-defined quantities (type **userdef**) it is possible to specify extended dimensions besides **scalar** and **vector**, i.e. the integer names the dimensions. In this case MpCCI needs the interpolation method (**ipol**). An other type are **global** quantities that are defined with the attribute **loc**. Currently, there are four possibilities for the interpolation of global quantities: **max**, **min**, **sum** and **prod**.

Finally the type **mesh** is recognized by MpCCI and handled in a special way, therefore, only the numbers by which a code refers to them must be specified.

The **SI** setting supports transformations of quantities when two codes refer to the same quantity in different units. For example, code A may use "mm" (millimeter) for the quantity "displacement", whereas code B uses "m" (meter). With **SI = 1000 * Q** in the quantity setting of code A, the values of both codes will have the same unit. The **SI** statement has the general form **SI = a * Q + b** where **a** and **b** are real numbers and **a** is nonzero. It means that the relation between the quantity value w.r.t. the unit used by the code (**Q**) and the quantity value w.r.t. the corresponding reference unit (**SI**) is given by the equation **SI = a * Q + b**. Typically, the reference unit will be the corresponding **SI** unit, but this is not necessary: it is only necessary that all codes use the same reference unit for a given quantity. The linear shift **b** in the equation is provided for a quantity such as "temperature" with several common units (Celsius and Fahrenheit).

For all types of quantities holds: different numbers must be used for different quantities. Negative numbers are allowed.

The instructions **env**, **pre_exec**, **exec**, **post_exec** specify the default environment for a code, the *startup_commands*, and are identical in meaning to those fields in the **jobs** block of the input file. This means, that one can specify general code-specific but job-independent parameters for a code in the **code** block instead of repeating them for each computation in latter blocks. This will make the MpCCI input file much more readable.

By using **add_exec_args** additional command line arguments can be given for the **exec** command of each process which belongs to the code *codename*. In **id_string** an identifier for the code can be stored.

The parameter informations are only relevant for the MpCCI GUI, so the description is omitted here.

The information concerning the synchronization points of the code are only relevant for the MpCCI GUI.

Example:

```

code struc
% Define coupling quantities of the code:
temperature    (no = 7,    loc = node, dim = scalar, type = field);
pressure        (no = 13,   loc = elem, dim = scalar, type = field);
geschwindigkeit(no = 100,  loc = node, dim = vector, type = field);
kraft           (no = 1000, loc = elem, dim = vector, type = flux);
mesh            (no = 101);

% Define specific environment variables for the code
% (may be overwritten in the "jobs" block):
env(STRUC_LICENSE_DIR) = "/home/struc/license_dir";
exec = "/home/struc/bin/struc";
% Id string for this code:
id_string = "FEM_CODE 2.4.5";
end

```

4.3 Quantities block

```

quantities
% how do the quantities correspond to each other?
code_i.quantity_j = code_k.quantity_l;
% enumeration of the parameters for all code blocks (for MpCCI GUI only):
parameter parameter_name_1( default = default_value_1, doc = "doc_string_1" );
parameter parameter_name_2( default = default_value_2 );
parameter parameter_name_3;
end

```

This section describes how the coupling quantities defined for each code in the `code` blocks match. The instruction `codei.quantityj = codek.quantityl` specifies that *quantity_j* of *code_i* is the same physical quantity as *quantity_l* of code *code_k*.

This mechanism enables new codes of different application areas to use MpCCI, because the knowledge about quantities and their type is not present in the MpCCI source. Also, this mechanism enables different codes to use their own local numbering of coupling quantities, independent of the other ones. MpCCI will perform the mapping of coupling types internally.

For instance, if we couple a fluid code requiring the mesh velocity with a structures code requiring forces, then the `quantities` section looks like:

```

quantities
% how do the quantities correspond to each other?

```

```

    cfd.velocity = struc.geschwindigkeit;
    cfd.force    = struc.kraft;
end

```

In the above example, the structures code happens to use the German words for velocity and force, so that it has to be specified that 'velocity = geschwindigkeit'. An equality of two quantities is valid when they have the same type and dimension. They may reside at different positions of the computational grids, however.

Currently there are some restrictions due to the interpolation algorithms: The above matching conditions for the quantities are necessary for the communication between the codes. Values for quantities of type **flux** must be sent to quantities with the location specification **loc = node** and values for quantities of type **field** must be located at the nodes on the sending side.

The parameter informations are only relevant for the MpCCI GUI, so the description is omitted here.

4.4 Control block

```

control
    tracefile ( name = "file_name", trace-par2, ..., trace-parn );
    check_mesh_quality ( on|off );
    mesh_quality_nodes ( tol =  $\epsilon$  );
    mesh_quality_quad ( quad-par1, ..., quad-parn );
    mesh_quality_quad8 ( quad8-par1, ..., quad8-parn );
    mesh_quality_quad9 ( quad9-par1, ..., quad9-parn );
    mesh_quality_triangle ( triangle-par1, ..., triangle-parn );
    mesh_quality_triangle6 ( triangle6-par1, ..., triangle6-parn );
    skip_element_checks = false|true;
    timeout = seconds;
    allow_core_dump ( off|on );
    restart_file_name = "restart_file_name_stem";
    restart_file_access = none|read|write;
    contact_detection ( contact-par1, ..., contact-parm );
end

```

The optional control block specifies some general steering parameters and some parameters for algorithms used by MpCCI internally. It can be used to specify:

tracefile: Information for the tracefile, currently written in format HDF5⁵. The tracefile can be used to visualize the coupled computation with a suitable visualization tool. For more information about tracefiles, please refer to the MpCCI SDK Tutorial. The **name** of the tracefile can be specified through

⁵Hierarchical Data Format, <http://hdf.ncsa.uiuc.edu>

```

control
    tracefile ( name = "test.ccv" );
end

```

If no name or an empty name is specified, no tracefile will be written. Besides the name, there is a number of switches (value: **on** or **off**) for the tracefile to be generated. With **trace_comm_values** MpCCI will write, in addition to the geometry data, communication values into the tracefile, default is **on**. The switches **trace_mesh_values** and **trace_mesh_change_values** will lead communication values of coupling type "mesh" and "mesh_change" into the tracefile (unless **trace_comm_values** is switched off). With **close_after_writing** MpCCI will close the tracefile each time after writing data into it, default is **off**. And finally, the switch **implicit_coupling_steps** tells MpCCI whether or not to generate a new coupling step in the tracefile during a call of the subroutine **CCI_Check_convergence**, default is **on**. The parameter **format** tells MpCCI the format of the tracefile to be written. **hdf5** is the current HDF format and the default setting. It is recommended for large tracefiles. The older HDF format **hdf4** is provided for compatibility reason to former MpCCI version.

```

control
    tracefile( name = "test.ccv",
               close_after_writing = on,           % default: off
               trace_comm_values = on,             % default: on
               trace_mesh_values = on,             % default: off
               trace_mesh_change_values = off,     % default: on
               implicit_coupling_steps = on,       % default: on
               format = hdf4                      % default: hdf5
            );
end

```

check_mesh_quality: This statements turns on (or off) the mesh quality checks during **CCI.Close_setup**. The checks give a warning if there are badly shaped elements or other inconsistencies of the mesh. By default it is turned on and some parameters of the mesh tests can be set in the subsequent items

- **mesh_quality_nodes**,
- **mesh_quality_triangle**,
- **mesh_quality_triangle6**,
- **mesh_quality_quad**,
- **mesh_quality_quad8** and
- **mesh_quality_quad9**.

The setting **check_mesh_quality(on)** may cause an MpCCI internal problem if the coordinates of the nodes are big. This problem can be avoided by turning the checks off or by raising the tolerance with **mesh_quality_nodes**. The checks are described in more detail in Appendix A.

mesh_quality_nodes: MpCCI checks for each coupling partition if the specified nodes are all different. That is, no two different nodes v_1 and v_2 may be the same within a given tolerance `tol`, i.e. $\|v_1 - v_2\| < tol$. This statement can be used to specify an alternate tolerance for this check. The default is $tol = 1^{-7}$.

```
mesh_quality_nodes( tol = 1e-5 );           % default 1e-7
```

mesh_quality_quad: Specify parameters for the mesh quality checks for quadrilaterals.

```
mesh_quality_quad(taper      = 0.4,          % default 0.35
                  aspect_ratio = 30.0,        % default 39.0
                  skew        = 60.0,        % default 60.0 degrees
                  warp        = 5.0 );        % default 5.0 degrees
```

The checks are described in more detail in Appendix A.1.

mesh_quality_triangle: Specify parameters for the mesh quality checks for triangles.

```
mesh_quality_triangle( aspect_ratio = 30.0,    % default 39.0
                      skew          = 45.0 );  % default 45.0 degrees
```

The checks are described in more detail in Appendix A.2.

mesh_quality_quad8, mesh_quality_quad9, mesh_quality_triangle6: Specify parameters for the mesh quality checks for CCI_ELEM_QUAD8, CCI_ELEM_QUAD9 and CCI_ELEM_TRIANGLE6. The syntax and the meaning of the parameters are the same for these three element types, so we only give an example for CCI_ELEM_QUAD8.

```
mesh_quality_quad8(taper      = 0.4,          % default 0.35
                  angle       = 150.0,        % default 150.0 degrees
                  area        = 0.0 );        % default 0.0
```

The checks are described in more detail in Appendix A.3.

There are two more surface element types in MpCCI: CCI_ELEM_PENTAGON and CCI_ELEM_HEXAGON. For interpolation these elements are decomposed into triangles (CCI_ELEM_PENTAGON) or into quadrilaterals (CCI_ELEM_HEXAGON). The element checks are therefore done on the elements of the decomposition. So the parameters set in `mesh_quality_triangle` and `mesh_quality_quad` are also relevant for CCI_ELEM_PENTAGON and CCI_ELEM_HEXAGON (see also Appendix A.4). The shape of the faces of the three-dimensional elements are checked in the same way.

skip_element_checks: If this parameter is set no element checks will be performed. The default value is *false*.

timeout: The `timeout` statement specifies the maximum allowed time in seconds between two consecutive messages arriving at the control process. This is a primitive method for dead-lock detection. The default is 0, which means that this mechanism is not active.

allow_core_dump: If this parameter is set a core dump will be written in case of abnormal termination. The default value is *off*.

restart_file_name and **restart_file_access:** With the `restart file` settings the neighborhood relations can be stored in files to be reused and skip the neighborhood search which can be very time-consuming.

The `restart_file_name` defines the base of the file names: if `'xxx'` is set, `'xxx.1'` will keep the data of process 1, `'xxx.2'` that of process 2, etc. If `restart_file_access` is set to `write`, the neighborhood relations will be written to files. Then, `restart_file_access` can be set to `read` in order to reuse this information. Default setting is `none`.

contact_detection: This statement could be used to specify attributes of the grids and to define the contact detection algorithm and matching criterion to use in former MpCCI versions. The user should use the contact block instead. See Sec. 4.5 about the contact block for the informations about the parameters.

4.5 Contact block

This optional block is used to specify attributes of the grids and to define the contact detection algorithm and matching criterion to use. This block substitutes the `contact_detection` statement of the control block.

contact

```

    type = PP|PE|EE|nearest_neighbor;
    overlap = full|partial;
    warn_orphaned_points = on|off;
    user_performs_contact_search = false|true;
    global_dim = n;
    coupling_dim = m;
    pre_contact_search = search_alg (
        search_par_name_1 = search_par_value_1,
        ...
        search_par_name_n = search_par_value_n
    );
    matching_criterion = match_crit_name (
        match_par_name_1 = match_par_value_1,
        ...
        match_par_name_n = match_par_value_n
    );
    default_alg (
        default_alg_par_name_1 = default_alg_par_value_1,
        ...
        default_alg_par_name_n = default_alg_par_value_n
    );
    alg alg_name_1 (
        alg_par_name_1 = alg_par_value_1,
        ...
        alg_par_name_n = alg_par_value_n
    );

```

```

mesh_pair ( job_name1/meshIdm, job_name2/meshIdn ) : alg_name1;
end

```

type: Default setting for the grid **type** is **PE** that switches to the standard interpolation for non-matching grids. For matching grids, i.e. the nodes of one grid are (almost) the same of the other, the **type** must be **PP**. With setting the **type** to **EE** for non-matching grids the user can switch to a neighborhood search and an interpolation based on intersection methods. If **type** is set to **nearest_neighbor** interpolation and neighborhood search is substituted by the nearest neighbor algorithm: For each node of the local grid the node of the remote grid is computed which has the minimal distance from the local grid point. The value on this node will then be transferred to the node of the remote grid. The value of **skip_element_checks** of the control block is set to *off* automatically.

overlap: The **overlap** region can be **partial** or **full**, which is the default value. If the **overlap** is **partial**, no brute force search for orphaned nodes or elements is executed. This can be useful for grid type **EE**.

warn_orphaned_points: The setting **warn_orphaned_points** defines if the test for orphaned points should be made. A warning will be given if orphaned points are found. The test can take some time, so if the overlap is partial this switch should be set to **off**. Default is **on**.

user_performs_contact_search: The setting **user_performs_contact_search** enables the user to switch to a user-defined contact search algorithm, default is **false**. The definition of a contact search algorithm by the user is only possible if the interface "user def. interpolation" is installed. This interface is not available by now.

global_dim: The setting defines the global dimension, default is 3 (see 2.4).

coupling_dim: The setting defines the dimension of the coupling region, default is 2 (see 2.4). For instance, for a coupling of two codes in the 2-dimensional space (along a 1-dimensional coupling line), the contact block has to be added with the following two statements:

```

contact
...
global_dim    = 2;
coupling_dim = 1;
...
end

```

pre_contact_search and matching_criterion: For the **pre_contact_search** algorithm and for the **matching_criterion**, parameters may be specified in brackets. If no parameters for the pre-contact search algorithm or matching criterion are specified, then defaults are used. The specification of pre-contact search algorithms and matching criteria may also be omitted entirely; in this case, default for **pre_contact_search** algorithm is **bucket** and for the **matching_criterion** is **minimal_distance**. The possible settings are described in more detail in Chap. 5.

default_alg: The item **default_alg** allows the user to redefine the internal defaults, e.g. **type**, **global_dim** or **coupling_dim**, for the neighborhood search algorithm. There are four different combinations possible for the global and the coupling dimension: (3,2), (3,3), (2,2), and (2,1).

alg and mesh_pair: The item `alg` allows the user to define specific neighborhood search algorithms. Settings of the contact block, e.g. `type`, `global_dim` or `pre_contact_search`, can be summarized with a name for the algorithm. By giving this name in the `mesh_pair` definition the defined neighborhood search algorithm can be applied for a pair of meshes each identified by `jobname` and `meshId`. Thus, different neighborhood search algorithms can be used within a coupling of codes with different meshes.

In the following example an algorithm “mySpherContactAlg” is defined with different settings for the contact search of grids consisting of spherical quadrilaterals as one can see at the `pre_contact_search` item (`bucket_sphquad`). The dimension items are left out as they are properly defaulted. Then, the algorithm defined is set to be used for the neighborhood search between mesh 2 of “myJob1” and mesh 4 of “Jobname2”.

```

contact
...
alg mySpherContactAlg (
  type = EE,
  overlap = partial,
  warn_orphaned_points = off,
  pre_contact_search =
    bucket_sphquad(
      bbox_expansion = 2,
      bucket_expansion = 1.5
    ),
  matching_criterion =
    intersection_sphquad(
      rejection = 0.01,
      equals_zero = 2.0e-6,
      perform_tests = false,
      scale_result = false,
    ),
);
mesh_pair ( myJob1/2, Jobname2/4 ) : mySpherContactAlg;
end

```

4.6 Switches block

```

switches
  switch_namei = valuei;
...
group_namej :

```

```

        switch_namek = valuek;
        ...
    ...
end

```

The optional switches block defines the general behavior of MpCCI. For this purpose we distinguish between groups and switches. A simple example of a switch is the integer switch `output_level`, which determines how much output MpCCI writes:

```

switches
    output_level = 3;
end

```

The following switches are available:

output_level	0	no output, default value
	1	print message when entering and leaving a MpCCI subroutine
	2	print input and output arrays as well
	3	print maximal output
debug_level	0	no debugging output, default value
	1	produce debugging output

A group identifies a part of the program with a given name. The group mechanism is used to specify for which part of MpCCI the switch holds. For instance,

```

switches
    cci_put_nodes:
        output_level = 3;
end

```

specifies that an output level of 3 must be used for MpCCI subroutine `CCI_Put_nodes`.

Some groups are subdivided into subgroups so that it is also possible to define a switch value for several parts of MpCCI at the same time. It is also possible to do this, and define a special switch value for one subgroup which overwrites the default. For instance,

```

switches
    cci_subroutines:
        output_level = 3;
    cci_put_nodes:
        output_level = 2;
end

```

specifies that for all MpCCI subroutines an output level of 3 must be used, except for MpCCI subroutine `CCI_Put_nodes`, for which an output level of 2 is used.

The ordering in which switches are specified for groups and subgroups is irrelevant. So, in the above example, it would have been equivalent to specify the switch `output_level` for group `cci_put_nodes` before that for group `cci_subroutines` instead. More than one switch may be specified for a group. The construct '*group_name*:' merely specifies that all subsequent switches apply to the indicated group.

If a switch is specified at the start of a `switches` block, then it applies to all groups, except those for which another value has been specified. So for example,

```
switches
  output_level = 3;
  cci_put_nodes:
    output_level = 2;
end
```

specifies a default output level of 3 for all groups. The output level of MpCCI subroutine `CCI.Put_nodes` will be 2.

The following group names are available:

```
cci_subroutines
  cci_init
  cci_init_with_id_string
  cci_setup
    cci_def_partition
    cci_def_nodes
    cci_def_elems
    cci_def_comm
    cci_def_sync_point
    cci_def_search_tags
    cci_close_setup
  cci_putget
    cci_put_nodes
    cci_get_nodes
    cci_put_elems
    cci_get_elems
  cci_sendrecv
    cci_send
    cci_isend
    cci_recv
    cci_reach_sync_point
    cci_wait
    cci_iprobe
    cci_check_convergence
  cci_remeshing
    cci_remesh
    cci_mod_nodes
    cci_close_remesh
  cci_info
    cci_parameter_info
    cci_general_info
    cci_sync_point_info
```

```

    cci_comm_info
    cci_get_id_string
    cci_finalize
ccv_tracefile
neighborhood_search
    linear_search
    bucket_search
timing

```

The indentation in this list shows the hierarchy of groups and subgroups. The group `cci_subroutines` comprises some artificial subgroups and all MpCCI subroutines starting with the CCI_ prefix. On the other hand, the subgroup `cci_send` belongs to the subgroup `cci_sendrecv` and of course to the group `cci_subroutines`. An other top-level group is `neighborhood_search` that refers to information about the used search algorithms. Finally, a switch `> 0` for the group `timing` will generate runtime information of the subroutines and output it in `CCIFinalize`.

The interpretation of a switch by a group depends on the group. Some parts of MpCCI may not use a particular switch at all.

4.7 Jobs block

This section specifies the codes that participate in the coupled computation, and how they are to be started. We use the term 'job' instead of 'code' to allow for multiple occurrences of a single code in the coupled computation.

To allow for any type of programming model, different processes of the parallel code may be started in different ways, which is indicated by the process groups. For an SPMD code there is only one process group. For a host-node parallelization there are two: one containing only one process called the host, and one containing the node processes. A job may also consist of more than two process groups.

A simple example for a jobs block is:

```

jobs
    struc1 = struc(
        nprocs = 10,
        env(STRUC_JOB) = "flap"
    );

    ... % job2

end

```

The process group name `struc` after the arbitrary jobname "struc1" identifies the code information already specified for `struc` in a `code` block. Since the startup information for the code is already known, one does not need to repeat it. It is allowed, however, to override the defaults specified in the `code` block.

For each process group the following specifications can be given: `pwd`, `pre_exec`, `exec`, `post_exec`, `nprocs` and `env`.

The `pwd` describes in what directory the job will run, and this directory will be set by the `CCI_Init` call. The execution string `exec` specifies the command needed to start up the process. The command will be searched for first in the directory indicated by `pwd`, and then in the directories in the normal search path for executables. The `nprocs` command describes how many processes are to be started.

Environment variables can be specified optionally using the `env` construct. This design allows for the case where different process groups or codes require different values for the same environment variable.

Optionally one can use the `pre_exec` and `post_exec` scripts to be executed before and after the coupled computation runs. The scripts should not read standard input. When the script is run, only the working directory is set as specified in the input file. Other environmental settings are not in effect (system defaults). Because the scripts are run prior to starting the coupled computation, any environment variables that are set are not in effect anymore during computation.

The `pre_exec` and `post_exec` facilities are useful for example when a code requires its input files to be distributed to the local disks of the hosts where it will run. By doing so, I/O is optimized because different processes will perform I/O on different hard disks. The scripts are executed with the specified arguments. The name of a host list file for this process group is added by the startup procedure as final argument. The tasks specified by the `pre_exec` and `post_exec` must be done during the MpCCI startup procedure since it is unknown beforehand on what hosts the process group will run, and because with some scheduling systems (e.g. EASY), one does not have access to the remote hosts until the processes are available.

The specified environment variables and working directories are in effect after the `CCI_Init` call. The default environment variables and working directories depend on the platform and MPI implementation used.

An example job specification that shows all settings is as follows:

jobs

```
% Define environment variables which are set in all jobs
% (may be overwritten in each job):
env( ENVVAR5 ) = "value5";
env( ENVVAR6 ) = "value6";

% Define the directory of the MpCCI distribution (for license mechanism).
% Valid for all jobs (unless overwritten there). Overwrites the default
% value from the "addcontrol" block.
cci_home_dir = "/home/user/mpcci";

% Job 1 consists of two groups of processes: 1 host and 3 nodes.
% It executes the code "CodeName1" (this must be the name of one of
% the code blocks above).
JobName1 = CodeName1 (
```

```

    % Define startup information and environment variables
    % for all process groups of this job. They overwrite
    % the defaults from the corresponding code block "CodeName1"
    % and from the above environment variables which are set
    % in all jobs, but the settings here may be overwritten
    % in each separate process group below.
    pwd      = ".",
    pre_exec  = "ex3.pre arg1",
    exec      = "ex3Binary1",
    post_exec = "ex3.post arg1 arg2",
    nprocs    = 1,
    env( ENVVAR7 ) = "value7",
    env( ENVVAR8 ) = "value8"
)
: host (
    % Define startup information and environment variables
    % for the 'host' process group. This overwrites all
    % previous defaults.
    pwd      = ".",
    pre_exec  = "host.pre arg1",
    exec      = "host",
    post_exec = "host.post arg1 arg2",
    nprocs    = 1,
    env( ENVVAR7 ) = "value7a",
    env( ENVVAR8 ) = "value8a"
)
+ nodes (
    pwd      = ".",
    pre_exec  = "ex3.pre arg1",
    exec      = "node",
    post_exec = "ex3.post arg1 arg2",
    nprocs    = 2,
    env( ENVVAR7 ) = "value7b",
    env( ENVVAR8 ) = "value8b"
);
% Additional process groups would be allowed here.
% You would add them as further summands.

% Job 2 consists of one process group only and executes the code
% "CodeName2".
JobName2 = CodeName2 (
    pwd      = ".",

```



```

        exec    = "ex3Binary2",
        nprocs = 1
    );

```

end

The above example defines two jobs `JobName1` and `JobName2`. The first job uses a host-node parallelization with 1 host process and 2 node processes. The second job is an SPMD job with 1 process.

The code ids given to jobs during coupled computation are determined by the order in which they were specified in the MpCCI input file. The first job gets code id 1, and the i -th job gets id i .

MpCCI needs the home directory of the MpCCI distribution to check the validity of the license. Usually the name is available through the environment variable `CCI_HOME`. With `cci_home_dir` the search for the license file can be redirected.

4.8 Parameters block

The parameters block is optional. It enables the user to provide values of different kind (integer, float, string) to the codes involved. A value is identified by a string given in the form

```

parameter
    stringi = valuei;
end

```

The codes can query these values via `CCI_Parameter_info`. An example illustrates the use of the parameters block:

parameters

```

% Default settings for all jobs:
temperature = 12.34;
pressure    = 67.89;
nSteps      = 1000;

% Specific settings for job 1 (overwrite the above defaults):
JobName1( temperature = 1234.5678,
          pressure     = 5678.12,
          mode         = "sequential",
          additional    = 5678 );

% Specific settings for job 2:
JobName2( temperature = 1200.0 );

```

end

In this example the integer value 1000 is assigned to the parameter "nSteps". This setting is valid and accessible by the MpCCI subroutine `CCI_Parameter_info` for all codes. Then, there is a specification of values for `JobName1`. The jobname has to be the one already used in the previous `jobs` block. This statement defines four parameters. The value of parameter "temperature" is changed to 1234.5678 for `JobName1` because this setting is more specific. The parameter "mode" is of type "string", so the value has to be written in quotation-marks. `JobName2` will operate with a temperature of 1200.0 . The names of parameters are not allowed to contain any blanks.

Note: the MpCCI GUI will present the parameters only if they are defined in the code or quantities blocks before!

4.9 Coupling block

The coupling block is optional. In this block synchronization points and communicators can be defined and the matching of the definitions can be specified.

coupling

% Definition of synchronization points:

syncpt *job_name1*(*sync_point_id1*) :

 send(*send_quantity_id1/send_mesh_id1, send_quantity_id2/send_mesh_id2, ...*),

 recv(*recv_quantity_id1/recv_mesh_id1, recv_quantity_id2/recv_mesh_id2, ...*);

% matching of the synchronization points of the different jobs:

match_syncpt

job_name1(*sync_point_id1, sync_point_id2, ...*)

 = *job_name2*(*sync_point_ida, sync_point_idb, ...*)

 = *job_name3*(*sync_point_idA, sync_point_idB, ...*)

 = ... ;

% Redefinition of the predefined communicators:

comm *job_name1*(with *job_name2*) :

 send(*send_quantity_id1/send_mesh_id1, send_quantity_id2/send_mesh_id2, ...*),

 recv(*recv_quantity_id1/recv_mesh_id1, recv_quantity_id2/recv_mesh_id2, ...*);

% Switch for the redefined communicators:

set_communicators = true;

% User-defined communicators:

comm *job_name1*(*comm_id1*) :

 send(*send_quantity_id1/send_mesh_id1, send_quantity_id2/send_mesh_id2, ...*),

 recv(*recv_quantity_id1/recv_mesh_id1, recv_quantity_id2/recv_mesh_id2, ...*);

comm *job_name1*(*comm_id2*) :

 sendrecv(*sendrecv_quantity_ida/mesh_ida, sendrecv_quantity_idb/mesh_idb, ...*);

```

% matching of communicators of the different jobs:
match_comm job_name1( comm_id1 ) = job_name2( comm_ida );
end

```

Synchronization points

A synchronization point initiates a number of send and receive operations defined here for all codes which take part in a synchronization point. This has to be done for all these codes in the following way:

```

syncpt JobName( SyncPointId ) : send( quantityId1/meshId1, quantityId2/meshId2, ...),
                                recv( QuantityId1/MeshId1, QuantityId2/MeshId2, ...);

```

It is possible to define a synchronization point containing e.g. a number of send (or receive) operations only. In this case, the `recv` part (or the `send` part) can be left out. The meshes can be named by the `meshIds` or by `idStrings` defined with `CCI.Def_mesh`. If the codes have just one mesh, the mesh specification can be omitted.

To define the synchronization point with id `syncPointId` for job `jobName` the subroutines `CCI.Sync_point.info( syncPointId, ...)` and `CCI.Def_sync_point( syncPointId, ...)` must be called in the initialization phase of the code. To start the action `CCI.Reach_sync_point( syncPointId, ...)` must be called (after `CCI.Close_setup`). Instead of defining a synchronization point in the input file the user may call `CCI.Def_sync_point` in the initialization part of each code concerned.

If synchronization points are used for communication one can specify which synchronization points of the codes match each other even if they are not defined in the coupling block. If no `match_syncpt` is given although there are synchronization points defined it is assumed that matching synchronization points of the different codes have the same identifier. If the matching is done explicitly this must then be done with the help of the `match_syncpt` statement:

```

match_syncpt  jobName1( syncPointId1, syncPointId2, ...)
              = jobName2( idSyncPoint1, idSyncPoint2, ...)
              = jobName3( syncPoint1, syncPoint2, ...)
              = ... ;

```

This statement is interpreted as follows: The synchronization point `syncPointId1` of code `jobName1` matches with the synchronization point `idSyncPoint1` of code `jobName2` etc. and the synchronization point `syncPointId2` of code `jobName1` matches with the synchronization point `idSyncPoint2` of code `jobName2` etc.. **Caution:** Every code from the job block must be in that enumeration! It is not allowed to repeat the `match_syncpt` statement, i.e. every synchronization point you want to use in your code must be in that enumeration, even those you are going to define in your code without the `syncpt` statement described above. If a code does not take part in a synchronization communication you must write '-'

in the enumeration instead of a synchronization point id. For example: `syncPoint1` in the enumeration belonging to the code `jobName3` may be equal to '-' instead of being an integer.

If synchronization points are defined in the code and no matching criterion is given in the input file then matching synchronization points of different codes must have the same identifier.

We give an example for synchronization point definition via the coupling block:

coupling

```
syncpt JobName1( 2 ) : send( temperature1/1, temperature2/1 ),
                        recv( temperature0/2 );
syncpt JobName2( 1 ) : send( Temperatur0/1 ),
                        recv( Temperatur1/1, Temperatur2/2 );

match_syncpt JobName1( 2, 5 )
              = JobName2( 1, 6 );
```

end

In this example the quantities `temperature1` and `temperature2` will be sent by code `JobName1` at the synchronization point with the id 2. The `meshId` is 1 in both cases. In addition, the code `JobName1` may receive the quantity `temperature0` on mesh 2 at the same synchronization point 2.

Caution: No check will be done whether there is a sending code for a corresponding quantity due to the quantities block. That means that it is up to the user to decide whether code `JobName1` will actually receive values for `temperature0`. The user can be sure that the code does not receive values, which were not intended to be received. Whether there is a receiving code for the quantities which are sent will not be checked either. But, if there is more than one code sending a quantity an error will occur. For there are only two codes involved in the coupled computation of the example above no error message will be given.

For code `JobName2` the following communication is defined at synchronization point 1: `Temperatur0` on mesh 1 will be sent and `Temperatur1` on mesh 1 and `Temperatur2` on mesh 2 may be received. How the quantities of the codes match each other is up to the quantities block.

In this example there are two synchronization points defined for each code: synchronization points 2 and 5 for code `JobName1` and synchronization point 1 and 6 for code `JobName2`. For there is only one synchronization point defined for each code in the input file the other synchronization points can only be defined in the codes themselves.

The `match_syncpt` statement implies that the synchronization point 2 of code `JobName1` matches the synchronization point 1 of code `JobName2`, and the synchronization point 5 of code `JobName1` matches the synchronization point 6 of code `JobName2`.

Communicators

For the definition of communicators we distinguish between two types of communicators:

- predefined communicators (i.e. `CCI_COMM_RCODE`)

- user-defined communicators.

For the specification of the predefined communicators we use the following type of the `comm` statement:

```
comm JobName1( with JobName2 ) : send( quantityId1/meshId1, quantityId2/meshId2, ...),
                                recv( QuantityId1/MeshId1, QuantityId2/MeshId2, ...);
```

It is possible to define a communicator for send (or receive) operations only. In this case, the `recv` part (or the `send` part) can be left out. Moreover, a '*' can be used instead of naming all quantities. This wild card is also very helpful when coupling more than two codes. As another easement, it is possible to name the mesh by the meshIds or by idStrings defined with `CCI_Def_mesh`. If the codes have just one mesh, the mesh specification can be omitted. Finally, the send and the receive part can be combined with `sendrecv` if the quantities are exactly the same for both directions.

The `set_communicators` statement tells MpCCI that the predefined communicators `CCI_COMM_RCODE` are redefined according to the above settings, i.e. `CCI_Def_comm` is called implicitly during `CCI_Close_setup` for these communicators (unless the code itself has called `CCI_Def_comm`).

```
set_communicators = true;
```

For user-defined communicators the `comm` statement must be used in the following way:

```
comm JobName1( CommId1 ) : send( quantityId1/meshId1, quantityId2/meshId2, ...),
                                recv( QuantityId1/MeshId1, QuantityId2/MeshId2, ...);
```

In both cases the communicators are then prepared to send the quantities specified in the `send` part of the statement and prepared to receive the quantities specified in the `recv` part of the statement.

The possibilities mentioned above for naming the quantities easily can be used for user-defined communicators too. The following definition for instance covers the case that the send and the receive quantities are equal:

```
comm JobName1( CommId2 ) : sendrecv( quantityIdA/meshA, quantityIdB/meshB, ...);
```

To define a user-defined communicator with id `commId` for job `jobName` the subroutines `CCI_Comm_info( commId, ...)` and `CCI_Def_comm( commId, ...)` must be called in the initialization phase of the code. For predefined communicators this can be omitted.

For user-defined communicators it is necessary to specify how the defined communicators match each other (the predefined communicators provide this information by definition). We use the `match_comm` statement to store the matching information:

```

match_comm      jobName1( commId1 )
                = jobName2( commIdA );

```

The parameters `commId*` must be identifiers of user-defined communicators. This statement is interpreted as follows: The communicator `commId1` of job `jobName1` matches with the communicator `commIdA` of job `jobName2`. The `match_comm` must be called separately for every pair of user-defined communicators which are specified in the input file.

Communicators and synchronization points can also be defined in the initialization phase of the codes with the help of the subroutines `CCI_Def_comm` and `CCI_Def_sync_point`.

4.10 Additional block

The `additional` block is optional. It can be used to define and initiate some additional features for the coupling. In the current MpCCI version the following features are available:

- Coordinate transformation
- Computation of the intersection points between the elements (or their projection) of two meshes.
- Determination of the non-orphaned nodes and elements of the meshes

The `additional` block of the MpCCI input file has the following form:

```

additional
% Coordinate transformation:
coord_system job_name(
    mesh_id = mesh_id,
    global_dim = dim,
    type = cart|cyl|sph,
    equals_zero = double_value,
    origin = (x,y,z)|(x,y),
    unit_vectors = (v1,v2,v3)|(w1,w2)
);
% Compute intersection points:
inters_points ( job_id1/mesh_id1, job_id2/mesh_id2);
% (Non-)orphaned nodes and elements:
non_orphaned_objects ( job_id/mesh_id );
end

```

Coordinate transformation

For a mesh of a certain code a coordinate transformation for the node coordinates can be defined. For the parameters are the same as in the MpCCI subroutine CCI_Def_coord_transform a description is left out here. A coordinate transformation for job JobName1 for mesh 1 can be defined in the following way:

```
coord_system    JobName1 (  mesh_id      = 1,
                             global_dim   = 3,
                             type         = cart,
                             equals_zero  = 1.0e-6,
                             origin       = ( 0.5, 0.5, 0.5),
                             unit_vectors = ( 0.5, 0.0, 0.0,
                                              0.0, 0.5, 0.0,
                                              0.0, 0.0, 1.0 ) );
```

Intersection points

The subroutine CCI.Get.intersection_points must not be used without the `inters_points` statement in the `additional` block. For mesh pairs for which an EE neighborhood algorithm was defined an `inters_points` statement may be defined.

```
inters_points ( JobName1/1, JobName2/1 );
```

This statement holds for both meshes, so getting intersection points is prepared for both meshes specified here. In the current version one mesh must not compute the intersection points with more than five remote meshes.

Non-orphaned objects

If the identifier of the non-orphaned nodes or elements of a mesh should be given in CCI.Get.nodes or CCI.Get.elems the `non_orphaned_objects` statement must be used to prepare this. This statement can also be used to get the identifiers of the orphaned nodes and elements. This holds for `partial` and `full` overlaps (see `contact` block). The `non_orphaned_objects` statement overwrites the `warn_orphaned_points` statement of the control block.

If the user-defined node identifiers shall be returned in the call of CCI.Get.nodes in addition a coordinate transformation must be defined otherwise the MpCCI internal identifiers (i.e. the position in the coordinate array in the call of CCI.Def.nodes) is returned there. Please remember that all elements must be defined in one call of CCI.Def.elems if a coordinate transformation is defined.

```
coord_system    JobName2 (  mesh_id      = 1,
                             global_dim   = 3,
                             type         = cart,
                             equals_zero  = 1.0e-6,
                             origin       = ( 0.0, 0.0, 0.0 ),
                             unit_vectors = ( 1.0, 0.0, 0.0,
                                              0.0, 1.0, 0.0,
                                              0.0, 0.0, 1.0 ) );

non_orphaned_objects ( JobName2/1 );
```

4.11 Include Mechanism

Typically the different input blocks described above that refer to the configuration of a code are located in different files, which have to be merged into one using an include mechanism: In the MpCCI input file, one includes the required configuration files. The MpCCI environment variable `CCI_PATH` is the search path used for finding the configuration files.

A typical MpCCI input file for a coupling between a structure code (`struc`) and a fluid code (`cf`d) will look like this:

```
include "struc-config.cci";           % specific settings for struc
include "cf-d-config.cci";           % specific settings for cf-d
include "struc-cf-d-config.cci";      % how do the quantities of
                                     % struc and cf-d match

% first block of the input file defined by the user
control
...
end

% second block of the input file defined by the user
... etc.
```

In this example, `struc-config.cci` and `cf-d-config.cci` describe the code configuration files for code `struc` and `cf`d, and `struc-cf-d-config.cci` describes additional information for the coupling of `struc` and `cf`d. The latter will include a definition of how coupling quantities used by the different codes match.

5 Neighborhood Search

This chapter explains the basic algorithms needed to realize the transfer of the coupling quantities. To implement this transfer correctly, we need to know exactly how the contact surfaces fit together (see e.g. Fig. 6 on page 16). Based on the grid information provided by the user the **MpCCI** task is to determine the necessary communication path between pairs of processes of different codes, and to establish interpolating coupling quantities.

The basic problem is to determine how the coupling regions fit together. For this purpose, we start from two in general separate definitions of the coupling surface: for example one by a fluid and one by a solid code. It is the users' responsibility to make sure that both are specified in the same global coordinate system. This neighborhood information is important because it determines in what way communication between processes of different codes will take place. The results from neighborhood computation must be such that they can be used for interpolating coupling quantities. The difficulties of the neighborhood search are that coupling regions could be distributed over several partitions of several processes and that initial distances are allowed between the coupling surfaces of the different codes.

5.1 Relationships between geometric components

Each interpolation type in **MpCCI** needs a special kind of relation between the geometric components of different meshes. These relations are the result of the neighborhood search and can be set in the **MpCCI** input file by the user. We distinguish between three kinds of relations: Point-element relations for standard interpolation, element-elements relations for mappings based on intersection algorithms and point-point relations for matching grids and nearest neighbor interpolation. Geometric relationships are connected with one pair of meshes. For different mesh pairs different geometric relationships can be computed.

<i>Type</i>	<i>Abbreviation</i>	<i>Additional Result</i>	<i>Interpolation or grid type</i>
point-element	PE	local coordinates of the point	standard
element-elements	EE	size of intersection areas	intersection
point-point	PP	— (not yet integrated into this context, separate solution)	matching grids nearest neighbor

Table 6: Types of geometric relationships.

5.1.1 Point-Element Relationships (PE)

To applicate standard interpolation for each node of one grid one suited element of the other grid must be computed. In addition the local coordinates of the point w.r.t. the chosen element are gained while the neighborhood computation takes place. For this purpose a criterion to reject or accept pairs of elements and points must be used. For surface coupling a preceeding projection must be carried out because in general the target point q will not lie exactly in an element of the source surface.

In MpCCI the neighborhood search is done in two steps: First a number of elements is computed most likely to contain the point or the projection of the point. This will be done in the pre-contact search. In the second step the local coordinates of the point w.r.t. those elements will be computed and according to a matching criterion the best match will be chosen among the result of the pre-contact search.

In this context a point may be a node of an element or its midpoint.

5.1.2 Element-Elements Relationships (EE)

The mapping methods based on the intersection algorithm are for communication of data between the elements of a local and a remote mesh. Therefore in the neighborhood search an element-elements relationship is computed. For each element of one mesh several elements of the other mesh are chosen. In this case the matching criterion is based on the fact whether the elements of the different meshes have common points. In case of surface coupling a projection must precede as is mentioned in the last subsection. The result of the neighborhood search for one element E are the remote elements which have a non-empty intersection with E and the size of the resulting overlap regions.

5.1.3 Point-Point Relationships (PP)

Point-point relations are used in case of matching grids.

This relation is also base of the nearest neighbor algorithm which simply needs the number of the nearest node of the remote mesh to a local node. No further informations such as barycentric coordinates are necessary for the nearest neighbor interpolation. In the current version of MpCCI the nearest neighbor interpolation is not yet integrated in the context of PP-relations and must be set in the input file separately.

5.2 Search Algorithms

The algorithms needed to find the pairs of elements and points are search algorithms. The most straightforward of such an algorithm is to compare all pairs of points and elements (linear search), like for example in [?]. However, as the following example shows, this may be inefficient in comparison with the time needed to solve the problem on a subregion. Take for example a cubic subregion with a uniform grid of $n_V = l \times l \times l$ nodes. Then for explicit solvers and for the most efficient implicit solvers the computing time on that subregion will be

$$t_{\text{subregion}} = \mathcal{O}(n_V). \quad (1)$$

A single face of this cubic region has size $n_A = l \times l$ nodes. If we now assume both the number of points and elements to be $\mathcal{O}(n_A)$, the computational cost of the linear search algorithm becomes

$$t_{\text{search}} = \mathcal{O}(n_A^2) = \mathcal{O}(l^4) = \mathcal{O}(n_V^{\frac{4}{3}}). \quad (2)$$

Comparing equation 2 with 1, we see that the amount of time needed for the linear search algorithm may increase faster with problem size than that for the solution of a problem on a subregion. Therefore, better search algorithms are needed. This aspect becomes even more relevant when we have to consider moving meshes or adaptive grids, where neighborhood relations are not fixed for the integration time.

The more efficient search algorithms are based on restricting the search region by exploiting the geometric nature of the data (elements and points). Here we mention only some promising algorithms without going into detail. Efficient search methods are oct-tree search algorithms [??], bucket algorithms [???], and vectorized advancing front search neighbour-to-neighbour algorithms [?]. The oct-tree and bucket algorithms are the most promising. The advancing front algorithm can be highly efficient but depends on detailed knowledge of the topology. This solution is therefore not attractive as a general solution to be provided by MpCCI.

Experiences with these search algorithms in fluid-structure interaction computations [?] indicate that these advanced search algorithms may fail in some cases so that it is necessary to switch between different search algorithms. For additional robustness one of these algorithms should be a linear search algorithm. Furthermore, to obtain the best fit, a local neighborhood search should be performed once an element has been found. This also enables us to deal efficiently with small mesh changes in which an update of the neighborhood computation is wanted instead of a total recomputation.

5.3 Pre-contact Search Algorithms and Matching Criteria

The basic search method for matching and non-matching grids implemented by MpCCI consists of four steps:

1. **pre-contact search:** MpCCI generates pairs of points and elements (non-matching grids) and pairs of points (matching grids) that are located close to each other.
2. **selection:** MpCCI applies a detailed matching criterion to all pairs that were generated in step 1 and selects the best matches.
3. **brute force search:** If desired (i.e. if `overlap = partial` is set in the `contact` block of the MpCCI input file), MpCCI performs a brute force linear search for all points that did not occur in step 1 and selects the best matches according to the matching criterion. There if one exception to this: If an intersection matching criterion is chosen no brute force search is performed.
4. **reject:** MpCCI rejects all pairs for which the matching criterion is not fulfilled.

The pre-contact search algorithms and matching criteria are described in tables of the form:

<i>Name</i>	<i>Description</i>	<i>Parameters</i>
name	description	parameter_name.1 = $i[default_i]$ parameter_name.2 = $r[default_r]$

where i and r denote integer and real values respectively. Default values for parameters are indicated in brackets.

In former versions of MpCCI only one kind of neighborhood computation could be carried out in one run of the coupled computation. This restriction is no longer effective.

5.3.1 Pre-Contact Search Algorithms in MpCCI

Most search algorithms which are currently implemented in MpCCI are based on a bucket search algorithm. We must take into account

- matching and non-matching grids
- the two-dimensional and the three-dimensional space
- one to three-dimensional coupling regions
- point-element, element-elements and point-point relationships, depending on the chosen type of interpolation.

We explain the basic algorithm - the bucket search algorithm - using as an example a matching grid in the two-dimensional space with a two-dimensional coupling region for point-point relationship.

General principle: The aim of a bucket pre-contact search algorithm is as follows. Given a set of points $P = \{p_1, \dots, p_n\}$ and a set of bounding boxes $B = \{b_1, \dots, b_m\}$ for a set of geometric objects $Q = \{q_1, \dots, q_n\}$, compute all pairs (p_i, b_j) of points p_i , $i = 1, \dots, n$ and bounding boxes b_j , $j = 1, \dots, m$ such that $p_i \in b_j$.

A straightforward algorithm to do this is to examine all pairs of points p_i and bounding boxes b_j . This leads to an amount of work of $\mathcal{O}(nm)$ which becomes unacceptably large for large n and m . To improve on this we use a bucket algorithm [??]. Practical experience with these algorithms [?] has shown that, although possibly theoretically inferior, they can compete with more theoretically optimal algorithms in many application areas. The reason is that optimality deals with theoretical limit cases, which are rarely reached in practice. Many examples demonstrating this fact are given in [?].

The bucket algorithm now proceeds as follows. A regular grid of buckets, uniform in each coordinate direction, is superimposed on the volume region as in Fig. 29.a. The size of the cells is h_i in the i -th coordinate direction. Each cell of this grid is called a bucket, and is referred to by its bucket multi-index $\bar{k} = (k_1, k_2)$ in 2-D or $\bar{k} = (k_1, k_2, k_3)$ in 3-D.

In the first step, we compute, for each bounding box b_j , the bucket indices $\bar{k}$ of the buckets occupied by b_j , and we store all these pairs $(\bar{k}, j)$ in an efficient data structure $\mathcal{M}$ for retrieving all bounding boxes b_j for a given bucket multi-index $\bar{k}$. In C++, such efficient data structures are part of the Standard Template Library [?] and are not further discussed here. Typically, a tree-like data structure is used for this.

In the second step, we compute for each point p_i the bucket multi-index $\bar{k}$ of the bucket that p_i occupies, which is trivial. Using the constructed mapping $\mathcal{M}$, we compute all b_j that occupy this bounding box and add all such pairs (p_i, b_j) to the result of the pre-contact search. In this way, a super set of all pairs (p_i, b_j) is constructed for which $p_i \in b_j$ is fulfilled. Optionally, one can check if $p_i \in b_j$ before adding a pair (p_i, b_j) , but this is not done in the MpCCI implementation.

Application to matching grids contact search: With the matching grid contact search, $P = \{p_1, \dots, p_n\}$ is a set of points of a coupling partition of one code, and B is a set of bounding boxes of points $Q = \{q_1, \dots, q_m\}$ of a coupling partition of another code. The aim of the contact-detection algorithm

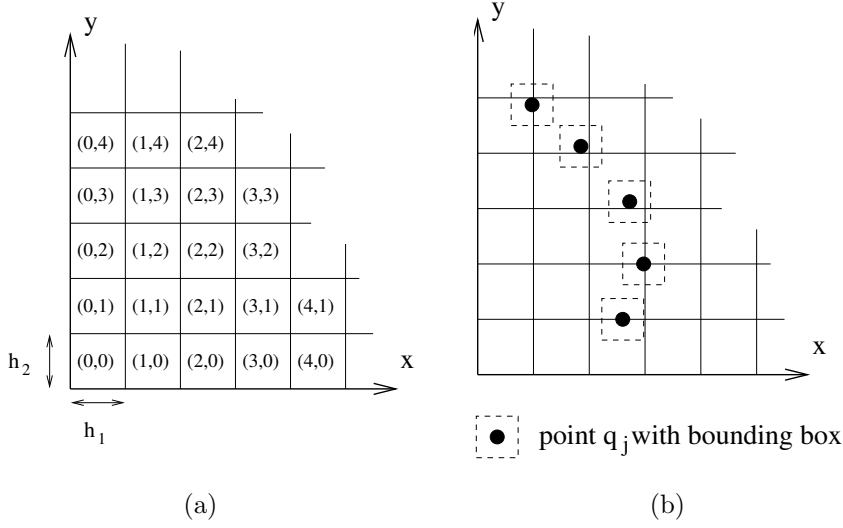


Figure 29: Bucket search in 2-D. Figure a) shows a bucket grid with buckets referenced by the bucket multi-index; in b) construction of the mapping from bucket multi-index k to $b_j \in B$ is presented.

is now to find all pairs (p_i, q_j) such that $\|p_i - q_j\|_2 < \epsilon$ where ϵ is a given tolerance and $\|x\|_2$ is defined by $\|x\|_2 = \sqrt{\sum_i x_i^2}$.

The bounding box b_j for a point q_j is constructed large enough to contain all points s such that $\|q - s\| \leq \epsilon$, where ϵ is the matching grid tolerance. In this way we are certain that the pre-contact search algorithm will find all pairs of points that match within the given tolerance. That is, the brute force search can be omitted in this case. We use a bucket grid with $h_i = r\epsilon$ in all directions, where r is a scaling factor.

Table 7 compares the bucket algorithm with linear search for a surface grid in 3-D. We used $\epsilon = 10^{-6}$ and took the bucket size $h_i = 4\epsilon$ in all coordinate directions. Note that the bucket algorithm finds the exact pairs of points. This is because if $\|p_i - q_j\| < \epsilon$ then $p_i \in B(q_j)$. Furthermore, for the particular meshes chosen, the distance between any two p_k and p_l with $k \neq l$ was larger than $4\sqrt{3}\epsilon$, the maximal distance between points in $B(q_j)$, so that each bounding box $B(q_j)$ can contain only one point p_i . Note that the linear search algorithm requires quadratic computing time. The observed complexity of the bucket algorithm is approximately $\mathcal{O}(n)$, which is better than its theoretical worst-case $\mathcal{O}(n \log n)$.

5.3.2 Matching criteria in MpCCI

Different types of interpolation need different types of information from the neighborhood search. These informations are the result of pre-contact search and the application of a matching criterion.

For example the linear interpolation needs the local coordinates of a node of the remote grid with respect to an element of the local grid, while an interpolation based on intersection methods needs the intersection area of a local and remote elements. For all matching criteria a rejection condition is implemented. These are described in the following sections.

n	<i>bucket search</i> <i>time [s]</i>	<i>number of</i> <i>comparisons</i>	<i>linear search</i> <i>time [s]</i>	<i>number of</i> <i>comparisons</i>
100	0.005	100	0.005	10,000
625	0.04	625	0.18	390,625
2500	0.11	2,500	3.29	6,250,000
5625	0.24	5,625	1.70	31,640,625
10000	0.47	10,000	54.76	100,000,000

Table 7: Real time of search algorithms

Time measurements show seconds on SUN Sparc Ultra and number of comparisons made with bucket search and linear search. Although the time measurement was not done recently the comparison of values should explain the advantage of the bucket search algorithm.

Currently the following types of matching criteria are available:

1. **minimal_distance** The result of this criterion is the relation between a node and an element. The local coordinates of the node of one grid w.r.t. a suitable element of another grid are computed.
2. **intersection** The result is the relation of one element of one grid with several elements of another grid. The areas of the intersections of the elements are computed.
3. **nearest_neighbor** The result is the relation of a node of one grid with a node of another grid. On the remote grid the node nearest to the node on the local grid is chosen.

5.4 Matching Grids

Pre-Contact Search Algorithms

Table 8 lists the available pre-contact search algorithms for matching grids. The default algorithm is bucket.

<i>Name</i>	<i>Description</i>
none	No pre-contact search algorithm. This amounts to a brute force linear search for all points.
bucket	A bucket search algorithm.

Table 8: Pre-contact algorithms for matching grids.

Pre-Contact Algorithm none

This pre-contact algorithm simply does not generate any pairs of points, so that the search algorithm itself amounts to a brute force linear search for all points. The algorithm does not support any parameters.

Pre-Contact Algorithm bucket

The bucket search algorithm has been explained above.

Matching Criteria

There is only one matching criterion for matching grids and this is the distance between two points. Two points p_1 and p_2 match if their distance is less than a specified tolerance.

Table 9 shows the available matching criteria for matching grids.

<i>Name</i>	<i>Description</i>	<i>Parameters</i>
minimal_distance	Distance between two points.	tol = $r[10^{-6}]$

Table 9: Matching criteria for matching grids.

5.5 Non-Matching Grids

The neighborhood search for two coupled codes with non-matching grids is based on a decomposition of the elements in

- triangles for two-dimensional coupling regions with the exception of spherical quadrilaterals, which are not decomposed at all.
- tetrahedrons for three-dimensional coupling regions,
- line segments for one-dimensional coupling regions.

MpCCI performs the pre-contact search algorithms and matching criteria for non-matching grids only to meshes of the decomposition elements (triangles, tetrahedrons and line segments), for example quadrilaterals are first split up into two triangles, before the contact search is carried out.

Pre-Contact Search Algorithms

Table 10 lists the available pre-contact search algorithms for non-matching grids. The default algorithm is bucket.

Pre-Contact Algorithm bucket and bucket_sphquad

The bucket algorithm is as described in Sec. 5.3.1. The two parameters `bbox_expansion` and `bucket_expansion` apply to the construction of bounding boxes for elements, and to the construction of the bucket grid respectively. The result of the pre-contact algorithm is a point-point relation (PP) or a point-element relation (PE) or an element-elements relation (EE) depending on the `type` chosen in the `contact` block of the input file. If the `bucket` algorithm is chosen for a pair of meshes consisting of spherical triangles it is recommended to choose both parameters greater than the default values because the curved surface of a spherical triangle is not taken into account when the bounding boxes are created.

<i>Name</i>	<i>Description</i>	<i>Parameters</i>	<i>Result</i>
none	No pre-contact search algorithm. This amounts to brute force linear search for all points.	–	
all	Pre-contact algorithm that generates all pairs of points and elements. Equivalent to brute force linear search, but requires a lot of memory. Only used to experiment.	–	
bucket	A bucket search algorithm.	$\text{bbox_expansion} = r[1.1]$ $\text{bucket_expansion} = r[1.0]$	PP, PE or EE
bucket_sphquad	A bucket search algorithm for a pair of meshes on the sphere consisting of spherical quadrilaterals.	$\text{bbox_expansion} = r[1.1]$ $\text{bucket_expansion} = r[1.0]$	EE

Table 10: Pre-contact algorithms for non-matching grids.

Parameter `bbox_expansion` For each element in the grid which has been restricted to an element of the decomposition, we first construct a bounding box just large enough to contain all nodes of the element. These bounding boxes are enlarged, so that they are equal in size in all coordinate directions by taking the largest dimension of the bounding box from the previous step. Then the bounding boxes are magnified by multiplying it with the eligible factor `bbox_expansion` with respect to their centers. Figure 30 summarizes the procedure.

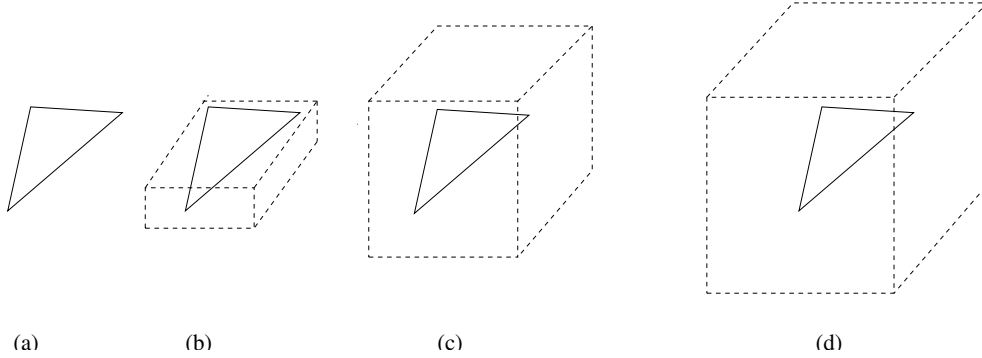


Figure 30: Construction of bounding boxes for: a) without bounding box, b) minimal bounding box, c), equal size in all directions, d) enlargement with a factor `bbox_expansion`.

Parameter `bucket_expansion` The algorithm proceeds in computing the average over the sizes of all constructed bounding boxes. Then a regular grid of buckets with a size that is a factor `bucket_expansion`

larger than the average element bucket size is superimposed on the volume region. The origin of the bucket grid is $(0, 0, 0)$. The bounding boxes, which have an overlap with an bucket, are assigned to this bucket.

Matching Criteria

Table 11 lists the available matching criteria for non-matching grids. The default matching criterion is `minimal_distance`.

<i>Name</i>	<i>Description</i>	<i>Parameters</i>	<i>Pre-contact search</i>
<code>minimal_distance</code>	Computes local coordinates.	$\theta_1 := r[0.0]$ $\theta_2 := r[1.0]$ $\theta_3 := r[0.0]$ $\tau := r[0.001]$	PE
<code>intersection</code>	Computes intersection areas.	$equals_zero := r[1.0E - 6]$ $perform_tests := b[true]$ $scale_result := b[true]$ $\tau := r[0.0]$	EE
<code>intersection_sphtri</code>	Computes intersection areas for meshes of spherical triangles.	$equals_zero := r[1.0E - 6]$ $perform_tests := b[true]$ $scale_result := b[true]$ $\tau := r[0.0]$	EE
<code>intersection_sphquad</code>	Computes intersection areas for meshes of spherical quadrilaterals.	$equals_zero := r[1.0E - 6]$ $perform_tests := b[true]$ $scale_result := b[true]$ $\tau := r[0.0]$	EE
<code>nearest_neighbor</code>	Determines the nearest node of the remote mesh to a node of the local mesh.	no parameters	

Table 11: Matching criteria for non-matching grids.

Matching criterion `minimal_distance`

Surface coupling in the three-dimensional space Given a triangle $\triangle ABC$, and a point p , the minimal distance matching criterion computes the minimal distance between p and all points inside the triangle. This is the basis for computing the best match.

To detect the element with the shortest distance to a specific point lying in one of these buckets the algorithm has only to consider the triangles of the bounding boxes assigned to this bucket. MpCCI computes the minimal distance between the particular point p and all points of each extracted triangle $\triangle ABC$.

The rejection criterion for `minimal_distance` is configurable using the parameters θ_1 , θ_2 , θ_3 , and τ . All

points for which

$$\theta_1 \cdot (|u| + |v| + |w| - 1) + \theta_2 \cdot \frac{d_t^2}{|\Delta ABC|} + \theta_3 \cdot \frac{d_n^2}{|\Delta ABC|} > \tau \quad (3)$$

are rejected. In Eq. 3, u , v , and w denote the barycentric coordinates of the projection of p' of p onto the plane defined by ΔABC , $d_n = \|p - p'\|$, and d_t is the distance between p' and the point p'' inside ΔABC that has minimal distance to p , (see Figure 31).

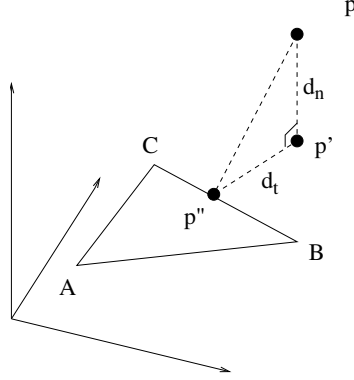


Figure 31: Minimal distance matching and rejection criteria.

In this neighborhood search the position of the nodes or element centers regarding the element, to which the node belongs, is fixed with barycentric coordinates as described in Sec. 5.1. If the `minimal_distance` criterion is chosen for a pair of meshes consisting of spherical triangles it is recommended to choose the `rejection` parameter greater than the default value because the curved surface of a spherical triangle is not taken into account, when d_n and d_t (see Eq. 3) are computed.

Volume coupling in the three-dimensional space In the current implementation only the parameters θ_1 and τ are used. The rejection criterion is

$$\theta_1 \cdot (|u| + |v| + |w| + |z| - 1) > \tau \quad (4)$$

Among the elements which are not rejected an element is chosen that have the smallest left hand side.

Line coupling in the two-dimensional space The rejection criterion for `minimal_distance` is configurable using the parameters θ_1 , θ_2 , θ_3 , and τ . The rejection criterion is

$$\theta_1 \cdot (|u| + |v| - 1) + \theta_2 \cdot d_t^2 + \theta_3 \cdot d_n^2 > \tau \quad (5)$$

Among the elements which are not rejected an element is chosen that have the smallest left hand side.

Volume coupling in the two-dimensional space In the current implementation only the parameters θ_1 and τ are used. The rejection criterion is

$$\theta_1 \cdot (|u| + |v| + |w| - 1) > \tau \quad (6)$$

Among the elements which are not rejected an element is chosen that have the smallest left hand side.

Matching criterion intersection

All matching criteria based on intersection have the same rejection condition. They differ in the fact whether a projection must be carried out or not. An example which includes projection is the "surface" type in the three-dimensional space:

$$\Delta ABC \cap \Pi(\Delta DEF) > \tau, \quad (7)$$

where ΔABC is a triangle of the local mesh and $\Pi(\Delta DEF)$ is the projection of the triangle ΔDEF of the remote mesh on the plane of ΔABC . All elements of the remote mesh which fulfill this inequation are chosen.

An example which excludes projection is the "volume" type in the two-dimensional space:

$$\Delta ABC \cap \Delta DEF > \tau, \quad (8)$$

If the parameter `perform_tests` has the value `true`, then the user is informed about incomplete and non disjunctive intersection areas.

If the parameter `scale_result` has the value `true`, then incomplete and non-disjunctive results will be scaled. The sum of the intersection areas is scaled by a factor, so that for the area of a local element holds:

$$|\Delta ABC| = factor \sum_{\Delta DEF} |\Delta DEF \cap \Delta ABC| \quad (9)$$

The parameter `equals_zero` is a bound for those real numbers which are treated as zero in the intermediate results of the algorithm.

Matching criterion nearest_neighbor

The `nearest_neighbor` type does not need any parameters. The `nearest_neighbor` algorithm is activated with the following command of the contact block of the input file:

```
type = nearest_neighbor;
```

5.6 Initial Distance Tracking

In this section we describe the initial distance tracking implemented in MpCCI for some surface elements only. At the moment, initial distance tracking applies only to the mesh coordinates. For initial distance tracking, we must distinguish between triangles and quadrilaterals.

Suppose that p is a point of a mesh of a remote code that must be interpolated from a mesh of triangles (Fig. 31). Now, instead of simply interpolating p from the three nodes of the triangle we also add the initial distance in form of a vector $\alpha \cdot \mathbf{n}$, where n is the unit outward normal of the triangle.

That is, for the initial meshes, we compute the projection of p onto the plane defined by the triangle and obtain u , v , and α such that

$$p = uA + vB + (1 - u - v)C + \alpha \mathbf{n}. \quad (10)$$

This equation is also used for interpolating p later on.

For quadrilaterals, we proceed in a similar way by first partitioning the quadrilateral into four triangles. Then we project the point p onto the planes defined by each of the four triangles, and choose that triangle for which $|u| + |v| + |1 - u - v|$ is smallest. Initial distance tracking now proceeds in the same way as for triangles.

6 Interpolation

6.1 Introduction

MpCCI will offer various built-in interpolation schemes so that the interpolation aspects can be totally hidden from the codes if desired. The interpolation scheme depends on the settings in the MpCCI input file and the element types defined in the code. The nearest neighbor interpolation is independent from element definitions and can therefore be used for both surface and volume coupling (see section 6.6).

The coupling may be further complicated by the movement of the contact surface itself. For example, with fluid-structure interactions, the fluid exerts a force on the structure, which may deform under that force. This means that the grid points of the contact surface can be coupling quantities also. Also other forms of changes of the contact surface can occur like

- mesh sliding, where the meshes move with respect to each other without deforming,
- remeshing, where a completely new mesh is generated for the same geometry on one or both sides, or when an old mesh is refined.
- dynamic load balancing by a code, where the distribution of the coupling surface over the processes of a code changes.

To investigate what is needed to perform interpolation methods, consider interpolating quantities from the fluid to the solid code (Fig. 6 on page 16). The quantities on the source grid (quadrilateral) are assumed at the nodes and the quantities on the target grid (triangles) are requested at the nodes. The quadrilateral cell index and local coordinates of the target points with respect to the source surface are needed to perform the interpolation.

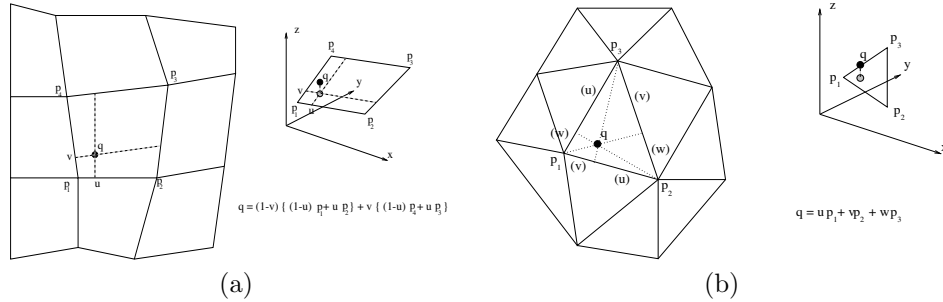


Figure 32: Local cell coordinates of a target point q on a source surface. Ruled surfaces are used for quadrilaterals (a), and barycentric coordinates are used for triangles (b).

We find these cell indices and local coordinates by projecting the target points onto the source surface in the neighborhood computations (see Sec. 5.1). Figure 32 shows an example of this, where the location of a target point q is indicated with respect to a quadrilateral element (a) or a triangular element (b).

Using the local coordinates and cell number of each of the target points, bilinear interpolation from a source grid consisting of quadrilaterals would look like:

```

for  $i = 1, n_{\text{req}}$ 

     $i_c = \text{icell}(i)$  // icell(i) contains the cell number of point i
     $u = \text{uloc}(i)$  // uloc(i) and vloc(i) contain the corresponding
     $v = \text{vloc}(i)$  // local coordinates.

    determine the values  $v_{00}, v_{10}, v_{01}, v_{11}$  at the four corner points
     $p_{00}, p_{10}, p_{01}, p_{11}$  of the quadrilateral number  $i$ 

    // bilinear interpolation
     $\text{val}(i) = (1 - v) \cdot ((1 - u) \cdot v_{00} + u \cdot v_{10}) +$ 
                $v \cdot ((1 - u) \cdot v_{01} + u \cdot v_{11})$ 

end for

```

In the above pseudo-code, the target surface may consist of any type of elements. The coding for the interpolation on the source grid as shown above is completely independent of the topology of the target grid.

6.2 Coupling Regions

MpCCI supports different kind of coupling regions:

- **Three-dimensional space** Surface, volume, sphere, where the mid-point must be the origin of a cartesian coordinate system, and sets of points.
- **Two-dimensional space** Line, volume and sets of points.

In the current version all grids must be defined in cartesian coordinates even meshes on the sphere.

6.3 Available Interpolation

Using the results of the neighborhood computation (PE, EE or PP), interpolation can be performed. For this purpose, we subdivide the coupling quantities into three categories:

- *Non-conservative interpolated quantities* are functions of the spatial coordinates and time. Examples are temperature, pressure, and velocity. The mesh point coordinates constitute a special quantity which may be handled by other methods as well (see Sec. 6.7).

Quantities relying on non-conservative interpolation must be of `type = field` in the `code` block of the MpCCI input file.

- *Conservative interpolated quantities* are amounts of conserved quantities crossing the contact surface through a surface element. Examples of these quantities are the heat flux through an element area and the force exerted on an element area. Non-conservative interpolation (e.g. bilinear) is meaningless for such quantities. A common property of these quantities is that their sum must be preserved during interpolation. One of the methods to achieve this is described in Sec. 6.4.2. In general conservativity in MpCCI is defined as the perserving of the sum of the values. For volume coupling regions the above explanations for surfaces are not valid, but MpCCI also supports sum-preserving mappings in this case. Quantities relying on conservative interpolation must be of `type = flux` in the `code` block of the MpCCI input file.
- *Mesh points* are the nodes of the elements defining the surface mesh. The interpolation of mesh points is important to handle mesh deformation. Mesh interpolation can be handled as non-conservative interpolation, but more specialized methods (e.g. initial distance tracking) are possible as well. Mesh interpolation is discussed in Sec. 6.7. The quantity name `mesh` in the `code` block of the input file is reserved for transferring mesh data.

The interpolation schemes supported by MpCCI on non-matching grids are shown in table 12.

<i>Name</i>	<i>Non-conservative</i>	<i>Conservative</i>	<i>Neighborhood Search</i>
Standard	yes	yes	PE
Intersection	yes	yes	EE
Nearest neighbor	yes	no	separate solution

Table 12: Interpolation types.

6.4 Standard Interpolation

All interpolation types based on PE neighborhood computation are called standard interpolation.

6.4.1 Standard Non-Conservative Interpolation

In the case of surface interpolation the receiver mesh has been projected on the sender mesh to gain the result of the neighborhood search.

From nodes on nodes or element midpoints The interpolation supported by MpCCI is based directly on the element definition. The source values s_i to be interpolated are located at the nodes and the target quantity t is required at points inside the elements by the local coordinates u and v (Fig. 33). A straightforward interpolation is done using the base functions as follows:

$$t = \sum_i N_i(u, v) s_i \quad (11)$$

Vectors (e.g. velocities) are interpolated component wise. This interpolation is constant, linear or quadratic depending on the source element.

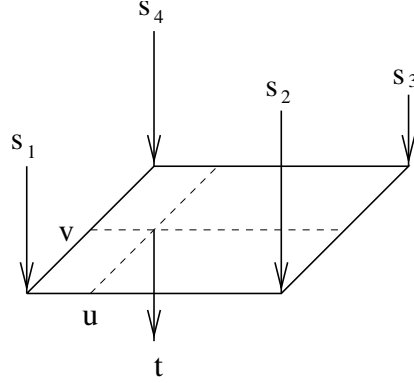


Figure 33: Non-conservative interpolation. Data are interpolated on a quadrilateral element from the source points s_i at the nodes to a target point t .

From elements on nodes or element midpoints There is also a simple mapping implemented in MpCCI to transfer data from an element E with midpoint M on the local mesh on points P (nodes or midpoints) of the remote mesh:

$$v_P = v_M$$

where v_M is the value on E and v_P is the value transferred to P (Fig. 34). Of course the pair (P, E) must be a result of the neighborhood search.

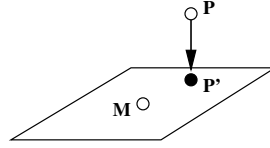


Figure 34: Non-conservative mapping. Data are mapped from a quadrilateral element on the target point P .

6.4.2 Standard Conservative Interpolation

In the case of surface interpolation the sender mesh has been projected on the receiver mesh to gain the result of the neighborhood search. Conservativity is ment as perserving the sum of the values.

From nodes or elements on nodes The standard conservative interpolation supported by MpCCI is also done based on the element definition. However, the situation is opposed to that for the fields, because here the source value is located at a single point, and the target locations are the nodes, see Fig. 35. By

evaluating the basis functions N_i that define the element in the point where the coupling quantity is given, we obtain four weights

$$w_i = N_i(u, v), \quad (12)$$

one for each corner, that satisfy

$$\sum_i w_i = 1 \quad (13)$$

because of

$$\sum_{i=1, \dots, m} N_{k_i}(u, v) \equiv 1 \quad \text{for all } (u, v) \in E, \quad (14)$$

Each node i then gets a portion $w_i \cdot s$ of the given coupling value s . Therefore, by its definition, this interpolation preserves the sum of the quantities⁶. Vectors (e.g. forces) are interpolated componentwise.

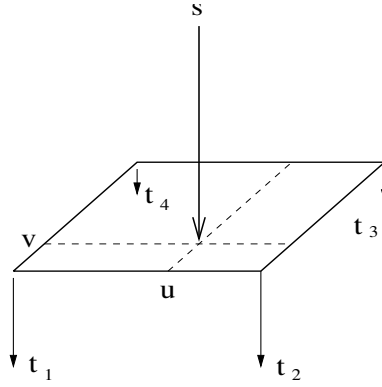


Figure 35: Conservative interpolation. Data are interpolated on a quadrilateral element from a single source point s to the target points t_i at the corners.

From nodes or elements on elements There is also a simple mapping implemented for the transfer on elements. If (P, E) is a result of the neighborhood computation, where P is a node or an element midpoint from the sender mesh and E an element of the receiver mesh, the value v_P on P is added to the value v_E of E :

$$v_E + = v_P$$

in C++-notation (Fig. 36).

⁶Note that this can also be called a *redistribution*.

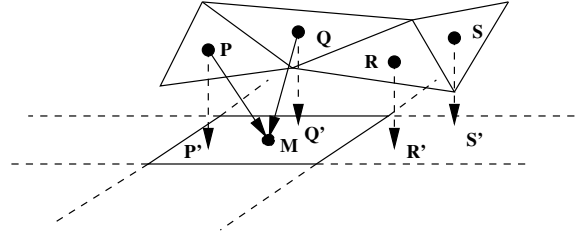


Figure 36: Conservative mapping. Data are mapped on a quadrilateral element from the target points P and Q .

6.5 Mappings based on intersection method

This algorithm is based on the area of the intersection of the elements of both meshes. In the case of surface coupling the elements of one mesh are projected on the other to compute the intersection area. Quantities which are to be transferred with the help of intersection methods must be defined as `loc = elem` in the code block of the MpCCI input file.

The algorithm implemented for volume coupling in the three-dimensional space is based on the idea described in [?].

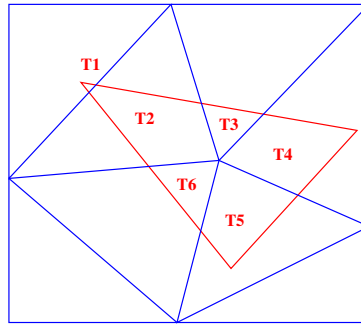


Figure 37: Intersection of triangles

6.5.1 Non-Conservative Mappings based on intersection methods

For the non-conservative mapping a weighted average of the elements of the source mesh intersecting an element of the target mesh is computed taking into account the areas of the overlap regions. In the case of surface coupling the source elements are projected onto the target elements during the neighborhood search.

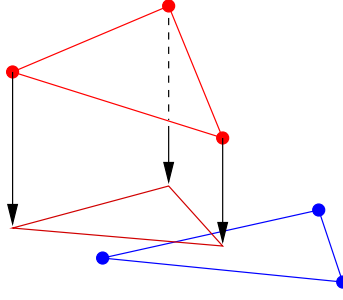


Figure 38: Projection of triangles

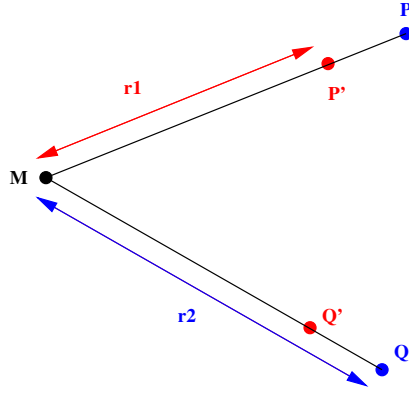


Figure 39: Projection of points P and Q from a sphere (midpoint M) with radius r_2 onto the sphere with radius r_1

We give the formula for triangular meshes on sender and receiver side:

$$w_E = \frac{1}{|E|} \sum_{D \in G} |\Pi_E(D) \cap E| v_D$$

where E is an element on the target mesh with value w_E , G is the source mesh and v_D is the value of element D on the source mesh. Π denotes the projection explained above and $|\cdot|$ is the size of an area.

6.5.2 Conservative Mappings based on intersection methods

Again the sum of values on the source grid is preserved by this method. In the case of surface coupling the target elements are projected onto the source elements in the neighborhood search.

We give the formula for triangular meshes on sender and receiver side:

$$w_E = \sum_{D \in G} \frac{|\Pi_E(D) \cap E|}{|D|} v_D$$

where E is an element on the target mesh with value w_E , G is the source mesh and v_D is the value of element D on the source mesh. Π denotes the projection explained above and $|\cdot|$ is the size of an area.

6.6 Nearest neighbor interpolation

The nearest neighbor interpolation is a simple way to transfer data from the local to a remote mesh. MpCCI simply computes pairs of grid points, one on the local and one on the remote mesh. To a local grid point the algorithm determines the grid point of the remote mesh which has the smallest distance to the local point.

This type of data transfer is independent from element definitions and can therefore be used for surface and volume coupling. To let MpCCI carry out nearest neighbor interpolation the user has to set `type = nearest_neighbor` in the contact block of the MpCCI input file. The element checks are automatically skipped then, so that no call of `CCI_Def_elems` is necessary.

6.7 About Remeshing

An important topic to be dealt with is remeshing. We say that remeshing takes place whenever there is a change in the definition of the coupling surface or the distribution of the coupling surface over the processes of an application.

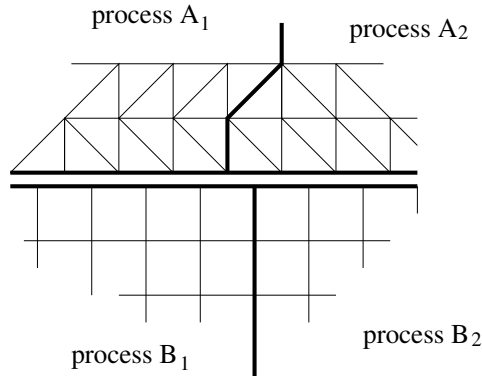


Figure 40: Original meshes, two-dimensional example. There are two codes A and B each using two processes A_1 , A_2 and B_1 , B_2 respectively.

To explain the different types of remeshing that can occur, consider first an original mesh as in Fig. 40.

We will use this mesh in the examples. For ease of exposition, the mesh is two-dimensional and the coupling surface is one dimensional.

There are four types of remeshing, namely mesh deformation, mesh sliding, mesh refinement and the rearrangement of grid points due to load balancing requirements. These types are explained below.

With mesh deformation, one of the sides of the coupling surface deforms, and the coupling surface owned by the processes on the other side must deform with it, compare Fig. 41 with 40. An example of such a situation is the case of a fluid interacting with a solid structure, where the solid structure deforms under the force exerted on it by the fluid.

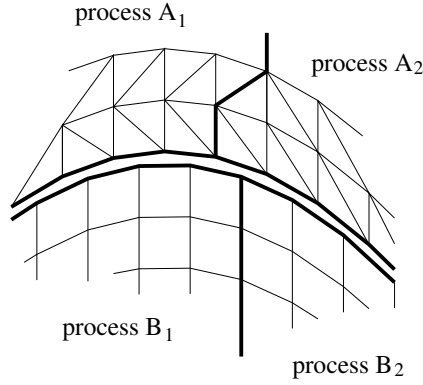


Figure 41: Mesh deformation.

With mesh sliding, the way in which the contact surfaces are connected changes radically. As a consequence, the neighborhood relations must be continuously updated. Compare Fig. 42 with 40 for an example.

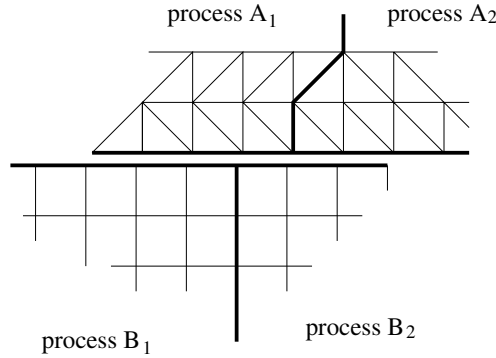


Figure 42: Mesh sliding.

A code is allowed to do remeshing and load balancing by itself. With remeshing it is meant that the code generates a new mesh for itself because the old mesh was too coarse or too skewed at some points. A special case of this remeshing is mesh refinement. Compare Fig. 43.a and 43.b with 40 for examples of mesh refinement and load balancing. One way to handle remeshing and load balancing is to completely recompute the neighborhood relations. This would then involve all processes of the coupled computation and would thus be a collective operation.

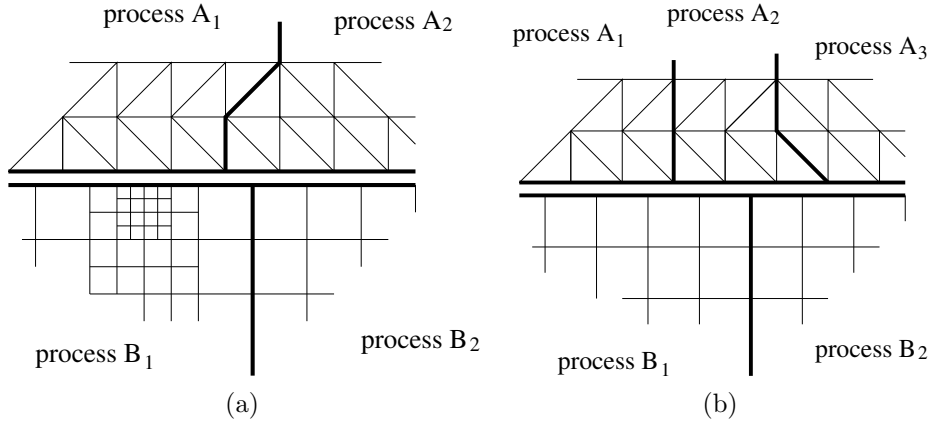


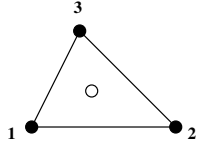
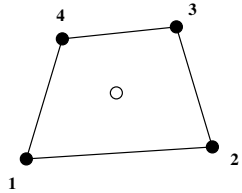
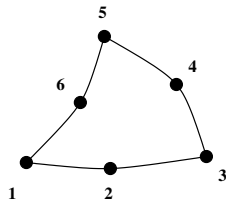
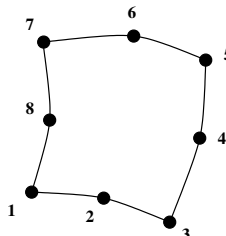
Figure 43: Mesh refinement (a) and load balancing by a code (b).

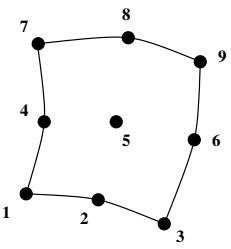
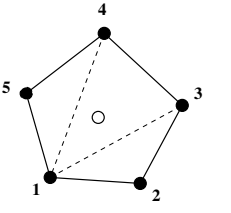
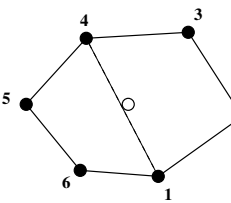
Several methods are available for handling moving meshes. See [??] for more details.

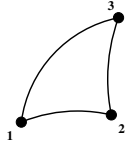
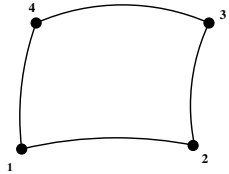
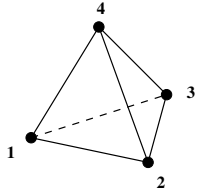
In the current version MpCCI supports the remeshing in a straight-forward and simple way only: changes of the mesh of one code can be defined after a coupling step and the remote code has to be informed about the changes (see Sec. 3.5). This will lead to a new setup of the coupling, i.e. a complete neighborhood search from scratch.

6.8 Elements in MpCCI

In table 13 we summarize the available element types and the interpolation types which can be used. If the spherical element types are used make sure that both meshes are built with the same spherical element type. The spherical elements are not compatible because the spherical quadrilateral cannot be subdivided into two spherical triangles.

<i>Name</i>	<i>Coupling Regions</i>	<i>Interpolation types</i>	<i>Figure</i>
CCI_ELEM_TRIANGLE	Surface in 3D (incl. sphere), volume in 2D	Standard interpolation (linear), intersection method	
CCI_ELEM_QUAD	Surface in 3D (incl. sphere), volume in 2D	Standard interpolation (linear), intersection method	
CCI_ELEM_TRIANGLE6	Surface in 3D (incl. sphere)	Standard interpolation (quadratic)	
CCI_ELEM_QUAD8	Surface in 3D (incl. sphere)	Standard interpolation (quadratic)	

<i>Name</i>	<i>Coupling Regions</i>	<i>Interpolation types</i>	<i>Figure</i>
CCI_ELEM_QUAD9	Surface in 3D (incl. sphere)	Standard interpolation (quadratic)	
CCI_ELEM_PENTAGON	Surface in 3D (incl. sphere)	Standard interpolation (linear on triangles)	
CCI_ELEM_HEXAGON	Surface in 3D (incl. sphere)	Standard interpolation (linear on quadrilaterals)	

<i>Name</i>	<i>Coupling Regions</i>	<i>Interpolation types</i>	<i>Figure</i>
CCI_ELEM_SPH_TRIANGLE	Sphere	Standard interpolation (linear), intersection method	
CCI_ELEM_SPH_QUAD	Sphere	Intersection method	
CCI_ELEM_LINE	Surface in 2D	Standard interpolation (linear), intersection method	
CCI_ELEM_TETRAHEDRON	Volume in 3D	Standard interpolation (linear), intersection method	

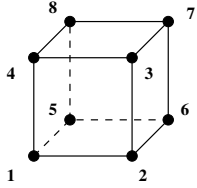
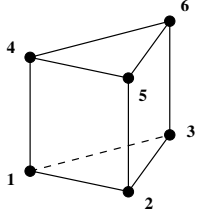
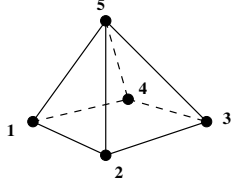
<i>Name</i>	<i>Coupling Regions</i>	<i>Interpolation types</i>	<i>Figure</i>
CCI_ELEM_HEXAHEDRON	Volume in 3D	Standard interpolation (linear), intersection method	
CCI_ELEM_PRISM	Volume in 3D	Standard interpolation (linear), intersection method	
CCI_ELEM_PYRAMID	Volume in 3D	Standard interpolation (constant), intersection method	

Table 13: Element types.

7 An Example

This section describes an example of two codes coupled with each other. For ease of exposition, we consider a two-dimensional example with a one-dimensional coupling region. The term *coupling surface* or *coupling region* used before, now refers to this coupling line.

The example is the coupling of a fluid (F) with a structures code (S). The coupling method considered is one where the structures code requires forces and heat fluxes at the nodes, that are computed by the fluid code at the cell-centers. The fluid code on the other hand requires the new mesh coordinates and velocities from the structures code at its nodes. The structures code computes these quantities at its nodes. The grids do not match at the grid interfaces. The coupling is depicted in Fig. 44. The example code fragments are in FORTRAN 77.

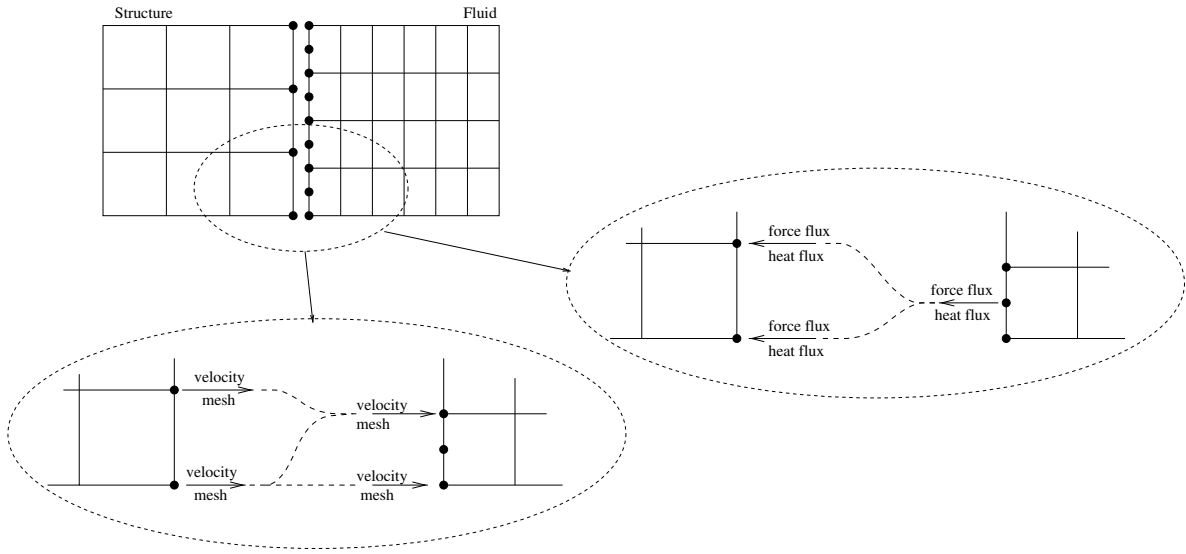


Figure 44: Two-dimensional coupling example of a structures with a fluid code.

7.1 Startup and Initialization

In the coupled computation, both code S and F use 4 processes. Code S only wants to call MpCCI subroutines from the two processes that lie at the grid interface. The fluid code wants to call MpCCI subroutines from all its processes, which implies of course that some processes of code F do not have a coupling partition. The jobs block of the MpCCI input file may look the following way

```
jobs
  JobName1 = S (
```

```

    pwd    = ".",
    exec   = "binary1",
    nprocs = 2
);
JobName2 = F (
    pwd    = ".",
    exec   = "binary2",
    nprocs = 4
);
end

```

where `binary1` and `binary2` are the names of the executables.

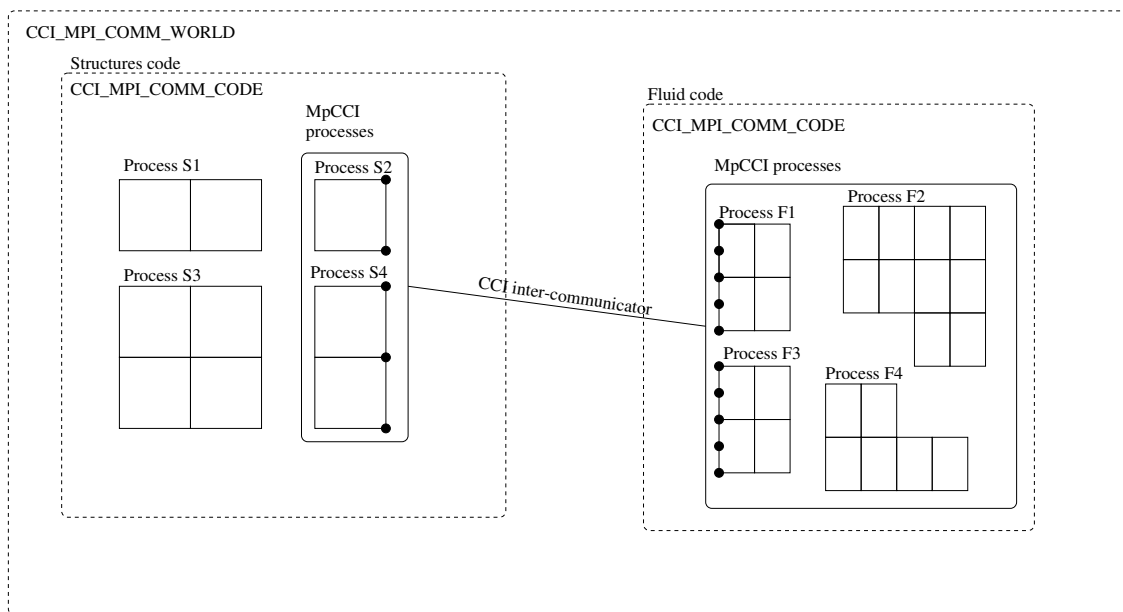


Figure 45: Startup of the coupled application.

The initialization of the coupled computation starts with the enrolling of both codes the coupled computation:

```

c    --- enroll in the coupled computation

    call CCI_INIT(ierr)

```

After the call to `CCI_Init`, the communicator `CCI_MPI_COMM_CODE` has been computed and contains all processes of the calling code. This communicator must be used for further code-internal communication. With MPI-2, `CCI_MPI_COMM_CODE` is simply equal to `MPI_COMM_WORLD` when each code was spawned separately by the control process if it existed. With MPI-1 on the other hand, where all processes of all codes are spawned simultaneously, `MPI_COMM_WORLD` contains all processes of all codes and can therefore not be used for code internal communication.

7.2 Coupling Definition

In the coupling definition phase, each of the codes specifies its coupling region to MpCCI. In this example only one mesh with one partition on each processor is used. For process `S4` this looks like:

```
c    --- specify partitions

    meshId = 1
    partId = 1

    call CCI_DEF_PARTITION(meshId, partId, ierror)

c    --- specify nodes

    globalDim = 2
    nNodes    = 3
    nNodeIds  = 0
    nodeIds   = 0
    dataType  = CCI_REAL

    coords(1) = 0.0e0 ! x coordinate of the first node
    coords(2) = 0.0e0 ! y coordinate of the first node
    coords(3) = 0.0e0 ! x coordinate of the second node
    coords(4) = 0.5e0 ! y coordinate of the second node
    coords(5) = 0.0e0 ! x coordinate of the third node
    coords(6) = 1.0e0 ! y coordinate of the third node

    call CCI_DEF_NODES(meshId, partId, globalDim, nNodes,
+                      nNodeIds, nodeIds, dataType, coords, ierror)
```

For the interpolation chosen in this example information concerning the topology of elements is necessary. This will be defined as follows:

```
c    --- specify elements by defining their nodes

    nElems      = 2
```

```

nElemIds      = 0
elemIds       = 0
nElemTypes    = 1
elemTypes     = CCI_ELEM_LINE
nNodesPerElem = 2

elemNodes(1)  = 1
elemNodes(2)  = 2
elemNodes(3)  = 2
elemNodes(4)  = 3

call CCI_DEF_ELEMS(meshId, partId, nElems, nElemIds,
+                  elemIds, nElemTypes, elemTypes, nNodesPerElem,
+                  elemNodes, ierror)

```

The above pseudo-code must be executed only by the MpCCI processes, so for code S, only processes 2 and 4 should execute this code. In this example, `globalDim` equals 2. Since every element is made up by two nodes in both codes, `nNodesPerElem` equals 2. The values of the parameters `nNodes`, `nElems` and `elemTypes` for the other processes are given in Tab. 14. The processes F2 and F4 need not define a mesh.

<i>process</i>	<i>nNodes</i>	<i>nElems</i>	<i>elemTypes</i>
S2	2	1	CCI_ELEM_LINE
S4	3	2	CCI_ELEM_LINE
F1	3	2	CCI_ELEM_LINE
F3	3	2	CCI_ELEM_LINE

Table 14: Values of parameters in interface definition for the example.

Note that in Tab. 14, the nodes at the centers of the cells of code F have not been specified. This is because these nodes are derived nodes, whose coordinates can be computed from the other nodes. The definition of the derived nodes as a function of the other nodes is part of the element definition.

Besides the definition of the topology of the coupling region, each code has to specify the locations of the coupling quantities for the coupling with each of the other codes separately. It can be specified in the MpCCI input file if coupling values reside at the element centers. The code blocks and the quantities block of the MpCCI input file should look the following way:

```

code S
  mesh(      id = 6 );
  velocity( id = 7, loc = node, dim = scalar, type = field );
  force(    id = 8, loc = node, dim = 2,      type = flux );
  heat(     id = 9, loc = node, dim = scalar, type = flux );
end

```

```

code F
  mesh(      id = 10 );
  velocity( id = 11, loc = node, dim = scalar, type = field );
  force(    id = 12, loc = elem, dim = 2,      type = flux );
  heat(     id = 13, loc = elem, dim = scalar, type = flux );
end

quantities
  S.velocity = F.velocity;
  S.force    = F.force;
  S.heat     = F.heat;
end

```

MpCCI can only map fluxes to the nodes and from elements to elements. Fields can be interpolated from nodes and from elements to elements.

In the definition phase other subroutines may be called for defining synchronization points, communicators, search tags etc. For instance the definition of a communicator in the structure code could look as follows

```

localCommId    = 123
remoteCodeId   = 2
remoteCommId    = 321
nQuantityIds   = 2
quantityIds(1) = 6
quantityIds(2) = 7
nLocalMeshIds  = 1
localMeshIds   = 1

call CCI_DEF_COMM( localCommId, remoteCodeId, remoteCommId,
+                  nQuantityIds, quantityIds, nLocalMeshIds,
+                  localMeshIds, ierror)

```

The remote communicator 321 must be defined in the remote code which is the fluid code in this example. The communicator 321 should be able to communicate the quantities 10 and 11 which match the quantities 6 and 7 of the structure code as defined in the input file above.

The third step of the initialization phase is to define the MpCCI processes to MpCCI. Thus after the coupling regions have been completely specified, CCI_CLOSE_SETUP must be called to commit to the definition of the regions and perform the initial neighborhood computation. This is done by calling subroutine CCI_CLOSE_SETUP with an argument that indicates if the calling process is a MpCCI process or not.

```

c      --- specify MpCCI processes to MpCCI

      if ( I_am_a_MpCCI_process ) then

          included = 1

      else

          included = 0

      end if

      call CCI_CLOSE_SETUP(included, ierror)

```

In this example the processes S2, S4, F1 and F3 must call CCI_CLOSE_SETUP with the value `included = 1`. F2 and F4 must call CCI_CLOSE_SETUP with the value `included = 0`.

After this call, only the above defined MpCCI processes S2, S4, F1 and F3 may call MpCCI subroutines, with the exception of CCI_FINALIZE, which must be called by all processes of the coupling. After this the coupled computation can start.

7.3 Coupled Computation

The exchange of coupling values through a coupling partition is simple. Here we only give pseudo-code for sending and receiving of coupling values between two codes. More details are algorithm-dependent and therefore omitted.

Code S specifies coupling values at the nodes, and can therefore call CCI_PUT_NODES for node-wise specification:

```

c      --- specify values at the nodes for the first quantity

      meshId      = 1
      partId      = 1
      quantityId  = 6
      quantityDim = 2
      nNodes      = 3
      nNodeIds    = 0
      nodeIds     = 0
      dataType    = CCI_REAL
      values(1)   = ...
      values(2)   = ...
      ...
      values(6)   = ...

```



```

    call CCI_PUT_NODES(meshId, partId, quantityId, quantityDim,
+                      nNodes, nNodeIds, nodeIds, dataType,
+                      values, ierror)

c    --- and for the second quantity

    quantityId = 7
    quantityDim = 1
    values(1) = ...
    values(2) = ...
    values(3) = ...

    call CCI_PUT_NODES(meshId, partId, quantityId, quantityDim,
+                      nNodes, nNodeIds, nodeIds, dataType,
+                      values, ierror)

c    --- send values

    nQuantityIds = 2
    quantityIds(1) = 6
    quantityIds(2) = 7
    nLocalMeshIds = 2
    localMeshIds(1) = 1
    localMeshIds(2) = 1
    comm = 123

    call CCI_SEND(nQuantityIds, quantityIds,
+                nLocalMeshIds, localMeshIds, comm, ierror)

```

The line `comm = 123` could be exchanged by `comm = CCI_COMM_RCODE(2)` in this example.

The specification of coupling values by code F is different because there the values reside at the cell-centers:

```

c    --- specify values at the elements

    meshId = 1
    partId = 1
    quantityId = 12
    quantityDim = 2
    nElems = 2
    nElemIds = 0

```

```

elemIds          = 0
nDataPointsPerElem = 1
dataPointsPerElem = 1
dataType         = CCI_REAL
values(1)        = ...
values(2)        = ...
...
values(6)        = ...

call CCI_PUT_ELEMS(meshId, partId, quantityId, quantityDim,
+                  nElems, nElemIds, elemIds,
+                  nDataPointsPerElem, dataPointsPerElem,
+                  dataType, values, ierror)

c      --- fill array with the second quantity

quantityId       = 13
quantityDim      = 1
values(1)        = ...
values(2)        = ...
values(3)        = ...

call CCI_PUT_ELEMS(meshId, partId, quantityId, quantityDim,
+                  nElems, nElemIds, elemIds,
+                  nDataPointsPerElem, dataPointsPerElem,
+                  dataType, values, ierror)

c      --- send values

nQuantityIds     = 2
quantityIds(1)   = 12
quantityIds(2)   = 13
nLocalMeshIds    = 2
localMeshIds(1)  = 1
localMeshIds(2)  = 1
remoteCodeId     = 1
comm             = CCI_COMM_RCODE(remoteCodeId)

call CCI_SEND(nQuantityIds, quantityIds,
+            nLocalMeshIds, localMeshIds, comm, ierror)

```

For the quantities 12 and 13 there is no user defined communicator, so the predefined communicator

CCI_COMM_RCODE(1) must be used.

The pseudo-code for receiving coupling values is analogous.

7.4 Termination

Termination of the coupled computation must be done by calling subroutine CCI_FINALIZE. After calling this subroutine, the calling process is still alive, but may not use MpCCI calls anymore. The caller can continue its own termination sequence by closing output files and so on. Of course, any output file of a code may also be closed before calling CCI_FINALIZE. In pseudo-code:

```
c    --- termination sequence of a code

    ... some statements for cleaning up, closing files etc....

c    --- exit MpCCI

    call CCI_FINALIZE(ierrror)

c    --- continue termination sequence

    ... last statements for cleaning up, closing files etc....
```

In this design, the termination sequence of the codes can be used without modification. Only the CCI_FINALIZE call must be added. The MpCCI control process will not kill the codes, but just terminate the coupling.

A Mesh quality checks by MpCCI

One of MpCCI's tasks is to check the mesh consistency and quality at the coupling surface. This check is done once in the subroutine `CCI_Close_setup`, where the specified node coordinates, topology data and (potentially) global numbers of nodes and elements for all partitions are committed to MpCCI. At the moment, the following checks are performed:

- **doubly defined nodes:** It is checked whether all nodes on one partition are distinct from each other. If this is not the case, an error message is given.
- **degenerate elements:** For each element it is checked whether its node numbers are distinct from each other. If this is not the case, a warning is given.

In addition, separate checks for different element types of MpCCI like quadrilateral elements (A.1), triangles (A.2), derived quadrilaterals and triangles (A.3) and pentagons and hexagons (A.4) are performed.

A.1 Checks for quadrilaterals

Table 15 shows the parameters defining the tests and their default values. Alternative values can be specified using the `mesh_quality_quad` statement of the control block.

<i>Name</i>	<i>Description</i>
<code>taper = r[0.35]</code>	Minimal allowed taper.
<code>aspect_ratio = r[39.0]</code>	Maximal aspect ratio.
<code>skew = r[60.0°]</code>	Maximal skew angle.
<code>warp = r[5.0°]</code>	Maximal warp angle.

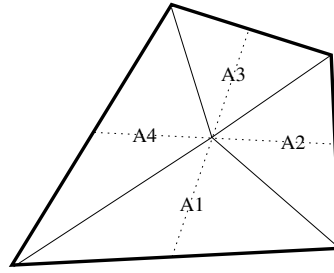
Table 15: Parameters for the quadrilateral mesh quality tests.

For each quadrilateral Q the following specific tests are performed. If one of them fails a warning is given. All tests are based on the fact, that the middle points of the four sides of Q lie in a plane. Thus, these four middle points define a plane quadrilateral Q_{plane} which is related to the original quadrilateral Q .

- **taper:** The quotient

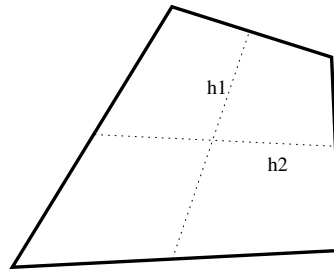
$$\tau = 4 \frac{\min(A_1, A_2, A_3, A_4)}{A_1 + A_2 + A_3 + A_4}$$

is a measure of the quadrilateral's taper.



The common point of the four triangles is the intersection of the diagonals of Q_{plane} . Per default, τ is supposed to be greater than 0.35.

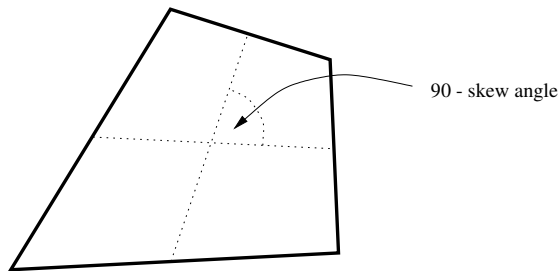
- **aspect_ratio:** The ratio of the lengths of the diagonals in Q_{plane} is called the 'aspect ratio':



$$AR = \frac{\max(h_1, h_2)}{\min(h_1, h_2)}$$

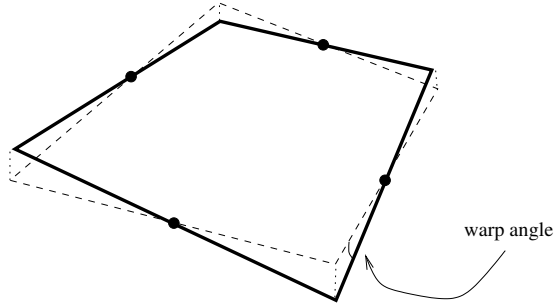
Per default, AR should be less than 39.0.

- **skew:** The following figure defines the skew angle which, by default, is supposed to be less than 60.0 degrees.



The skew angle is related to the angle between the two diagonals in Q_{plane} .

- **warp:** The maximal warp angle is a measure of the curvature of Q . For each side of Q , the corresponding warp angle is defined as the angle between the side and the plane of Q_{plane} which is, in a sense, the average plane of Q .



By default, the maximal warp angle of the four sides is supposed to be less than 5.0 degrees.

A.2 Checks for triangles

Table 16 shows the parameters defining the tests and their default values. Alternative values can be specified using the `mesh_quality_triangle` statement of the control block.

<i>Name</i>	<i>Description</i>
<code>aspect_ratio = r[39.0]</code>	Maximal aspect ratio.
<code>skew = r[45.0°]</code>	Maximal skew angle.

Table 16: Parameters for the triangle mesh quality tests.

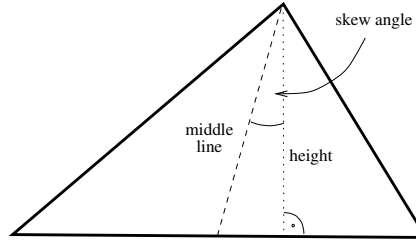
For each triangle the following specific tests are performed. If one of them fails a warning is given.

- **aspect_ratio:** As for quadrilaterals, we can define an aspect ratio for each side of the triangle: Let h be the height and l be the length of the side. Then we set:

$$\text{AR} = \frac{\max(h, l/2)}{\min(h, l/2)}$$

Per default, the maximal aspect ratio of the three sides should be less than 39.0.

- **skew:** At each node, a skew angle can be defined as the angle between the height and the middle line:



By default, the maximum of the three skew angles is supposed to be less than 45.0 degrees.

A.3 Checks for CCI_ELEM_QUAD8, CCI_ELEM_QUAD9 and CCI_ELEM_TRIANGLE6

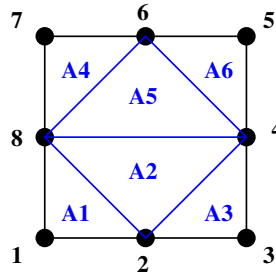
Table 17 shows the parameters defining the tests and their default values. Alternative values can be specified using the `mesh_quality_quad8` statement of the control block for CCI_ELEM_QUAD8, the `mesh_quality_quad9` statement for CCI_ELEM_QUAD9 and the `mesh_quality_triangle6` statement for CCI_ELEM_TRIANGLE6. The parameters are the same for these three element types.

<i>Name</i>	<i>Description</i>
taper = r[0.35]	Minimal allowed taper.
angle = r[150.0°]	Minimal angle between the nodes of an edge.
area = r[0.0]	Minimal allowed area of the triangles of the decomposition.

Table 17: Parameters for mesh quality tests of CCI_ELEM_QUAD8, CCI_ELEM_QUAD9 and CCI_ELEM_TRIANGLE6.

For each of the three element types the following specific tests are performed. If one of them fails a warning is given.

- **taper:** For this test the elements are decomposed into triangles:
 - CCI_ELEM_QUAD8 is decomposed into six triangles:

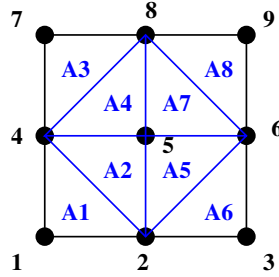


We call the quotient

$$\tau = 8 \frac{\min(A_1, \frac{A_2}{2}, A_3, A_4, \frac{A_5}{2}, A_6)}{A_1 + A_2 + A_3 + A_4 + A_5 + A_6}$$

taper.

- CCI_ELEM_QUAD9 is decomposed into eight triangles:

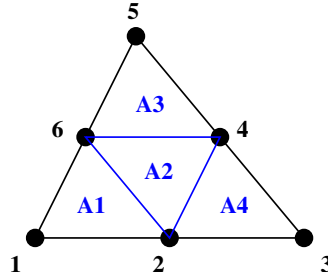


We call the quotient

$$\tau = 8 \frac{\min(A_1, A_2, A_3, A_4, A_5, A_6, A_7, A_8)}{A_1 + A_2 + A_3 + A_4 + A_5 + A_6 + A_7 + A_8}$$

taper.

- CCI_ELEM_TRIANGLE6 is decomposed into four triangles:



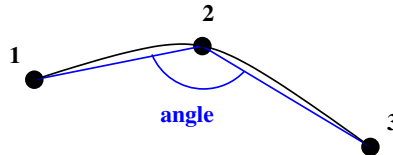
We call the quotient

$$\tau = 4 \frac{\min(A_1, A_2, A_3, A_4)}{A_1 + A_2 + A_3 + A_4}$$

taper.

Per default, τ is supposed to be greater than 0.35.

- **angle:** These elements are allowed to have curved edges. As a measure for the curvation the angle between the three nodes of an edge is determined. For example: The angle between the straight line through node 2 and 1 and the straight line through node 2 and 3.

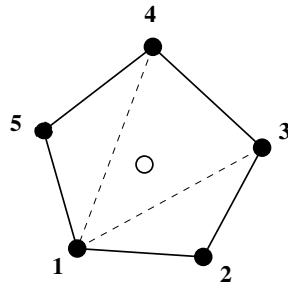


By default, the maximal warp angle of the four sides is supposed to be greater than 150.0 degrees. The angle is supposed to be in $[0, 180]$.

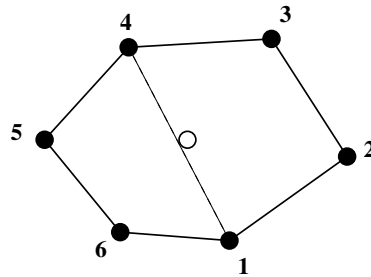
- **area:** The minimal area of the decomposition triangles can be given. The default value is 0.0 (no restriction).

A.4 Checks for pentagons and hexagons

The element type `CCI_ELEM_PENTAGON` is divided into three triangles for the interpolation. These triangles are checked like it is described above. The decomposition is done in the following way:



The element type `CCI_ELEM_HEXAGON` is divided into two quadrilaterals for the interpolation. These quadrilaterals are checked like it is described above. The decomposition is done in the following way:



References

- Adamidis, P., A. Beck, U. Becker-Lemgau, Y. Ding, M. Franzke, H. Holthoff, M. Laux, A. Müller, M. Münch, A. Reuter, B. Steckel, and R. Tilch, 1998: Steel Strip Production - A Pilot Application for Coupled Simulation with Several Calculation Systems. *Journ. Materials Processing Technology*, 330 – 336.
- Ahrem, R., P. Post, and K. Wolf, 2000: A Communication Library to Couple Simulation Codes on Distributed Systems for Multi-Physics Computations. E. D’Hollander, G. Joubert, F. Peters, and H. Sips, eds., *Parallel Computing: Fundamentals and applications*, Proceedings of the international conference ParCO 99, 47 – 55, Imperial College Press.
- Asano, T., M. Edahiro, H. Imai, M. Iri, and K. Murota, 1985: Practical Use of Bucketing Techniques in Computational Geometry, Vol. 2. G. T. Toussaint, ed., *Computational Geometry*, Machine Intelligence and Pattern Recognition, 153 – 195, North-Holland, Amsterdam.
- Becker-Lemgau, U., M. Hackenberg, W. Joppich, S. Mijaković, B. Steckel, T. Sontowski, and R. Tilch, 1999: Solution of Coupled Problems by Parallel Multigrid. H.-J. Bungartz, F. Durst, and C. Zenger, eds., *Lecture Notes in Computational Science and Engineering: High Performance Scientific and Engineering Computing*, 91 – 101, Springer Verlag.
- Becker-Lemgau, U., M. Hackenberg, B. Steckel, and R. Tilch, 1998a: Interpolation Management in the GRISSLi Coupling-Interface for Multidisciplinary Simulations. *Computational Fluid Dynamics '98*, 1266 – 1271, John Wiley & Sons.
- Becker-Lemgau, U., M. Hackenberg, B. Steckel, and R. Tilch, 1998b: Parallel Multigrid in the Simulation of Metal Flow. *Advances in Parallel Computing*, vol. 12, 195 – 202, Elsevier Science BV (NL).
- Benson, D. J. and J. O. Hallquist, 1990: A Single Surface Contact Algorithm for the Post-Buckling Analysis of Shell Structures. *Computer Methods in Applied Mechanics and Engineering*, **78**, 141 – 163.
- Brakkee, E., K. Wolf, D. P. Ho, and A. Schüller, 1997: The COupled COmmunications LIBrary. *Proceedings of the Fifth Euromicro Workshop on Parallel and Distributed Processing, London, UK, January 22-24*, 155 – 162, IEEE Computer Society Press, Los Alamitos, California.
- Brakkee, E., K. Wolf, D. P. Ho, and A. Schüller, 1998: Coupling Industrial Simulation Codes on Parallel Computers Using a Communications Library. *Advances in Parallel Computing, Vol. 12*, ParCo 97, 419 – 427, Elsevier Science B.V.
- Cebral, J. R. and R. Löhner, 1996: Conservative Load Projection and Tracking for Fluid-Structure Problems. AIAA Paper 96-0797.
- Ding, Y., A. Müller, M. Münch, and A. Reuter, 1997: The GRISSLi Coupling Library. *High Performance Computing on Hewlett-Packard Systems (HiPer'97)*, 171 – 180.
- Grandy, J., 1999: Conservative Remapping and Region Overlays by Intersecting Arbitrary Polyhedra. *Journal of Computational Physics*, **148**, 433 – 466.

- Gropp, W., E. Lusk, and A. Skjellum, 1994: *Using MPI: Portable Parallel Programming with the Message Passing Interface*. Scientific and Engineering Computation, MIT Press, Cambridge, Mass., The latest developments on MPI are found at <http://www.mcs.anl.gov/Projects/mpi/index.html> .
- Knuth, D., 1973: *Sorting and searching*. The Art of Computer Programming, Vol. 3, Addison-Wesley, Massachusetts.
- Kuo, Y., S.-Y. Hwang, and H. Hu, 1989: A Data Structure for Fast Region Searches. *IEEE Design & Test of Computers*, **6**(5), 20 – 28.
- Löhner, R., 1988: Some Useful Data Structures for the Generation of Unstructured Grids. *Applied Numerical Mathematics*, **4**, 123 – 135.
- Löhner, R., 1995: Robust, Vectorized Search Algorithms for Interpolation on Unstructured Grids. *Journal of Computational Physics*, **118**, 380 – 387.
- Löhner, R., C. Yang, J. Cebal, J. D. Baum, H. Luo, D. Pelessone, and C. Charman, 1995: Fluid-Structure Interaction Using a Loose Coupling Algorithm and Adaptive Unstructured Grids. AIAA Paper 95-2259.
- Maman, N. and C. Farhat, 1995: Matching Fluid and structure Meshes for Aeroelastic Computations. *Computers & Structures*, **54**(4), 779 – 785.
- Setrakian, A., G. Karagiorgis, H. Storm, J. Clinckenmaillie, P. Post, and S. Vlachoutsis, 2000: Open Interface for Coupling of Industrial Simulation Codes on Parallel Systems. T. Kvamsdal, I. Enevoldsen, K. Herfjord, C. Jenssen, K. Mehr, and S. P. Norsett, eds., *Computational Methods for Fluid-Structure Interaction: Proceedings from the International Symposium on Computational Methods for Fluid-Structure Interaction*, 99 – 107, Trondheim, Tapir.
- Snir, M., S. Otto, S. Huss-Lederman, D. Walker, and J. Dongarra, 1996: *MPI: The complete reference*. The MIT Press, Cambridge, Massachusetts, London, England.
- Stepanov, A. and M. Lee, 1994: The Standard Template Library. Technical Report HPL-94-34, Hewlett-Packard Laboratories, 1501 Page Mill Road, Palo Alto, CA 94304.
- Zhang, Z., 1993: *Finite Element Procedures for Contact-Impact problems*. Oxford University Press.

General Index

- addcontrol block, 98
- additional block, 117
- alg, 106
- angle, 103
- area, 103
- aspect_ratio, 103
- block
 - addcontrol, 98
 - additional, 117
 - code, 98
 - contact, 104
 - control, 101
 - coupling, 113
 - jobs, 109
 - parameters, 112
 - quantities, 100
 - switches, 106
- blocking, 21
- bucket search, 123
- bucket search algorithm, 122
- cart, 118
- cci_home_dir, 112
- CCI_Abort, **92**, 96
- CCI_ANY_CODE, 75
- CCI_ANY_QUANTITY, 44
- CCI_BOOL, 22
- CCI_Check_convergence, **75**, 95
- CCI_CLIENT_HOME, 20
- CCI_Close_remesh, **78**, 96
- CCI_Close_setup, **54**, 95
- CCI_Comm_info, **85**, 96
- CCI_COMM_RCODE, 44
- CCI_CONTINUE, 75
- CCI_CONVERGED, 75
- CCI_COORD_CART, 50
- CCI_COORD_CYL, 51
- CCI_COORD_SPH, 51
- CCI_Def_comm, **43**, 95
- CCI_Def_coord_transform, **50**, 95
- CCI_Def_elems, **34**, 95
- CCI_Def_mesh, **27**, 95
- CCI_Def_nodes, **31**, 95
- CCI_Def_partition, **29**, 95
- CCI_Def_search_tags, **49**, 95
- CCI_Def_sync_point, **46**, 95
- CCI_DIVERGED, 75
- CCI_DOUBLE, 22
- CCI_DOUBLE_PRECISION, 23
- CCI_ELEM_HEXAGON, 37
- CCI_ELEM_HEXAHEDRON, 37
- CCI_ELEM_LINE, 37
- CCI_ELEM_PENTAGON, 36
- CCI_ELEM_PRISM, 38
- CCI_ELEM_PYRAMID, 38
- CCI_ELEM_QUAD, 35
- CCI_ELEM_QUAD8, 35
- CCI_ELEM_QUAD9, 36
- CCI_ELEM_SPH_QUAD, 39
- CCI_ELEM_SPH_TRIANGLE, 38
- CCI_ELEM_TETRAHEDRON, 37
- CCI_ELEM_TRIANGLE, 35
- CCI_ELEM_TRIANGLE6, 36
- CCI_Finalize, **79**, 96
- CCI_FLOAT, 22
- CCI_General_info, **81**, 96
- CCI_Get_elems, **72**, 95
- CCI_Get_id_string, **87**, 96
- CCI_Get_intersection_points, **88**, 96
- CCI_Get_nodes, **69**, 95
- CCI_HOME, 112
- CCI_Init, **24**, 95
- CCI_Init_with_id_string, **26**, 95
- CCI_INT, 22
- CCI_INTEGER, 23
- CCI_Iprobe, **64**, 95
- CCI_Isend, **63**, 67, 95
- CCI_LOCALMESHID, 64, 65

- CCI_LOGICAL, 23
- CCI_LONG_DOUBLE, 22
- CCI_MATCHING_GRIDS, 82
- CCI_MATCHING_INFO, 81
- CCI_MAX_NQUANTITIES, 47, 64, 65, 84
- CCI_MAX_STRING_LEN, 80
- CCI_Mod_nodes, **77**, 96
- CCI_MPI_COMM.CCODE, 54
- CCI_MPI_COMM.CODE, 24
- CCI_MPI_COMM.CWORLD, 54
- CCI_MPI_COMM.WORLD, 24
- CCI_MPI_ICOMM.CCODE, 54
- CCI_MPI_ICOMM.CODE, 25
- CCI_MYCODEID, 24, 44
- CCI_NCODES, 24, 44
- CCI_NON_MATCHING_GRIDS, 82
- CCI_NQUANTITIES, 64, 65
- CCI_Parameter.info, **80**, 96
- CCI_PATH, 119
- CCI_Put_elems, **58**, 95
- CCI_Put_nodes, **56**, 95
- CCI_QUANTITY_DIM, 81
- CCI_QUANTITY_INFO, 81
- CCI_QUANTITY_INFO_LENGTH, 81
- CCI_QUANTITY_IPOL, 82
- CCI_QUANTITY_IDS, 64, 65
- CCI_QUANTITY_LOCATION, 82
- CCI_QUANTITY_LOCATION_ELEM, 82
- CCI_QUANTITY_LOCATION_GLOBAL, 82
- CCI_QUANTITY_LOCATION_NODE, 82
- CCI_QUANTITY_TYPE, 82
- CCI_QUANTITY_TYPE_FIELD, 82
- CCI_QUANTITY_TYPE_FLUX, 82
- CCI_QUANTITY_TYPE_MESH, 82
- CCI_QUANTITY_TYPE_USERDEF, 82
- CCI_Reach_sync_point, **68**, 95
- CCI_REAL, 23
- CCI_REAL8, 23
- CCI_REAL4, 23
- CCI_Recv, **65**, 95
- CCI_Remesh, **76**, 96
- CCI_REQUEST_NULL, 67

- CCI_Send, **61**, 95
- cciserver, 18
- CCI_Status, 22
- CCI_STATUS_SIZE, 22
- CCI_STOP, 75
- CCI_STRING, 22
- CCI.Sync_point.info, **83**, 96
- CCI_TOTAL_REMESH, 78
- CCI.Wait, **67**, 95
- CCI.Wtick, **93**, 96
- CCI.Wtime, **94**, 96
- check_mesh_quality, 102
- close_after_writing, 102
- code block, 98
- collective, 21
- comm, 116
- communicator, 12, 43, 113
- contact block, 104
- contact_detection, 104
- control block, 101
- coord_system, 118
- coordinate transformation, 118
- coupling block, 113
- coupling server, 18
- coupling steps, 75
- coupling_dim, 105
- cyl, 118
- data types, 22
- debug_level, 107
- default_alg, 105
- EE, 121
- element types, 141
- element-elements relationship, 121
- env, 99
- equals_zero, 118
- exec, 99
- field, 99
- field interpolation, 134
- flux, 99
- flux interpolation, 135

- format, 102
- full overlap, 105
- geometric relationships, 120
- global, 99
- global_dim, 105, 118
- grid type, 105
- group switches, 107
- HDF, 101
- ierror, 22
- implicit_coupling_steps, 102
- include mechanism, 119
- input file, 97
- interpolation
 - bilinear, 133
 - fields, 134
 - flux, 135
- interpolation types, 134
- inters_points, 118
- intersection points, 118
- ipol, 99
- jobs block, 109
- libcciclient.a, 19
- loc, 99
- local, 21
- match_comm, 116
- match_syncpt, 114
- matching_criterion, 105
- max, 99
- mesh, 99
- mesh deformation, 140
- mesh quality
 - check, 103
 - nodes, 103
 - quadrilaterals, 103
 - quadrilaterals8, 103
 - quadrilaterals9, 103
 - triangles, 103
 - triangles6, 103

- mesh_pair, 106
- min, 99
- minimal_distance, 126
- non-blocking, 21
- non-local, 21
- non_orphaned_objects, 118
- nprocs, 110
- one-side collective, 21
- origin, 118
- output_level, 107
- overlap, 105
- parameters block, 112
- partial overlap, 105
- PE, 120
- point-element relationship, 120
- point-point relationship, 121
- post_exec, 99
- PP, 121
- pre-contact search, 122
- pre_contact_search, 105
- pre_exec, 99
- prod, 99
- pwd, 110
- Q, 99
- quantities block, 100
- quantity, 97
- quantity units, 99
- restart_file_access, 104
- restart_file_name, 104
- return status, 22
- return value, 22
- runtime information, 109
- search
 - bucket algorithm, 122
 - linear, 121
 - oct-tree, 122
 - vectorized advancing front, 122
- search algorithm, 121

- set communicators, 116
- SI, 99
- skew, 103
- sph, 118
- startup information, 99
- sum, 99
- switches block, 106
- synchronization point, 14, 46, 113
- syncpt, 114

- taper, 103
- timeout, 103
- timing, 109
- tolerance, 103
- trace_comm_values, 102
- trace_mesh_change_values, 102
- trace_mesh_values, 102
- tracefile, 75, 101
- two-side collective, 21
- type (contact), 105
- type (quantity), 99
- Types of coordinate systems, 50

- unit_vectors, 118
- user_performs_contact_search, 105
- userdef, 99

- warn_orphaned_points, 105
- warp, 103
- where, 99

List of MpCCI Functions

CCI_Abort, **92**
CCI_Check_convergence, **75**
CCI_Close_remesh, **78**
CCI_Close_setup, **54**
CCI_Comm_info, **85**
CCI_Def_comm, **43**
CCI_Def_coord_transform, **50**
CCI_Def_elems, **34**
CCI_Def_mesh, **27**
CCI_Def_nodes, **31**
CCI_Def_partition, **29**
CCI_Def_search_tags, **49**
CCI_Def_sync_point, **46**
CCI_Finalize, **79**
CCI_General_info, **81**
CCI_Get_elems, **72**
CCI_Get_id_string, **87**
CCI_Get_intersection_points, **88**
CCI_Get_nodes, **69**
CCI_Init, **24**
CCI_Init_with_id_string, **26**
CCI_Iprobe, **64**
CCI_Isend, **63**
CCI_Mod_nodes, **77**
CCI_Parameter_info, **80**
CCI_Put_elems, **58**
CCI_Put_nodes, **56**
CCI_Reach_sync_point, **68**
CCI_Recv, **65**
CCI_Remesh, **76**
CCI_Send, **61**
CCI_Sync_point_info, **83**
CCI_Wait, **67**
CCI_Wtick, **93**
CCI_Wtime, **94**

List of MpCCI Constants

CCI_ANY_CODE, 75
CCI_ANY_QUANTITY, 44
CCI_BOOL, 22
CCI_COMM_RCODE, 44
CCI_CONTINUE, 75
CCI_CONVERGED, 75
CCI_COORD_CART, 50
CCI_COORD_CYL, 51
CCI_COORD_SPH, 51
CCI_DIVERGED, 75
CCI_DOUBLE, 22
CCI_DOUBLE_PRECISION, 23
CCI_ELEM_HEXAGON, 37
CCI_ELEM_HEXAHEDRON, 37
CCI_ELEM_LINE, 37
CCI_ELEM_PENTAGON, 36
CCI_ELEM_PRISM, 38
CCI_ELEM_PYRAMID, 38
CCI_ELEM_QUAD, 35
CCI_ELEM_QUAD8, 35
CCI_ELEM_QUAD9, 36
CCI_ELEM_SPH_QUAD, 39
CCI_ELEM_SPH_TRIANGLE, 38
CCI_ELEM_TETRAHEDRON, 37
CCI_ELEM_TRIANGLE, 35
CCI_ELEM_TRIANGLE6, 36
CCI_FLOAT, 22
CCI_INT, 22
CCI_INTEGER, 23
CCI_LOCALMESHID, 64, 65
CCI_LOGICAL, 23
CCI_LONG_DOUBLE, 22
CCI_MATCHING_GRIDS, 82
CCI_MATCHING_INFO, 81
CCI_MAX_NQUANTITIES, 47, 64, 65, 84
CCI_MAX_STRING_LEN, 80
CCI_MPI_COMM_CCODE, 54
CCI_MPI_COMM_CODE, 24
CCI_MPI_COMM_CWORLD, 54
CCI_MPI_COMM_WORLD, 24
CCI_MPI_ICOMM_CCODE, 54
CCI_MPI_ICOMM_CODE, 25
CCI_MYCODEID, 24, 44
CCI_NCODES, 24, 44
CCI_NON_MATCHING_GRIDS, 82
CCI_NQUANTITIES, 64, 65
CCI_QUANTITY_DIM, 81
CCI_QUANTITY_INFO, 81
CCI_QUANTITY_INFO_LENGTH, 81
CCI_QUANTITY_IPOL, 82
CCI_QUANTITYIDS, 64, 65
CCI_QUANTITY_LOCATION, 82
CCI_QUANTITY_LOCATION_ELEM, 82
CCI_QUANTITY_LOCATION_GLOBAL, 82
CCI_QUANTITY_LOCATION_NODE, 82
CCI_QUANTITY_TYPE, 82
CCI_QUANTITY_TYPE_FIELD, 82
CCI_QUANTITY_TYPE_FLUX, 82
CCI_QUANTITY_TYPE_MESH, 82
CCI_QUANTITY_TYPE_USERDEF, 82
CCI_REAL, 23
CCI_REAL8, 23
CCI_REAL4, 23
CCI_REQUEST_NULL, 67
CCI_STATUS_SIZE, 22
CCI_STOP, 75
CCI_STRING, 22
CCI_TOTAL_REMESH, 78